

# 算法设计与分析

北郊 HH

2025 年 5 月

## 目录

|          |                                |          |
|----------|--------------------------------|----------|
| <b>1</b> | <b>chapter1: 导论</b>            | <b>1</b> |
| 1.1      | 基本概念 . . . . .                 | 1        |
| 1.2      | 复杂度 . . . . .                  | 1        |
| 1.2.1    | $O$ 记号: 渐进上界记号 . . . . .       | 1        |
| 1.2.2    | $\Omega$ 记号: 渐进下界记号 . . . . .  | 1        |
| 1.2.3    | $\Theta$ -记号: 紧渐进界记号 . . . . . | 2        |
| 1.2.4    | $o$ -记号 . . . . .              | 2        |
| 1.2.5    | $\omega$ -记号: 非紧下界记号 . . . . . | 2        |
| 1.2.6    | 渐进记号的性质 . . . . .              | 2        |
| 1.3      | 平摊分析 . . . . .                 | 3        |
| 1.3.1    | 聚集分析 . . . . .                 | 3        |
| 1.3.2    | 记账法 . . . . .                  | 3        |
| 1.3.3    | 势能法 . . . . .                  | 3        |
| 1.4      | 递推关系 . . . . .                 | 3        |
| 1.4.1    | 主定理 . . . . .                  | 4        |
| 1.4.2    | Akra-Bazzi 定理 . . . . .        | 5        |
| 1.4.3    | 应用示例 . . . . .                 | 6        |
| 1.4.4    | 递推关系求解练习 . . . . .             | 7        |
| 1.5      | 本章作业 . . . . .                 | 7        |
| 1.5.1    | 势函数规范化构造及其平摊代价不变性证明 . . . . .  | 7        |
| 1.5.2    | 聚集法计算平摊代价 . . . . .            | 8        |
| 1.5.3    | 主定理运用 . . . . .                | 8        |
| <b>2</b> | <b>chapter2 归纳法</b>            | <b>9</b> |
| 2.1      | 归纳算法举例 . . . . .               | 9        |
| 2.1.1    | 多项式求值 . . . . .                | 9        |
| 2.1.2    | 生成排列 . . . . .                 | 9        |
| 2.1.3    | 基数排序 . . . . .                 | 10       |
| 2.1.4    | 寻找多数元素 . . . . .               | 11       |
| 2.1.5    | 棋子移动游戏 . . . . .               | 11       |

|          |                      |           |
|----------|----------------------|-----------|
| 2.2      | 本章习题                 | 11        |
| 2.2.1    | RADIXSORT 算法时间复杂性    | 11        |
| 2.2.2    | 寻找名人问题               | 12        |
| <b>3</b> | <b>chapter3 分治</b>   | <b>13</b> |
| 3.1      | 分治思想                 | 13        |
| 3.2      | 分治算法举例               | 14        |
| 3.2.1    | 归并排序                 | 14        |
| 3.2.2    | 选择问题: 寻找第 k 小元素      | 16        |
| 3.2.3    | 最值选择算法的改进            | 16        |
| 3.2.4    | 有序矩阵找第 k 小的数         | 18        |
| 3.2.5    | 多数元素问题               | 18        |
| 3.2.6    | 芯片测试问题               | 18        |
| 3.2.7    | 循环赛日程表               | 19        |
| 3.2.8    | 向量卷积                 | 20        |
| 3.2.9    | 棋盘覆盖                 | 21        |
| 3.2.10   | 最大子序列和问题             | 21        |
| 3.3      | 分治算法的改进              | 22        |
| 3.3.1    | 大整数乘法                | 22        |
| 3.3.2    | 矩阵乘法                 | 22        |
| 3.3.3    | 最近点对问题               | 23        |
| 3.4      | 本章作业                 | 23        |
| 3.4.1    | 快排空间复杂度              | 23        |
| 3.4.2    | 多选问题                 | 24        |
| <b>4</b> | <b>chapter4 动态规划</b> | <b>25</b> |
| 4.1      | 最长公共子序列              | 26        |
| 4.2      | 最长上升子序列              | 26        |
| 4.3      | 矩阵链乘法                | 27        |
| 4.4      | 多源最短路径问题             | 31        |
| 4.5      | 0/1 背包问题             | 31        |
| 4.6      | 图像压缩问题               | 32        |
| 4.7      | 最大子段和                | 34        |
| 4.8      | 多段图问题                | 34        |
| 4.9      | 最优二分检索树问题            | 35        |
| 4.10     | 投资问题                 | 36        |
| 4.11     | 本章作业                 | 37        |
| 4.11.1   | 改进 floyd 算法          | 37        |
| 4.11.2   | 有向图的所有点对最短路径问题       | 37        |
| 4.11.3   | k-shattering 问题      | 38        |

|          |                          |           |
|----------|--------------------------|-----------|
| <b>5</b> | <b>chapter5: 贪心算法</b>    | <b>39</b> |
| 5.1      | 贪心算法思想                   | 40        |
| 5.2      | 贪心算法举例                   | 40        |
| 5.2.1    | Dijkstra 算法              | 40        |
| 5.2.2    | Kruskal 算法               | 41        |
| 5.2.3    | Prim 算法                  | 42        |
| 5.2.4    | 活动选择问题                   | 43        |
| 5.2.5    | 区间图着色问题                  | 45        |
| 5.2.6    | 最小延迟调度问题                 | 46        |
| 5.2.7    | 分数背包问题                   | 47        |
| 5.2.8    | 带期限作业排序问题                | 47        |
| 5.2.9    | 黑白点最大匹配问题                | 50        |
| 5.2.10   | 过河问题                     | 51        |
| 5.2.11   | 树上的最小顶点覆盖                | 51        |
| <b>6</b> | <b>chapter6: 图的遍历</b>    | <b>51</b> |
| 6.1      | 割点                       | 51        |
| 6.2      | 强连通分量                    | 51        |
| 6.3      | A* 算法                    | 51        |
| <b>7</b> | <b>chapter7: NP 完全问题</b> | <b>51</b> |
| 7.1      | 计算复杂度的分析                 | 52        |
| 7.1.1    | 平凡下界                     | 52        |
| 7.1.2    | 计数所需最少运算数                | 52        |
| 7.1.3    | 决策树 (Decision Tree)      | 53        |
| 7.2      | 判定问题                     | 53        |
| 7.3      | P 类和 NP 类                | 54        |
| 7.3.1    | 图灵机模型                    | 54        |
| 7.3.2    | P 类问题                    | 55        |
| 7.3.3    | NP 类问题                   | 55        |
| 7.4      | 多项式时间规约和 NP 完全性          | 56        |
| 7.5      | NP 完全性                   | 57        |
| 7.6      | Cook-Levin 定理            | 57        |
| 7.6.1    | 限制法                      | 58        |
| 7.6.2    | 局部替换法                    | 58        |
| 7.6.3    | 构件设计法                    | 60        |
| 7.7      | 本章作业                     | 61        |
| 7.7.1    | 平面加权图的最小生成树问题的计算复杂度      | 61        |
| 7.7.2    | 元素唯一性问题                  | 62        |
| 7.7.3    | 2SAT 问题                  | 63        |
| 7.7.4    | 2 着色问题                   | 64        |
| 7.7.5    | 规约问题的传递性证明               | 65        |

|           |                         |           |
|-----------|-------------------------|-----------|
| 7.7.6     | 集合覆盖问题是 NPC 问题          | 66        |
| 7.7.7     | 0-1 背包问题                | 67        |
| 7.7.8     | 子例程时间影响                 | 67        |
| <b>8</b>  | <b>chapter8: 回溯法</b>    | <b>67</b> |
| 8.1       | 回溯法基本原理                 | 67        |
| 8.2       | 回溯法应用                   | 69        |
| 8.2.1     | 图的三着色问题                 | 69        |
| 8.2.2     | 流水作业调度问题                | 70        |
| <b>9</b>  | <b>chapter9: 分支限界法</b>  | <b>72</b> |
| 9.1       | 分支-限界法基本原理              | 72        |
| 9.2       | 分支限界法的应用                | 75        |
| 9.2.1     | Cost 函数设计方法             | 75        |
| 9.2.2     | 关键实现技术                  | 76        |
| 9.2.3     | TSP 问题中的 MST 下界         | 76        |
| 9.2.4     | 背包问题中的线性松弛              | 77        |
| <b>10</b> | <b>chapter10: 随机算法</b>  | <b>77</b> |
| 10.1      | 导论                      | 77        |
| 10.2      | 随机算法举例                  | 79        |
| 10.2.1    | 随机快速排序                  | 79        |
| 10.2.2    | 随机选择                    | 79        |
| 10.2.3    | 多选问题                    | 79        |
| 10.2.4    | 随机取样                    | 80        |
| 10.2.5    | 素数测试                    | 81        |
| 10.2.6    | 验证串的相等性                 | 82        |
| 10.2.7    | 模式匹配                    | 82        |
| 10.2.8    | 货郎担问题                   | 82        |
| 10.3      | 随机游走算法                  | 83        |
| 10.4      | 二元布尔可满足问题               | 84        |
| 10.5      | 本章作业                    | 85        |
| 10.5.1    | 优惠券收集                   | 85        |
| 10.5.2    | 用公平硬币随机序列生成             | 86        |
| 10.5.3    | 矩阵互逆                    | 87        |
| 10.5.4    | 三元可满足问题                 | 88        |
| 10.5.5    | 拉斯维加斯型与蒙特卡洛型的期望运行时间推导关系 | 88        |
| <b>11</b> | <b>chapter11: 近似算法</b>  | <b>89</b> |
| 11.1      | 导入                      | 89        |
| 11.1.1    | 组合优化问题                  | 89        |
| 11.1.2    | 近似算法                    | 89        |

|        |                 |     |
|--------|-----------------|-----|
| 11.1.3 | 装箱问题            | 90  |
| 11.1.4 | 差界              | 90  |
| 11.1.5 | 相对性能界           | 90  |
| 11.1.6 | 近似方案            | 91  |
| 11.1.7 | 可近似性分类          | 91  |
| 11.2   | 可近似算法举例         | 92  |
| 11.2.1 | 装箱问题            | 92  |
| 11.2.2 | ETSP 问题         | 93  |
| 11.2.3 | 最小顶点覆盖问题        | 94  |
| 11.2.4 | 多机调度问题          | 95  |
| 11.3   | 不可近似问题          | 96  |
| 11.4   | 完全可近似问题         | 97  |
| 11.5   | 课后作业            | 101 |
| 11.5.1 | 独立集问题           | 101 |
| 11.5.2 | 最大子集和问题         | 101 |
| 11.5.3 | 无向图最大割集问题       | 103 |
| 11.5.4 | 平面 TSP 问题的最小权匹配 | 103 |
| 11.5.5 | 最大团问题           | 105 |
| 11.5.6 | 证明 TSP 是不可近似问题  | 106 |
| 11.5.7 | 瓶颈旅行商问题         | 107 |
| 11.5.8 | 最优回路不会自我交叉      | 108 |
| 11.5.9 | TSP 不可近似性       | 108 |

# 1 chapter1: 导论

定理 1. *Stirling* 公式:

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n (1 + \Theta(\frac{1}{n}))$$

## 1.1 基本概念

算法是在有限的步骤内求解某一问题，所使用的一组定义明确的规则。也就是在通过一组有限定义的指令来完成一些任务，给定一个初始状态，并相应的获取最终结果。

算法应该具有以下特性：可行性、确定性（每一步都被精确定义，不存在模糊操作）、有限性（程序可以没有这个特性，操作系统就是一个不关机就无限期执行的程序）、输入、输出。

算法是解决问题的抽象步骤，程序是算法的具体实现，算法通过编程语言转为程序，程序在计算机上运行。

算法分析有事前分析与事后测试两种基本方法。

**定义 1.** 对于任何一个计算步骤，如果它的一次执行代价总是以一个时间常数为上界，与输入数据或者使用何种算法无关，则称该计算步骤为“元运算”或“元操作”。

**定义 2.** 算法中某个语句或运算的执行次数称为其频度。

**定义 3.** 如果算法中的一个元运算具有最高频度，所有其他元运算频度均在它的频度的常数倍内，则称这个元运算为基本运算。

## 1.2 复杂度

### 1.2.1 $O$ 记号：渐进上界记号

设  $f(n)$  和  $g(n)$  是从自然数到非负正数集上的两个函数。如果存在正常数  $c$  和自然数  $n_0$ ，使得

$$\forall n \geq n_0, f(n) \leq c \cdot g(n)$$

则称  $f(n)$  是  $O(g(n))$  的，记为  $f(n) = O(g(n))$ <sup>1</sup>。也可以这样理解，当  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$  存在且不为无穷，那么就有  $f(n) = O(g(n))$ ，（但是 ppt 上似乎还要求极限不为无穷，私以为这里把极限为无穷也算作了极限存在的一种）

### 1.2.2 $\Omega$ 记号：渐进下界记号

设  $f(n)$  和  $g(n)$  是从自然数到非负正数集上的两个函数，如果存在正常数  $c$  和自然数  $n_0$ ，使得

$$\forall n \geq n_0, f(n) \geq c \cdot g(n)$$

则称  $f(n)$  是  $\Omega(g(n))$  的，记为  $f(n) = \Omega(g(n))$ 。也可以这样理解，当  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$  存在且不为 0，则表示  $f(n) = \Omega(g(n))$

<sup>1</sup>这种“=”的记号的本意其实是  $f(n) \in O(g(n))$ ，而对于出现在等式或不等式中的渐进记号表示的也是，其中的某一函数

### 1.2.3 $\Theta$ -记号：紧渐进界记号

如果存在 2 个正常数  $c_1, c_2$  和自然数  $n_0$ ，使得

$$\forall n \geq n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

则称  $f(n)$  是  $\Theta(g(n))$ ，记为  $f(n) = \Theta(g(n))$ ，也就是说， $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c > 0$ ，则  $f(n) = \Theta(g(n))$ 。  
可以推断， $f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$ ，此时称  $f(n)$  与  $g(n)$  同阶，

### 1.2.4 $o$ -记号

对于任意给定的  $c > 0$ ，都存在正整数  $n_0$ ，使得当  $n \geq n_0$  时有

$$f(n) < c \cdot g(n)$$

则称函数  $f(n)$  是  $o(g(n))$  的，记为  $f(n) = o(g(n))$  也就是说  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$ ，则  $f(n) = o(g(n))$ 。  
因此可以有：

$$f(n) = o(g(n)) \iff f(n) = O(g(n)) \wedge g(n) \neq O(f(n))$$

当  $n$  充分大时， $f(n)$  的阶比  $g(n)$  低。

### 1.2.5 $\omega$ -记号：非紧下界记号

对于任意给定的  $c > 0$ ，都存在正整数  $n_0$ ，使得当  $n \geq n_0$  时有  $f(n) > c \cdot g(n)$ ，则称函数  $f(n)$  是  $\omega(g(n))$  的，记为  $f(n) = \omega(g(n))$ ，当  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$  不存在， $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \rightarrow \infty$ ， $f(n) = \omega(g(n))$ 。

### 1.2.6 渐进记号的性质

1. 传递性：所有符号
2. 反身性： $\Theta, O, \Omega$
3. 对称性： $\Theta$
4. 互对称性：

$$f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$$

$$f(n) = o(g(n)) \iff g(n) = \omega(f(n))$$

#### 5. 算数计算

- $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$
- $O(f(n)) + O(g(n)) = O(f(n) + g(n))$
- $O(f(n)) * O(g(n)) = O(f(n) * g(n))$
- $O(c \cdot f(n)) = O(f(n))$
- $g(n) = O(f(n)) \Rightarrow O(f(n)) + O(g(n)) = O(f(n))$

算法的空间复杂度是对一个算法在运行过程中临时使用存储空间的渐进度量

### 1.3 平摊分析

#### 1.3.1 聚集分析

将所有的操作加起来，然后除以操作数，得到的结果就是平摊代价。

#### 1.3.2 记账法

对不同操作赋予不同的费用，这个均摊成本  $\hat{c}_{op}$  可能与实际成本  $c_{op}$  不同，但必须要满足的是，对于任意的合法的  $n$  个操作序列，必须有  $T(n) = \sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i$ 。

这种方法更常见于对一个已知的均摊代价进行巧妙的证明。例如：对于二进制计数器算法，因为每一个操作 1 翻转为 0 中的 1，都一定来自于一个操作：0 翻转为 1，而最终结果一定会剩余一定量的 1，所以操作：0 翻 1 的数量大于 1 翻 0 的数量。

表 1: Binary Counter Flip Operations: Real Cost vs. Amortized Cost

| OP                       | Real Cost $C_{OP}$ | Amortized Cost $\hat{C}_{OP}$ |
|--------------------------|--------------------|-------------------------------|
| flip (0 $\rightarrow$ 1) | 1                  | 2                             |
| flip (1 $\rightarrow$ 0) | 1                  | 0                             |

$$\begin{aligned}
 T(n) &= \sum_{i=1}^n C_i \\
 &= \#flip(0 \rightarrow 1) + \#flip(1 \rightarrow 0) \\
 &\leq 2 \cdot \#flip(0 \rightarrow 1) \\
 &\leq 2n
 \end{aligned}$$

#### 1.3.3 势能法

势能函数充当能量存储罐的作用，当操作简单时就会势能增加（相当于存钱）；当操作复杂时就会势能减少（相当于取钱）；那么每次操作的平摊代价 = 实际操作成本 + 存储罐的变化量。即：

$$a_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

那么对于  $n$  个操作的总平摊代价为：

$$\sum a_i = \sum c_i + \Phi(D_n) - \Phi(D_0)$$

如果势函数使得对所有的  $n$ ，都有  $\Phi(D_n) \geq \Phi(D_0)$ ，则总平摊代价就是总实际代价的一个上界，为了方便起见有时会定义  $\Phi(D_0) = 0$ 。

### 1.4 递推关系

（高中部分内容不再展开）



## 1.4.1 主定理

**定理 2.** 设  $a \geq 1, b \geq 1$  为常数,  $f(n)$  为函数,  $T(n)$  为非负正数, 且

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

那么就有

- $f(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0$  那么  $T(n) = \Theta(n^{\log_b a})$
- $f(n) = \Theta(n^{\log_b a})$ , 那么  $T(n) = \Theta(n^{\log_b a} \log n)$
- $f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0$ , 且对某个常数  $c < 1$  和所有充分大的  $n$  有  $a \cdot f(\frac{n}{b}) \leq c \cdot f(n)$ , 那么  $T(n) = \Theta(f(n))$

证明. 设  $n = b^k$ , 则有

$$\begin{aligned} T(n) &= a^k T(1) + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right), T(1) = c_1 \end{aligned}$$

## • 情况 1

$$\begin{aligned} T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{(b^{\log_b a - \varepsilon})^j}\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} (b^\varepsilon)^j\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \cdot \frac{b^{\varepsilon \log_b n} - 1}{b^\varepsilon - 1}\right) \\ &= c_1 n^{\log_b a} + O(n^{\log_b a - \varepsilon} \cdot n^\varepsilon) \\ &= \Theta(n^{\log_b a}) \end{aligned}$$

## • 情况 2

$$\begin{aligned}
T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\
&= c_1 n^{\log_b a} + \Theta\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right) \\
&= c_1 n^{\log_b a} + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{a^j}\right) \\
&= c_1 n^{\log_b a} + \Theta(n^{\log_b a} \log n) \\
&= \Theta(n^{\log_b a} \log n)
\end{aligned}$$

• 情况 3

$$\begin{aligned}
T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\
&\leq c_1 n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} c^j f(n) \quad \left(af\left(\frac{n}{b}\right) \leq cf(n)\right) \\
&= c_1 n^{\log_b a} + f(n) \frac{c^{\log_b n} - 1}{c - 1} \\
&= c_1 n^{\log_b a} + \Theta(f(n)) \quad (c < 1) \\
&= \Theta(f(n)) \quad (f(n) = \Omega(n^{\log_b a + \varepsilon}))
\end{aligned}$$

□

对于  $T(n) = 2T(\frac{n}{2}) + n \log n$ , 就是不能使用主定理的, 因为不存在  $\varepsilon > 0$  使得  $n \log n = \Omega(n^{1+\varepsilon})$ . 这个可以用递归树进行求解, 恰好这个拆解得到的是平衡树, 只需要对  $f(n)$  项求和即可。

$$\begin{aligned}
T(n) &= \sum_{i=0}^{\log n} n(\log n - i) \\
&= n \log^2 n = O(n \log^2 n)
\end{aligned}$$

### 1.4.2 Akra-Bazzi 定理

Akra-Bazzi 定理是分析分治算法时间复杂度的强大工具, 它推广了经典的主定理 (Master Theorem), 适用于更广泛的递归关系。

对于递归关系式:

$$T(n) = \sum_{i=1}^k a_i T(b_i n + h_i(n)) + f(n)$$

其中:

- $a_i > 0$  且  $0 < b_i < 1$  (对所有  $i = 1, \dots, k$ )
- $|h_i(n)| = O\left(\frac{n}{\log^2 n}\right)$  (扰动项)

- $f(n)$  满足某些正则条件

定义  $p$  为方程的解:

$$\sum_{i=1}^k a_i b_i^p = 1$$

则时间复杂度为:

$$T(n) = \Theta \left( n^p \left( 1 + \int_1^n \frac{f(u)}{u^{p+1}} du \right) \right)$$

表 2: 主定理与 Akra-Bazzi 定理对比

| 特征   | Akra-Bazzi 定理  |
|------|----------------|
| 适用范围 | 非均匀划分的递归式      |
| 扰动项  | 允许 $h_i(n)$ 存在 |
| 驱动函数 | $f(n)$ 形式更灵活   |
| 解的形式 | 积分表达式          |

### 1.4.3 应用示例

分析递归式  $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + n$ :

1. 确定参数:  $a_1 = a_2 = 1, b_1 = b_2 = 1/2$
2. 解方程:  $2 \cdot (1/2)^p = 1 \Rightarrow p = 1$
3. 计算积分:  $\int_1^n \frac{u}{u^2} du = \ln n$
4. 最终解:  $T(n) = \Theta(n(1 + \ln n)) = \Theta(n \log n)$

当递归式满足主定理条件时, Akra-Bazzi 定理给出的结果与主定理一致, 但前者适用性更广。

不过其实用数学归纳法会更舒适一些。例如估计以下复杂度:

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \Theta(n)$$

假设对任意的  $m < n$ , 都有  $T(m) < cm$  恒成立, 其中  $c$  为常数, 下面证明  $T(n) < cn$  即可证明出  $T(n) = \Theta(n)$

$$\begin{aligned}
 T(n) &= T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \Theta(n) \\
 &< T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + dn \\
 &< c\frac{n}{5} + c\frac{7n}{10} + dn \\
 &= \left(\frac{c}{5} + \frac{7c}{10} + d\right)n < cn
 \end{aligned}$$

因而只需取  $c > 10d$  即可。

## 1.4.4 递推关系求解练习

$$1. T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + cn$$

$$\text{令 } n = 2^k, T(2^k) = T(2^{k-1}) + T(2^{k-2}) + c \cdot 2^k$$

构建递归树知，若设  $T(2^{k-i})$  的结点数量是  $f[i]$ ，则有  $f[i] = f[i-1] + f[i-2]$

$$\text{且 } f[0] = f[1] = 1, \text{ 因此叶子结点数量为 } f[k] = \frac{1}{\sqrt{5}} \left[ \left( \frac{1+\sqrt{5}}{2} \right)^{k+1} - \left( \frac{1-\sqrt{5}}{2} \right)^{k+1} \right]$$

$$\frac{1}{\sqrt{5}} \left[ \left( \frac{1+\sqrt{5}}{2} \right)^{k+1} - \left| \left( \frac{1-\sqrt{5}}{2} \right)^{k+1} \right| \right] \leq f[k] \leq \frac{1}{\sqrt{5}} \left( \frac{1+\sqrt{5}}{2} \right)^{k+1}$$

$$\therefore f[k] = O(a^k) = O(n^{\log_2 a}), \text{ 其中 } a = \frac{1+\sqrt{5}}{2}$$

$$\text{而对于计算耗时为: } cn \cdot \sum_{i=1}^n \left( \frac{3}{4} \right)^{i-1} = O(n) > O(n^{\log_2 a})$$

$$\therefore T(n) = O(n)$$

$$2. T(n) = T(n-1) + \log n$$

$$\text{原式} = T(1) + \sum_{i=2}^n \log i$$

$$\text{下证: } \sum_{i=2}^n \log i \text{ 与 } n \log n \text{ 同阶}$$

$$\text{由 Stolz 定理知, } \lim_{n \rightarrow \infty} \frac{\sum_{i=2}^n \log i}{n \log n} = \lim_{n \rightarrow \infty} \frac{\log n}{n \log n - (n-1) \log(n-1)} = \lim_{n \rightarrow \infty} \frac{\log n}{\log(n-1)} = 1$$

$$\therefore T(n) = \Theta(n \log n)$$

## 1.5 本章作业

## 1.5.1 势函数规范化构造及其平摊代价不变性证明

假定有势函数  $\Phi$ ，对所有  $i \geq 1$  满足  $\Phi(D_i) \geq \Phi(D_0)$ ，但  $\Phi(D_0) \neq 0$ 。证明：存在势函数  $\Phi'$ ，使得  $\Phi'(D_0) = 0$ ，对于所有  $i \geq 1$ ，满足  $\Phi'(D_i) \geq 0$ ，且使用  $\Phi'$  的平摊代价与使用  $\Phi$  的平摊代价相同。

证明. 设新的势函数为：

$$\Phi'(D_i) = \Phi(D_i) - \Phi(D_0) \quad (i = 0, 1, 2, \dots)$$

## 1. 零势初始化：

$$\Phi'(D_0) = \Phi(D_0) - \Phi(D_0) = 0$$

2. 非负性保持 ( $\forall i \geq 1$ ):

$$\Phi'(D_i) = \Phi(D_i) - \Phi(D_0) \geq 0 \quad (\text{由已知条件 } \Phi(D_i) \geq \Phi(D_0))$$

3. 平摊代价不变性：对于任意操作，设实际代价为  $c_i$ ，有：

$$\begin{aligned}\hat{c}'_i &= c_i + \Phi'(D_i) - \Phi'(D_{i-1}) \\ &= c_i + (\Phi(D_i) - \Phi(D_0)) - (\Phi(D_{i-1}) - \Phi(D_0)) \\ &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= \hat{c}_i\end{aligned}$$

□

### 1.5.2 聚集法计算平摊代价

假定对一个数据结构执行一个由  $n$  个操作组成的操作序列，当  $i$  严格为 2 的幂时，第  $i$  个操作的代价为  $i$ ，否则代价为 1。请使用聚集法确定每个操作的平摊代价。

考虑  $n$  个操作序列的总代价：

$$T(n) = \sum_{\substack{k=0 \\ 2^k \leq n}}^{\lfloor \log_2 n \rfloor} 2^k + (n - \lfloor \log_2 n \rfloor - 1)$$

展开后可得：

$$T(n) = (2^{\lfloor \log_2 n \rfloor + 1} - 1) + (n - \lfloor \log_2 n \rfloor - 1) \leq 3n$$

因此平摊代价为：

$$\frac{T(n)}{n} \leq 3 = O(1)$$

### 1.5.3 主定理运用

设递推方程  $T(n) = 7T(\frac{n}{2}) + n^2$  给出了算法  $A$  在最坏情况下的时间复杂度函数，算法  $B$  在最坏情况下的时间复杂度函数  $W(n)$  满足递推方程  $W(n) = aW(\frac{n}{4}) + n^2$ 。试确定最大的正整数  $a$ ，使得  $W(n)$  的阶低于  $T(n)$  的阶。

$$\because T(n) = 7T\left(\frac{n}{2}\right) + n^2$$

$$\therefore a = 7, b = 2, f(n) = n^2$$

$$\therefore n^{\log_b a} = n^{\log_2 7}, f(n) = n^2 = O(n^{\log_2 7 - \varepsilon}), \varepsilon > 0$$

$$\therefore T(n) = \Theta(n^{\log_2 7})$$

$$\text{又} \because W(n) = aW\left(\frac{n}{4}\right) + n^2$$

$$\therefore n^{\log_b a} = n^{\log_4 a} = n^{\frac{1}{2} \log_2 a}, f'(n) = n^2$$

$$\text{if } a = 16, f'(n) = \Theta(n^{\log_4 16}), W(n) = \Theta(n^2 \log n) < \Theta(n^{\log_2 7}), \text{ 满足题意}$$

$$\text{if } a > 16, f'(n) = O(n^{\log_4 a - \varepsilon}), \varepsilon > 0, W(n) = \Theta(n^{\log_4 a}) < \Theta(n^{\log_2 7})$$

$$\therefore \sqrt{a} < 7, a < 49$$

$$\therefore a_{\max} = 48 > 16$$

$$\therefore a_{\max} = 48$$

注意  $a = 16$  的讨论是必要的。

## 2 chapter2 归纳法

### 2.1 归纳算法举例

#### 2.1.1 多项式求值

假设有  $n + 2$  个实数  $a_0, a_1, \dots, a_n$  和  $x$  的序列, 现在要对以下多项式求值:

$$P_n(x) = \sum_{i=0}^n a_i x^i$$

我们发现:

$$p_n(x) = (((((a_n x + a_{n-1})x + a_{n-2})x + a_{n-3}) \dots))x + a_0$$

所以, 我们得到具体表达式为:

$$p_{n-1}(x) = a_n x^{n-1} + a_{n-1} x^{n-2} + \dots + a_2 x + a_1$$

所以, 有递推关系:

$$p_j(x) = x p_{j-1}(x) + a_{n-j}$$

**Input:**  $n + 2$  个实数  $a_0, a_1, \dots, a_n$  和  $x$  的序列

**Output:** 表达式  $P_n(x)$  的值

```

1  $p \leftarrow a_n$ 
2 for  $j \leftarrow 1$  to  $n$  do
3    $p \leftarrow xp + a_{n-j}$ 
4 end
5 return  $p$ 
```

**Algorithm 1:** HORNER

#### 2.1.2 生成排列

显然, 对于重复次数  $f(n)$  有以下递推关系:

$$f(n) = n f(n-1) + n, n \geq 2$$

令  $f(n) = n! h(n)$ , 那么

$$n! h(n) = n(n-1)! h(n-1) + n$$

即

$$h(n) = h(n-1) + \frac{1}{(n-1)!}$$

解得

$$h(n) = h(1) + \sum_{i=1}^{n-1} \frac{1}{i!} < \sum_{i=1}^{\infty} \frac{1}{i!} = e - 1$$

所以

$$f(n) = n! h(n) < (e-1)n! < 2 \cdot n!$$

```

Input: 正整数  $n$ 
Output: 所有长度为  $n$  的排列

1 for  $j \leftarrow 1$  to  $n$  do
2    $P[j] \leftarrow j$ 
3 end
4 perm1(1)
5 perm1(m)
6 if  $m = n$  then
7   return P // output P
8 else
9   for  $j \leftarrow m$  to  $n$  do
10    swap( $P[j], P[m]$ )
11    perm1( $m+1$ )
12    swap( $P[j], P[m]$ ) //  $P[m..n] = m, m+1, \dots, n$ 
13  end
14 end

```

Algorithm 2: PERMUTATIONS1

而输出语句的复杂度是  $\Theta(n)$ ，所以最终  $T(n) = \Theta(n \cdot n!)$

这里用到的思想其实是一种累加法的变体。也就是考虑递推式

$$f(n) = g(n)f(n-1) + h(n)$$

只需进行合适的换元，令  $p(n)$  满足：

$$f(n) = \left( \prod_{i=1}^n g(i) \right) \cdot p(n)$$

这样得到：

$$\left( \prod_{i=1}^n g(i) \right) \cdot p(n) = \left( \prod_{i=1}^n g(i) \right) \cdot p(n-1) + h(n)$$

$$p(n) = p(n-1) + \frac{1}{\prod_{i=1}^n g(i)} h(n)$$

$$p(n) = p(1) + \sum_{i=1}^n \frac{1}{\prod_{j=1}^i g(j)} h(i)$$

由此，即可求出  $f(n)$ ，不过要注意无穷级数的作用。

### 2.1.3 基数排序

令  $L = \{a_1, a_2, \dots, a_n\}$  是一张有  $n$  个数的集合，其中每个数恰好有  $k$  位数字，就是说每个数具有  $d_k d_{k-1} \dots d_1$  形式，这里  $d_i$  是 0 到 9 中的一个数字。

我们研究一种排序算法对  $L$  进行非降序排列，使算法在几乎所有的实际应用中都以线性时间运行。

在这个问题中，我们对整数的大小  $k$  施行归纳，而不是对数  $n$  这个对象来用归纳法。

时间复杂度： $\Theta(kn)$ ，空间复杂度： $\Theta(10n)$

**Input:** 一个有  $n$  个数的表  $L = \{a_1, \dots, a_n\}$  和  $k$  位数字

**Output:** 按非非降序排列的表  $L$

```

1 for  $j = 1$  to  $k$  do
2   准备 10 个空表  $L_0, L_1, \dots, L_9$  while  $L \neq \emptyset$  do
3      $a \leftarrow L.top()$ 
4      $L.pop()$ 
5      $i \leftarrow a$  的第  $j$  位数字, 将  $a$  加入表  $L_i$  中。// 取个位为第 1 位数字
6   end
7    $L \leftarrow L_0$ 
8   for  $i = 1$  to 9 do
9      $L \leftarrow L \cup L_i$ 
10  end
11 end
12 return  $L$ 

```

**Algorithm 3:** 基数排序

### 2.1.4 寻找多数元素

令  $A[1..n]$  是一个整数序列,  $A$  中的整数  $a$  如果在  $A$  中出现的次数多于  $\lfloor n/2 \rfloor$ , 那么  $a$  称为多数元素。

- 初始, 令候选元素  $c = A[1]$ , 并将计数器置 1
- 从  $A[2]$  开始, 逐个扫描元素, 如果被扫描的元素和  $c$  相等, 则计数器加 1; 如果元素不等于  $c$ , 则计数器减 1
- 如果所有的元素都已经扫描完毕并且计数器大于 0, 那么返回  $c$  作为多数元素的候选者
- 如果在  $c$  和  $A[j]$  ( $1 < j < n$ ) 比较时计数器为 0, 那么对于  $A[j+1..n]$  上的元素递归调用 candidate 过程。

时间复杂度  $\Theta(n)$ , 空间复杂度  $\Theta(1)$

### 2.1.5 棋子移动游戏

洛谷题: <https://www.luogu.com.cn/problem/P1259>

对于最后处理的 4 对棋子是需要特殊对待的, 不然会在 2 白 2 空 2 黑的时候卡死。

## 2.2 本章习题

### 2.2.1 RADIXSORT 算法时间复杂性

当输入由下列区间中的  $n$  个正整数组成时, 以  $n$  为大小说明算法 RADIXSORT 的时间复杂性。

1.  $[1 \dots n]$



```

Input: 一个数组  $A$  和整数  $m, n$ 
Output: 数组  $A$  中的一个候选者
1  $j \leftarrow m; c \leftarrow A[m]; count \leftarrow 1;$ 
2 while  $j < n$  and  $count > 0$  do
3    $j \leftarrow j + 1;$ 
4   if  $A[j] = c$  then
5      $count \leftarrow count + 1;$ 
6   else
7      $count \leftarrow count - 1;$ 
8   end
9 end
10 if  $j = n$  then
11   return  $c$  // 获取候选者
12 else
13   return  $candidate(j + 1)$ 
14 end

```

Algorithm 4: candidate( $m$ )2.  $[1 \dots n^2]$ 3.  $[1 \dots 2^n]$ 

### 2.2.2 寻找名人问题

在一个房间里有  $n$  个人，其中一个名人。所谓名人，就是大家都认识他，但是他却不认识任何其他人。名人之外的其他人可能认识房间里另外的一部分人。你可以问房间里任何人问题，但是问题只能是：“你认识某某吗？”，对方回答“Yes”或者“No”。请设计一个时间复杂度为  $\Theta(n)$  的归纳算法，通过问问题，把房间中的名人找出来。

1. 初始化候选人：假设第一个人为候选名人  $candidate = 0$ 。
2. 遍历排除非名人：
3. 从第 2 个人到第  $n$  个人，依次与候选人比较：
  - 若候选人认识当前人，则候选人不是名人，更新候选人为当前人。
  - 若候选人不认识当前人，则当前人不是名人，保留候选人。
4. 验证候选人合法性：
  - 检查是否所有人认识候选人。
  - 检查候选人是否不认识任何人。

**Input:**  $n$  个元素的数组  $A[1..n]$

**Output:** 若存在则输出多数元素，否则输出 none

```

1  $c \leftarrow \text{candidate}(1)$ ;
2 // 获取候选者
3  $count \leftarrow 0$ ;
4 for  $j \leftarrow 1$  to  $n$  do
5   if  $A[j] = c$  then
6      $count \leftarrow count + 1$ ;
7     // 候选者进行计数
8   end
9 end
10 if  $count > \lfloor n/2 \rfloor$  then
11   return  $c$ ;
12 end
13 else
14   return none;
15 end

```

Algorithm 5: MAJORITY

### 3 chapter3 分治

把规模大的问题分解成小规模问题，要求子问题**互相独立**且**与原问题形式相同**，递归解决这些子问题，再把他们的解合并到原问题中，这样的策略称为**分治法**。

归纳法是**尾递归**思想，通过扩展子问题解的方式得到原问题的解；而分治法是划分的思想，再合并得到原问题的解。

#### 3.1 分治思想

分治范式通常由以下的步骤组成：

- **划分步骤**。在算法的这个步骤中，输入被分成  $p \geq 1$  个部分，每个子实例的规模严格小于  $n$ ， $n$  是原始实例的规模。尽管比 2 大的其他小的常数很普遍，但  $p$  的最常用的值是 2。 $p$  也可以具有与  $\log n$ ， $n^\varepsilon$  一样高的数量级 ( $0 < \varepsilon < 1$ )。
- **治理步骤**。如果问题的规模大于某个预定的阈值  $n_0$ ，这个步骤由执行  $p$  个递归调用组成，阈值是由算法的数学分析导出的。
- **组合步骤**。这个步骤是组合  $p$  个递归调用的解来得到期望的输出值。

分治法所能解决的问题一般具有以下几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题，即具有最优子结构性（问题的最优解所包含的子问题的解也是最优的）；

表 3: 基数排序 (RADIXSORT) 的时间复杂性分析

| 输入区间            | 时间复杂度            | 解释                                                                  |
|-----------------|------------------|---------------------------------------------------------------------|
| $[1 \dots n]$   | $O(n)$           | 最大位数 $d = \lceil \log_{10} n \rceil \approx O(1)$ , 视为常数, 故线性时间。    |
| $[1 \dots n^2]$ | $O(n^2)$         | 位数 $d = \lceil 2 \log_{10} n \rceil$ , 但数字范围扩大, 近似为 $d = O(n)$ 。    |
| $[1 \dots 2^n]$ | $O(n \cdot 2^n)$ | 位数 $d = \lceil n \log_{10} 2 \rceil = \Theta(n)$ , 且输入规模为 $2^n$ 量级。 |

- 利用该问题分解出的子问题的解可以合并为该问题的解;
- 该问题所分解出的各个子问题是相互独立的, 即子问题之间不包含公共的子问题。

## 3.2 分治算法举例

### 3.2.1 归并排序

```

Input:  $A[1 \dots n]$ 
Output: 非降序的  $A[1 \dots n]$ 

1  $t \leftarrow 1$ 
2 while  $t < n$  do
3    $s \leftarrow t; t \leftarrow 2s; i \leftarrow 0$ 
4   while  $i + t \leq n$  do
5      $\text{MERGE}(A, i + 1, i + s, i + t)$ 
6      $i \leftarrow i + t$ 
7   end
8   if  $i + s < n$  then
9      $\text{MERGE}(A, i + 1, i + s, n)$ 
10  end
11 end

```

Algorithm 6: BOTTOMUP-MERGESORT

如果  $n$  是 2 的幂时, 两个算法的执行比较次数相同。合并两个排好序的子数组, 所需要的元素比较次数在  $\frac{n}{2}$  和  $n - 1$  之间, 所以最小比较次数是:

$$c(n) = 2c\left(\frac{n}{2}\right) + \frac{n}{2}, n \geq 2$$

最多比较次数:

$$c(n) = 2c\left(\frac{n}{2}\right) + n - 1, n \geq 2$$

由主定理知,  $c(n) = \Theta(n \log n)$  如果  $n$  是任意的正整数, 则算法执行元素比较次数是:

$$c(n) = c(\lfloor n/2 \rfloor) + c(\lceil n/2 \rceil) + bn, n \geq 2$$

**Input:**  $A[1..n]$ **Output:** 非降序的  $A[1..n]$ 

```

1 if  $low < high$  then
2    $mid \leftarrow \lfloor (low + high)/2 \rfloor$ 
3   mergesort( $A, low, mid$ )
4   mergesort( $A, mid + 1, high$ )
5   MERGE( $A, low, mid, high$ )
6 end

```

**Algorithm 7:** MERGESORT**Input** : An array  $A$  of  $n$  elements, a number  $k$  such that  $1 \leq k \leq n$ **Output:** The  $k$ th smallest element in  $A$ 

```

1 select( $A, 1, n, k$ )
2  $p \leftarrow high - low + 1$ 
3 if  $p < 4$  then
4   | sort  $A$  and return  $A[k]$ 
5 end
6 let  $q = \lfloor p/5 \rfloor$ . Divide  $A$  into  $q$  groups of 5 elements each. If 5 doesn't divide  $p$ , then discard the
   remaining elements.
7 // Although some elements are discarded here, the subsequent grouping still
   takes them into account, so it will not affect the answer.
8 Sort each of the  $q$  groups individually and extract its median element. Let the set of medians be
    $M$ 
9  $mm \leftarrow \text{select}(M, 1, q, \lfloor q/2 \rfloor)$  // median of medians
10 Partition  $A$  into three arrays:
11  $A_1 \leftarrow \{A | A < mm\}$ ,  $A_2 \leftarrow \{A | A = mm\}$ ,  $A_3 \leftarrow \{A | A > mm\}$ 
12 if  $|A_1| \geq k$  then
13   | return select( $A_1, 1, |A_1|, k$ )
14 else
15   | if  $k \leq |A_1| + |A_2|$  then
16     | return  $mm$ 
17   | else
18     | return select( $A_3, 1, |A_3|, k - |A_1| - |A_2|$ )
19   | end
20 end

```

**Algorithm 8:** SELECT algorithm

时间复杂度是  $\Theta(n \log n)$ ，空间复杂度是  $\Theta(n)$ （开出一个临时数组就够用）。

### 3.2.2 选择问题：寻找第 $k$ 小元素

设  $A'_1$  表示小于等于  $mm$  的元素集,  $A'_3$  表示大于等于  $mm$  的元素集。所以:

$$|A'_1| \geq 3 \lceil \frac{\lfloor n/5 \rfloor}{2} \rceil \geq \frac{3}{2} \lfloor \frac{n}{5} \rfloor$$

$$|A'_3| \leq n - \frac{3}{2} \lfloor \frac{n}{5} \rfloor \leq n - \frac{3}{2} (\frac{n-4}{5}) = n - 0.3n + 1.2 = 0.7n + 1.2$$

令  $0.7n + 1.2 \leq \lfloor 0.75n \rfloor$ , 解得  $n \geq 44$  (这里的  $\frac{3}{4}$  选的很好, 不到  $\frac{4}{5}$ , 又超过  $\frac{2}{3}$ )  
所以有:

$$T(n) \leq \begin{cases} c & \text{if } n < 44 \\ T(\lfloor \frac{n}{5} \rfloor) + T(\lfloor \frac{3n}{4} \rfloor) + cn & \text{if } n \geq 44 \end{cases}$$

因而  $T(n) \leq \frac{cn}{1 - \frac{1}{5} - \frac{3}{4}} = 20cn$ 。

对于代码实现, 我们可以对快排进行改造:

```

1 void qsort(int l, int r){
2     // if(l >= r) return;
3     // 这个必须要注意掉, 否则就可能不输出了
4     int i = l, j = r, x = a[(l + r) >> 1];
5     while(i <= j){
6         while(a[i] < x) i++;
7         while(a[j] > x) j--;
8         if(i <= j){
9             swap(a[i], a[j]);
10            i++; j--;
11        }
12    }
13    if(k <= j) qsort(l, j);
14    else if(k >= i) qsort(i, r);
15    else{
16        printf("%d\n", a[j + 1]);
17        exit(0);
18    }
19 }
```

快排是需要  $O(n)$  的空间来存储递归中的左右边界的。运行时间最坏情况下为  $\Theta(n^2)$ , 整体复杂度为  $\Theta(n \log n)$ 。

### 3.2.3 最值选择算法的改进

**原始 MINMAX 算法** 原始算法要求输入规模  $n$  必须为 2 的幂次 ( $n = 2^k$ ), 其伪代码如下:

```

1 Function MinMax(A, low, high)
2   mid  $\leftarrow \lfloor (low + high)/2 \rfloor$ ;
3   if high - low = 2 then
4     return (A[low], A[high]) if A[low] < A[high] else (A[high], A[low]);
5   end
6   (x1, y1)  $\leftarrow$  MinMax(A, low, mid);
7   (x2, y2)  $\leftarrow$  MinMax(A, mid, high);
8   return (min(x1, x2), max(y1, y2));

```

**Algorithm 9:** Original MinMax Algorithm (Requires  $n = 2^k$ )

时间复杂度分析:

- 比较次数递推式:

$$C(n) = \begin{cases} 1 & n = 2 \\ 2C(n/2) + 2 & n > 2 \end{cases}$$

- 精确解:  $C(n) = \frac{3n}{2} - 2$

**改进后的 MINMAX 算法** 改进算法通过以下修改支持任意正整数  $n$ :

```

1 Function MinMax(A, low, high)
2   mid  $\leftarrow \lfloor (low + high)/2 \rfloor$ ;
3   if high - low = 1 then
4     return (A[low], A[low]);
5   else if high - low = 2 then
6     return (A[low], A[high]) if A[low] < A[high] else (A[high], A[low]);
7   (x1, y1)  $\leftarrow$  MinMax(A, low, mid);
8   (x2, y2)  $\leftarrow$  MinMax(A, mid, high);
9   return (min(x1, x2), max(y1, y2));

```

**Algorithm 10:** Modified MinMax Algorithm (Works for any  $n \in \mathbb{N}^+$ )

改进要点分析

- **新增递归基:** 处理  $n = 1$  的情况 (第 3-4 行)
- **非对称划分:** 采用  $\lfloor n/2 \rfloor$  和  $\lceil n/2 \rceil$  划分
- **通用性:** 支持任意正整数输入规模

**比较次数分析** 改进后的比较次数满足递推关系:

$$T(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 2 & n > 2 \end{cases}$$

### 理论结论

- 改进算法比较次数趋近于  $\lceil \frac{3n}{2} \rceil - 2$  但非严格相等
- 当  $n = 2^k$  时与原始算法完全一致
- 最坏情况下比较次数上界为  $3\lceil n/2 \rceil - 2$

#### 3.2.4 有序矩阵找第 $k$ 小的数

给定一个  $n \times n$  的矩阵，其中每一行和每一列的元素均按升序排列。需要找到矩阵中第  $k$  小的元素。

1. 二分查找区间：初始区间为矩阵的最小值和最大值：

$$l = \text{matrix}[0][0], \quad r = \text{matrix}[n-1][n-1]$$

2. 统计小于等于  $\text{mid}$  的元素个数：

- 对于中点  $\text{mid}$ ，统计矩阵中  $\leq \text{mid}$  的元素个数  $\text{count}$
- 利用每行有序的性质，在每行中用二分查找（`upper_bound`）统计

3. 调整区间：

$$\text{count} < k \Rightarrow l = \text{mid} + 1$$

$$\text{count} \geq k \Rightarrow r = \text{mid} - 1$$

4. 终止条件：当  $l > r$  时， $l$  即为第  $k$  小的元素

- 时间复杂度：  $O(n \log n \log C)$ 
  - 二分查找次数：  $O(\log C)$  ( $C = \max - \min$ )
  - 每次统计  $\text{count}$ ：  $O(n \log n)$
- 空间复杂度：  $O(1)$

#### 3.2.5 多数元素问题

根据多数元素的定义可知多数元素必定在数组中出现次数最多，而若将原数组分成 2 部分，则在其中一部分出现次数最多的必定是多数元素。

该算法的时间复杂度是  $O(n \log n)$ ，空间复杂度是  $O(\log n)$ 。

#### 3.2.6 芯片测试问题

问题描述：有  $n$  片芯片，已知其中好芯片至少比坏芯片多 1 片，需要通过测试从中找出 1 片好芯片。如何设计一个算法，使用最少测试次数，从中挑出 1 片好芯片？

测试方法：将芯片放到测试台上，2 片芯片互相测试，并报告结果“好”或“坏”。好芯片报告结果一定是正确的，坏芯片的报告不可靠（可能对也可能不对）。

如果  $n$  是偶数，那么将  $n$  片芯片两两一组做淘汰测试，剩下芯片构成子问题，进入下一轮分组淘汰。

```

Input: A sorted  $n \times n$  matrix where each row and column is sorted in ascending order, and an integer  $k$ 
Output: The  $k$ -th smallest element in the matrix
1  $n \leftarrow \text{len}(\text{matrix});$ 
2  $l \leftarrow \text{matrix}[0][0];$ 
3  $r \leftarrow \text{matrix}[n-1][n-1];$ 
4 while  $l \leq r$  do
5      $\text{mid} \leftarrow l + \lfloor \frac{r-l}{2} \rfloor;$ 
6      $\text{count} \leftarrow 0;$ 
7     for  $i \leftarrow 0$  to  $n-1$  do
8          $\text{count} \leftarrow \text{count} + \text{bisect\_right}(\text{matrix}[i], \text{mid});$ 
9     end
10    if  $\text{count} < k$  then
11         $l \leftarrow \text{mid} + 1;$  // 第  $k$  小元素在右半区间
12    else
13         $r \leftarrow \text{mid} - 1;$  // 第  $k$  小元素在左半区间
14    end
15 end
16 return  $l;$ 

```

**Algorithm 11:** Find K-th Smallest Element in Sorted Matrix

淘汰规则：如果报告结果为“好，好”，则任留 1 片，否则全部抛弃，对于  $n = 3$  单独测试，对于  $n = 2, n = 1$  的情况，则任取其一就是好芯片。

要证明分治算法的正确性，只需要证明子问题与愿问题性质相同且算法是有退出的出口的。

如果  $n$  是奇数，则任选 1 个芯片，用其他  $n-1$  个芯片对他进行测试，若是好芯片，则算法退出，否则丢弃他，转化为是偶数个的情况。

设  $W(n)$  是最坏情况下的测试次数，那么有：

$$W(n) = W\left(\frac{n}{2}\right) + O(n)$$

由主定理知， $W(n) = O(n)$

### 3.2.7 循环赛日程表

设有  $n = 2^k$  个运动员要进行网球循环赛。现要设计一个满足以下要求的比赛日程表：

1. 每个选手必须与其他  $n-1$  个选手各赛一次；
2. 每个选手一天只能赛一次；
3. 循环赛一共进行  $n-1$  天。

```

1 void sol(int x, int y, int size, int st){
2     if(size >= 2){

```



```

3      sol(x, y, size - 1, st);
4      sol(x + (1 << (size - 1)), y, size - 1, st + (1 << (size - 1)));
5      sol(x, y + (1 << (size - 1)), size - 1, st + (1 << (size - 1)));
6      sol(x + (1 << (size - 1)), y + (1 << (size - 1)), size - 1, st);
7  }
8  else{
9      plan[x][y] = plan[x + 1][y + 1] = st;
10     plan[x + 1][y] = plan[x][y + 1] = st + 1;
11 }
12 }
```

### 3.2.8 向量卷积

给定向量  $a = (a_0, a_1, \dots, a_n), b = (b_0, b_1, \dots, b_n)$ , 则有二者的卷积为  $c = (c_0, c_1, \dots, c_n)$ , 其中

$$c_i = \sum_{j=0}^i a_j b_{i-j}$$

而卷积与多项式乘法等价, 即令  $A(x) = a_0 + a_1x + \dots + a_{m-1}x^{m-1}, B(x) = b_0 + b_1x + \dots + b_{n-1}x^{n-1}$ , 则  $C(x) = A(x)B(x) = a_0b_0 + (a_0b_1 + a_1b_0)x + \dots + a_{m-1}b_{n-1}x^{m+n-2}$

如果采用暴力做法, 先逐项相乘, 再同类合并, 那么复杂度将是  $O(n^2)$ .

我们设计特定值  $x_1, x_2, \dots, x_{2n}$  求出  $A(x_j)$  和  $B(x_j), j = 1, 2, \dots, 2n$ , 进一步地计算出  $C(x_j)$ , 最后利用多项式插值方法, 根据  $C(x)$  在  $x = x_1, x_2, \dots, x_{2n}$  的插值点, 求出  $C(x)$ .

而基于此, 我们最关键的计算提速步骤就在于利用分治算法的思想对  $A(x_i)$  的求值步骤进行加速:

$$\begin{aligned}
 A_{\text{even}}(x) &= a_0 + a_2x + a_4x^2 + \dots + a_{n-2}x^{\frac{n-2}{2}} \\
 A_{\text{odd}}(x) &= a_1 + a_3x + a_5x^2 + \dots + a_{n-1}x^{\frac{n-2}{2}} \\
 A(x) &= A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)
 \end{aligned}$$

这里相当于把本来  $x$  是 1 的  $2n$  次方根, 变成了  $x^2$  是 1 的  $n$  次方根, 实现了问题的降级, 从根本上使得复杂度从  $O(n^2)$  变成了  $O(n \log n)$ .

而在这个过程中, 我们利用复数单位根的对称性可以进一步地加速, 即:

$$\begin{aligned}
 A(w_j) &= A_{\text{even}}(w_j^2) + w_j A_{\text{odd}}(w_j^2) \\
 A(w_{j+n}) &= A_{\text{even}}(w_j^2) - w_j A_{\text{odd}}(w_j^2)
 \end{aligned}$$

而如何通过  $C(w_i)$  求出  $c_i$ , 方法如下, 但是不要深究。

构造多项式:

$$D(x) = C(w_0) + C(w_1)x + \dots + C(w_{2n-1})x^{2n-1}$$

可以证明 (别管怎么证的):

$$D(w_i) = 2nc_{2n-i}$$

至此, 时间复杂度被优化到了  $O(n \log n)$

### 3.2.9 棋盘覆盖

其实就是这个题，分治思想：<https://www.luogu.com.cn/problem/P1228>

由此可想到将一个边长为  $2^k$  的正方形分成四个边长为  $2^{k-1}$  的正方形，特殊方格必在其中的一个正方形中。然后再中心摆上一个地毯，地毯缺少的一块朝向特殊方格所在正方形，这样相当于每个小正方形中有一个特殊方格。一直分治下去，直到边长为 2 结束。

### 3.2.10 最大子序列和问题

题目：<https://www.luogu.com.cn/problem/P1115>

1. 第一个数为一个有效序列
2. 如果一个数加上上一个有效序列得到的结果比这个数大，那么该数也属于这个有效序列。
3. 如果一个数加上上一个有效序列得到的结果比这个数小，那么这个数单独成为一个新的有效序列
4. 在执行上述处理的过程中实时更新当前有效序列的所有元素之和并取最大值。

这样代码就是：

```
1  for(int i=0;i<n;i++){
2      cin>>a;
3      f=max(f+a,a);
4      ans=max(f,ans);
5  }
```

而如果用分治算法解决这道题，只要求出左半部分和右半部分的最大子序列和，最后计算一定要包含中间的最大子序列和，三者取最大值就是答案。

```
1  int g(int l,int m,int r){//l,右边界、r,左边界、m,中间数
2      int max1=a[m],max2=a[m+1];//最大值
3      int sum1=0,sum2=0;//和，循环用
4      //从m向左延伸
5      for(int i=m;i>=l;i--){
6          sum1+=a[i];
7          max1=max(max1,sum1);
8      }
9      //从m+1向右延伸
10     for(int i=m+1;i<=r;i++){
11         sum2+=a[i];
12         max2=max(max2,sum2);
13     }
14     return max1+max2;
15 }
16
```

```

17 int he(int l,int r){
18     if(l==r){
19         return a[l];
20     }
21     int mid=(l+r)/2;
22     return max(he(l,mid),max(he(mid+1,r),g(l,mid,r)));
23 }

```

### 3.3 分治算法的改进

主要就是两个渠道：减少子问题数量、减少递归内部计算量。

#### 3.3.1 大整数乘法

设  $n$  是 2 的幂,  $u = w \cdot 2^{\frac{n}{2}} + x, v = y \cdot 2^{\frac{n}{2}} + z$ , 那么  $uv = wy2^n + (xy + wz)2^{\frac{n}{2}} + xz$ , 但是这样时间复杂度是  $\Theta(n^2)$  的。

我们可以考虑  $wz + xy = (w + x)(y + z) - wy - xz$ , 这样就只需要进行 3 个  $\frac{n}{2}$  的子问题了, 复杂度为  $\Theta(n^{\log 3}) = O(n^{1.59})$

#### 3.3.2 矩阵乘法

暴力做法是  $\Theta(n^3)$  的, 而如果只是粗暴的分区处理, 仍然是  $\Theta(n^3)$  的, 这里我们介绍 STRASSEN 算法, 增加加减法的次数来减少乘法次数。

1. 与朴素分治算法相同, 分解出左上, 左下, 右上, 右下四个子矩阵。
2. 创建 10 个  $\frac{n}{2} \times \frac{n}{2}$  的矩阵  $S_i$ , 每个  $S_i$  保存两个子矩阵的和或差。
3. 用子矩阵和  $S_i$  相乘, 递归地计算 7 个  $\frac{n}{2} \times \frac{n}{2}$  的矩阵  $P_i$ 。
4. 通过  $P_i$  的不同组合进行加减, 得到  $C$  的子矩阵。

$$\begin{aligned}
 S_1 &= B_{1,2} - B_{2,2} & S_2 &= A_{1,1} + A_{1,2} \\
 S_3 &= A_{2,1} + A_{2,2} & S_4 &= B_{2,1} - B_{1,1} \\
 S_5 &= A_{1,1} + A_{2,2} & S_6 &= B_{1,1} + B_{2,2} \\
 S_7 &= A_{1,2} - A_{2,2} & S_8 &= B_{2,1} + B_{2,2} \\
 S_9 &= A_{1,1} - A_{2,1} & S_{10} &= B_{1,1} + B_{1,2} \\
 P_1 &= A_{1,1} \cdot S_1 & P_2 &= S_2 \cdot B_{2,2} \\
 P_3 &= S_3 \cdot B_{1,1} & P_4 &= A_{2,2} \cdot S_4 \\
 P_5 &= S_5 \cdot S_6 & P_6 &= S_7 \cdot S_8 \\
 P_7 &= S_9 \cdot S_{10} \\
 C_{1,1} &= P_5 + P_4 - P_2 + P_6 & C_{1,2} &= P_1 + P_2 \\
 C_{2,1} &= P_3 + P_4 & C_{2,2} &= P_5 + P_1 - P_3 - P_7
 \end{aligned}$$

对于矩阵规模不是 2 的幂的情况, 只需要补 0 即可。

类似地，试只用三次乘法完成复数相乘  $(a + bi)(c + di) = (ac - bd) + (ad + bc)i$ 。

设

$$\begin{cases} \alpha = ac \\ \beta = bd \\ \gamma = (a + b)(c + d) \end{cases}$$

则：

$$(a + bi)(c + di) = (\alpha - \beta) + (\gamma - \alpha - \beta)i$$

### 3.3.3 最近点对问题

方法是前面提到过的，按找  $x$  坐标排序后割开成两部分，最终答案就是  $\delta_l, \delta_r, \delta'$  之间的最小值。但是关键的问题在于  $\delta'$ （左右之间的最小值）该怎么求。（直接用前面求出来的  $\delta = \min(\delta_l, \delta_r)$ ，这样得到一个高为  $\delta$ ，长为  $2\delta$  的矩形，显然对于一个正方形来说，两两距离超过边长的点最多有 4 个，所以这个最多有 8 个，这样只需要枚举其附近的 7 个点即可。

## 3.4 本章作业

### 3.4.1 快排空间复杂度

证明算法 QUICKSORT 的空间复杂度在  $\Theta(\log n)$  与  $\Theta(n)$  之间变动。并计算该算法的平均空间复杂度是多少？

快速排序的空间复杂度主要由递归调用栈的深度（栈中存储的是枢轴量）决定，而递归深度取决于划分的平衡性。

快速排序的平均空间复杂度由递归树的平均深度决定。通过分析划分的期望行为，可以证明平均深度为  $O(\log n)$ 。

划分点是随机选择的，划分后左右子数组大小的比例为任意  $(k, n - k - 1)$  的概率均等。设  $D(n)$  为递归树深度的期望值，满足以下递推关系：

$$D(n) = 1 + \frac{1}{n} \sum_{k=0}^{n-1} \max(D(k), D(n - k - 1))$$

由于  $D(k)$  和  $D(n - k - 1)$  对称，可简化为：

$$D(n) = 1 + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} D(k)$$

假设  $D(k) \leq c \log k$  对所有  $k < n$  成立，则：

$$D(n) \leq 1 + \frac{2c}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} \log k$$

通过积分近似求和（ $\sum \log k \approx n \log n - n$ ），可得：

$$D(n) \leq 1 + \frac{2c}{n} \left( n \log n - \frac{n}{2} \log \frac{n}{2} \right) \approx c \log n$$

因此， $D(n) = O(\log n)$ 。

### 3.4.2 多选问题

设  $S$  是包含  $n$  个不等数的集合, 给定整数  $r$  ( $1 \leq r \leq n$ ),  $K = \{k_1, k_2, \dots, k_r\}$  且  $1 \leq k_1 < k_2 < \dots < k_r \leq n$ 。要求在  $S$  中找出第  $k_1$  小, 第  $k_2$  小,  $\dots$ , 第  $k_r$  小的  $r$  个元素。例如,  $r = 3$ ,  $K = \{2, 4, 7\}$ , 则输出  $S$  中的第 2 小, 第 4 小, 第 7 小的元素。当  $r = 1$  时, 该问题就是一般选择问题。当  $r = n$  时, 该问题就变成了排序问题。

1. 设计一个最坏情况下时间复杂度为  $O(nr)$  的算法。
2. 若  $r > 1$ , 设计一个最坏情况下时间复杂度为  $O(n \log r)$  的算法。

时间复杂度  $O(nr)$  的算法

**Input:** 集合  $S$  包含  $n$  个不等数, 整数  $r$ , 集合  $K = \{k_1, k_2, \dots, k_r\}$  且  $1 \leq k_1 < k_2 < \dots < k_r \leq n$

**Output:** 集合  $S$  中第  $k_1$  小, 第  $k_2$  小,  $\dots$ , 第  $k_r$  小的  $r$  个元素

```

1 对  $K$  进行排序, 确保  $k_1 < k_2 < \dots < k_r$ ;
2 初始化结果数组  $R[1..r]$ ;
3 for  $i \leftarrow 1$  to  $r$  do
4   使用 SELECT 算法在  $S$  中找出第  $k_i$  小的元素  $x$ ;
5    $R[i] \leftarrow x$ ;
6 end
7 return  $R$ ;
```

**Algorithm 12:** MultipleSelectBasic

时间复杂度  $O(n \log r)$  的算法

1. 基本情况处理:
  - 若  $r = 1$ , 直接调用 SELECT 算法 (时间复杂度  $O(n)$ ) 返回第  $k_1$  小的元素
2. 预处理:
  - 对  $K$  排序, 确保  $k_1 < k_2 < \dots < k_r$
3. 选取枢纽元:
  - 计算  $K$  的中位数位置:  $k_{mid} = k_{\lfloor r/2 \rfloor}$
  - 使用 SELECT 算法在  $S$  中找到第  $k_{mid}$  小的元素  $pivot$
4. 划分数组:
  - $L = \{x \in S \mid x < pivot\}$
  - $E = \{x \in S \mid x = pivot\}$
  - $G = \{x \in S \mid x > pivot\}$
5. 递归处理子问题:

**Input:** 集合  $S$  包含  $n$  个不等数, 整数  $r > 1$ , 集合  $K = \{k_1, k_2, \dots, k_r\}$  且

$$1 \leq k_1 < k_2 < \dots < k_r \leq n$$

**Output:** 集合  $S$  中第  $k_1$  小, 第  $k_2$  小,  $\dots$ , 第  $k_r$  小的  $r$  个元素

```

1 if  $r = 1$  then
2   | return SELECT( $S, k_1$ );
3 end
4 对  $K$  进行排序, 确保  $k_1 < k_2 < \dots < k_r$ ;
5  $k_{mid} \leftarrow k_{\lfloor r/2 \rfloor}$ ;
6  $pivot \leftarrow$  SELECT( $S, k_{mid}$ );
7 将  $S$  划分为:  $L \leftarrow \{x \in S \mid x < pivot\}$ ;
8  $E \leftarrow \{x \in S \mid x = pivot\}$ ;
9  $G \leftarrow \{x \in S \mid x > pivot\}$ ;
10 初始化子问题的  $K$  集合:  $K_L \leftarrow \{k_i \in K \mid k_i < k_{mid}\}$ ;
11  $K_G \leftarrow \{k_i \in K \mid k_i > k_{mid}\}$ ;
12 递归解决子问题:  $R_L \leftarrow$  MultipleSelectOptimized( $L, K_L, |L|, |K_L|$ );
13  $R_G \leftarrow$  MultipleSelectOptimized( $G, K_G - |L| - |E|, |G|, |K_G|$ );
14 return  $R_L \cup \{pivot\} \cup R_G$ ;

```

**Algorithm 13:** MultipleSelectOptimized

- 左子问题  $K_L = \{k_i \in K \mid k_i < k_{mid}\}$ : 在  $L$  中递归求解
- 右子问题  $K_G = \{k_i \in K \mid k_i > k_{mid}\}$ : 在  $G$  中递归求解, 调整查询排名  $k'_i = k_i - |L| - |E|$

6. 合并结果:

- 返回  $R_L \cup \{pivot\} \cup R_G$

时间复杂度分析

- 递归深度:  $O(\log r)$  (每次  $K$  规模减半)
- 每层工作量:  $O(n)$  (SELECT 和划分)
- 总时间复杂度:  $O(n \log r)$

## 4 chapter4 动态规划

当子问题不独立时, 如果能够保存已解决的子问题的答案, 而在需要时再找出已求得的答案, 就可以避免大量重复计算, 从而得到多项式时间算法。而动态规划算法只解决每个子问题一次, 然后将其答案保存起来, 从而避免了每次遇到子问题时重新计算答案的工作。即动态规划通过空间开销来节省由于重复计算带来的时间开销。

**最优性原理:** 如果问题的最优解所包含的子问题的解也是子问题的最优解, 就称该问题具有最优子结构, 即满足最优性原理 (优化原则)。

并不是所有优化问题都适用优化原则, 有时对某个子问题的解不一定最优, 但延伸成整个问题的解时反而最优。

在证明一个问题满足最优子结构性质时，可以考虑反证法，我们后面在 0-1 背包问题中具体展开。

## 4.1 最长公共子序列

要证明最长公共子序列问题满足最优子结构性质，设  $X = \langle x_1, \dots, x_m \rangle$  和  $Y = \langle y_1, \dots, y_n \rangle$  的 LCS 为  $Z = \langle z_1, \dots, z_k \rangle$ 。当  $x_m = y_n$  时必有  $z_k = x_m$ ，否则可将  $x_m$  追加到  $Z$  得到更长子序列导致矛盾，此时  $Z_{k-1}$  必须是  $X_{m-1}$  和  $Y_{n-1}$  的 LCS，若存在更长的  $W$  则  $W \oplus x_m$  将形成更长公共子序列；当  $x_m \neq y_n$  时，若  $z_k \neq x_m$  则  $Z$  是  $X_{m-1}$  和  $Y$  的 LCS，若  $z_k \neq y_n$  则  $Z$  是  $X$  和  $Y_{n-1}$  的 LCS，否则可通过选择更长子序列构造矛盾。由此 LCS 的最优解总由子问题最优解构成，满足最优子结构性质。

$s_{[-1]}$  表示序列  $s$  的最后一个元素， $s_{[: -1]}$  表示  $s$  去掉最后一个元素后的子序列

$$\text{LCS}(s_1, s_2) = \begin{cases} 0, & \text{if } s_1 \text{ or } s_2 \text{ is empty} \\ \text{LCS}(s_{1[: -1]}, s_{2[: -1]}) + 1, & \text{if } s_{1[-1]} = s_{2[-1]} \\ \max(\text{LCS}(s_{1[: -1]}, s_2), \text{LCS}(s_1, s_{2[: -1]})), & \text{otherwise} \end{cases}$$

```

1 while (cin >> s1 >> s2){
2     memset(dp, 0, sizeof(dp));
3     int n1 = strlen(s1), n2 = strlen(s2);
4     for (int i = 1; i <= n1; ++i)
5         for (int j = 1; j <= n2; ++j)
6             if (s1[i - 1] == s2[j - 1])
7                 dp[i][j] = dp[i - 1][j - 1] + 1;
8             else
9                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
10    cout << dp[n1][n2] << endl;
11 }
```

## 4.2 最长上升子序列

先说一种最普遍的方法，因为所求为子序列，所以这道题很容易想到一种线性动态规划。我们需要求最长上升子序列，为了上升我们肯定要知道我们当前阶段最后一个元素为多少，为了最长我们还要知道当前我们的序列有多长。我们可以用前者来充当第一维描述：设  $F[i]$  表示以  $A[i]$  为结尾的最长上升子序列的长度，为了保证元素单调递增我们肯定只能从  $i$  前面的且末尾元素比  $A[i]$  小的状态转移过来：

$$F[i] = \max_{0 \leq j < i, A[j] < A[i]} \{F[j] + 1\}$$

初始值为  $F[0] = 0$ ，而答案可以是任何阶段中只要长度最长的那一个，所以我们边转移边统计答案。

复杂度： $O(n^2)$

贪心加二分查找。

我们肯定要知道我们当前阶段最后一个元素为多少，还有当前我们的序列有多长。前两种方法都是用前者做状态，我们为什么不可以用后者做状态呢？

设  $F[i]$  表示长度为  $i$  的最长上升子序列的末尾元素的最小值，我们发现这个数组的权值一定单调不降（仔细想一想，这就是我们贪心的来由）。于是我们按顺序枚举数组  $A$ ，每一次对  $F[]$  数组二分查找，找到小于  $A[i]$  的最大的  $F[j]$ ，并用它将  $F[j+1]$  更新。

期望复杂度： $O(n \log n)$

```

1  int lengthOfLIS(vector<int>& nums) {
2      vector<int> tails; // tails[i]表示长度为i+1的LIS的最小末尾元素
3      for (int num : nums) {
4          // 使用二分查找找到第一个大于等于num的元素位置
5          auto it = lower_bound(tails.begin(), tails.end(), num);
6          if (it == tails.end()) {
7              // 如果num比所有元素都大，可以扩展LIS长度
8              tails.push_back(num);
9          } else {
10             // 否则替换第一个大于等于num的元素，保持tails数组的最小性
11             *it = num;
12         }
13     }
14     return tails.size(); // tails的大小就是LIS的长度
15 }

```

要同时求出最长公共子序列的长度和序列，首先初始化二维数组  $c[m+1][n+1]$ （存储长度）和  $b[m+1][n+1]$ （存储方向），将  $c$  的第一行和第一列初始化为 0。然后遍历字符串  $X$  和  $Y$ ：当  $X[i-1] = Y[j-1]$  时，令  $c[i][j] = c[i-1][j-1] + 1$  并在  $b[i][j]$  标记””；否则比较  $c[i-1][j]$  和  $c[i][j-1]$ ，若  $c[i-1][j] > c[i][j-1]$  则令  $c[i][j] = c[i-1][j]$  且标记”↑”，否则令  $c[i][j] = c[i][j-1]$  且标记”←”。最终  $c[m][n]$  即为 LCS 长度。回溯时从  $b[m][n]$  开始：若标记为””则递归  $b[i-1][j-1]$  并记录  $X[i-1]$ ；若为”↑”则递归  $b[i-1][j]$ ；若为”←”则递归  $b[i][j-1]$ 。最后将记录的字符反转即得 LCS 序列。

### 4.3 矩阵链乘法

给定  $n$  个矩阵的链  $\langle A_1, A_2, \dots, A_n \rangle$ ，其中矩阵  $A_i$  的维度为  $p_{i-1} \times p_i$

设一个由  $n$  个矩阵组成的序列的可选的括号化方案为  $P(n)$ 。则  $P(n)$  递归式（卡特兰数）为：

$$P(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n \geq 2 \end{cases}$$

其闭合形式解为：

$$P(n) = \frac{1}{n} \binom{2n-2}{n-1} = \frac{(2n-2)!}{n((n-1)!)^2} \approx \frac{4^n}{4\sqrt{\pi n^{1.5}}}$$

我们的求解过程分以下几步：

1. 描述最优解的结构特征。
2. 递归定义最优解的值。



**Input:** 字符串  $X = x_1x_2 \cdots x_m$ , 字符串  $Y = y_1y_2 \cdots y_n$   
**Output:** LCS 长度  $c[m][n]$ , 方向矩阵  $b[1..m][1..n]$

```

1 for  $i \leftarrow 0$  to  $m$  do
2    $c[i][0] \leftarrow 0$ ;
3 end
4 for  $j \leftarrow 0$  to  $n$  do
5    $c[0][j] \leftarrow 0$ ;
6 end
7 for  $i \leftarrow 1$  to  $m$  do
8   for  $j \leftarrow 1$  to  $n$  do
9     if  $x_i == y_j$  then
10       $c[i][j] \leftarrow c[i-1][j-1] + 1$ ;
11       $b[i][j] \leftarrow \text{"}\nwarrow\text{"}$ ;
12    else
13      if  $c[i-1][j] \geq c[i][j-1]$  then
14         $c[i][j] \leftarrow c[i-1][j]$ ;
15         $b[i][j] \leftarrow \text{"}\uparrow\text{"}$ ;
16      else
17         $c[i][j] \leftarrow c[i][j-1]$ ;
18         $b[i][j] \leftarrow \text{"}\leftarrow\text{"}$ ;
19      end
20    end
21  end
22 end
23 return  $(c[m][n], b)$ ;

```

Algorithm 14: LCS-Length

3. 计算最优解的值。

4. 根据计算信息构造最优解。

定义  $A_{i:j}$  为  $A_i A_{i+1} \cdots A_j$  的乘积, 其中  $i \leq j$ 。如果  $A_{i:j}$  可以继续分割 (即  $i < j$ ), 利用括号化分割成  $A_{i:k}$  和  $A_{k+1:j}$ , 其中  $i \leq k < j$ 。

$A_{i:j}$  的括号化后的计算代价为:

$$\text{计算代价}(A_{i:j}) = \text{计算代价}(A_{i:k}) + \text{计算代价}(A_{k+1:j}) + \text{相乘代价}(A_{i:k}, A_{k+1:j})$$

关键性质:

- 子问题的最优括号化方案包含在原问题的最优括号化方案中
- 通过求出所有子问题的最优解, 就能组成原问题的最优解
- 在求解子问题的最优分割方案时, 需要对所有可能的分割点  $k$  进行考察

**Input:** 方向矩阵  $b[1..m][1..n]$ , 字符串  $X$ , 位置  $(i, j)$

**Output:** LCS 序列

```

1 if  $i == 0$  or  $j == 0$  then
2   | return;
3 end
4 if  $b[i][j] == "\searrow"$  then
5   | PRINT-LCS( $b, X, i - 1, j - 1$ );
6   | 输出  $x_i$ ;
7 else if  $b[i][j] == "\uparrow"$  then
8   | PRINT-LCS( $b, X, i - 1, j$ );
9 else
10  | PRINT-LCS( $b, X, i, j - 1$ );
11 end

```

**Algorithm 15:** PRINT-LCS

假设矩阵链乘问题不满足最优子结构性质, 即存在某个子问题的最优解不是构成原问题最优解的一部分。

考虑矩阵链  $A_1 A_2 \cdots A_n$  的最优括号化方案, 设其在位置  $k$  处分割:

$$A_{1:n} = (A_{1:k})(A_{k+1:n})$$

根据假设, 存在以下两种情形之一:

1.  $A_{1:k}$  不是子问题  $A_{1:k}$  的最优括号化方案
2.  $A_{k+1:n}$  不是子问题  $A_{k+1:n}$  的最优括号化方案

以第一种情形为例 (第二种情形对称), 设存在更优的  $A_{1:k}^*$  使得:

$$m^*[1, k] < m[1, k]$$

则可构造新的括号化方案:

$$A'_{1:n} = (A_{1:k}^*)(A_{k+1:n})$$

其总计算代价为:

$$\begin{aligned}
 m'[1, n] &= m^*[1, k] + m[k+1, n] + p_0 p_k p_n \\
 &< m[1, k] + m[k+1, n] + p_0 p_k p_n \\
 &= m[1, n]
 \end{aligned}$$

这与  $m[1, n]$  是最优解的初始假设矛盾。因此, 原假设不成立, 矩阵链乘必须满足最优子结构性质。

定义  $m[i, j]$  表示计算  $A_{i:j}$  的乘积所需标量乘法的最少次数。计算  $A_1 A_2 \cdots A_n$  的乘积所需标量乘法的最少次数可表示为  $m[1, n]$ 。

我们可以递归求解  $m[i, j]$ :

- 当  $i = j$  时,  $A_{i,i} = A_i$ ,  $m[i, i] = 0$ , 其中  $i = 1, 2, \dots, n$

```

Input: 矩阵维度序列  $p[0..n]$ 
Output: 最小乘法次数  $m[n, n]$  和最优分割方案  $s[1..n-1, 2..n]$ 

1 for  $i \leftarrow 1$  to  $n$  do
2    $m[i, i] \leftarrow 0$  // 初始化对角线
3 end
4 for  $l \leftarrow 2$  to  $n$  do
5   //  $l$  为链长度
6   for  $i \leftarrow 1$  to  $n - l + 1$  do
7      $j \leftarrow i + l - 1$ ;
8      $m[i, j] \leftarrow \infty$ ;
9     for  $k \leftarrow i$  to  $j - 1$  do
10       $q \leftarrow m[i, k] + m[k + 1, j] + p[i - 1] \times p[k] \times p[j]$ ;
11      if  $q < m[i, j]$  then
12         $m[i, j] \leftarrow q$ ;
13         $s[i, j] \leftarrow k$  // 记录最优分割点
14      end
15    end
16  end
17 end

```

Algorithm 16: MatrixChainOrder

- 当  $i < j$  时, 利用最优括号化方案:

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j, \quad \text{其中 } i \leq k < j$$

分段函数形式表示为:

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

定义  $s[i, j]$  表示  $A_{i:j}$  的最优分割方案中的值  $k$ , 即满足:

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$$

算法的时间复杂度是  $\Theta(n^3)$ , 空间复杂度是  $\Theta(n^2)$

**Input:** 分割矩阵  $s[1..n-1, 2..n]$ , 矩阵编号  $i, j$

**Output:** 最优括号化方案

```

1 if  $i == j$  then
2   | 输出 " $A_i$ ";
3 end
4 else
5   | 输出 "(" ;
6   | PrintOptimalParens( $s, i, s[i, j]$ );
7   | PrintOptimalParens( $s, s[i, j] + 1, j$ );
8   | 输出 ")" ;
9 end

```

**Algorithm 17:** PrintOptimalParens

**Input:**  $n \times n$  维矩阵  $l[1..n, 1..n]$ , 对于有向图  $G = (\{1, 2, \dots, n\}, E)$  中的边  $(i, j)$  的长度是  $l[i, j]$

**Output:** 矩阵  $D$  使得  $D[i, j] = i$  到  $j$  的距离

```

1  $D \leftarrow l$ ;
2 将输入矩阵  $l$  复制到  $D$  for  $k \leftarrow 1$  to  $n$  do
3   | for  $i \leftarrow 1$  to  $n$  do
4   |   | for  $j \leftarrow 1$  to  $n$  do
5   |   |   |  $D[i, j] = \min\{D[i, j], D[i, k] + D[k, j]\}$ ;
6   |   |   end
7   |   end
8   end

```

**Algorithm 18:** FLOYD 算法

#### 4.4 多源最短路径问题

#### 4.5 0/1 背包问题

$$\begin{aligned}
 & \max \sum_{i=1}^n v_i x_i \\
 & s.t. \begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0, 1\} \quad i = 1, \dots, n \end{cases}
 \end{aligned}$$

设  $(y_1, y_2, \dots, y_n)$  是上式的一个最优解, 下证  $(y_2, y_3, \dots, y_n)$  是下面子问题的最优解:

$$\begin{aligned}
 & \max \sum_{i=2}^n v_i x_i \\
 & s.t. \begin{cases} \sum_{i=2}^n w_i x_i \leq C - w_1 y_1 \\ x_i \in \{0, 1\} \quad i = 2, \dots, n \end{cases}
 \end{aligned}$$

**Input:** 物品集合  $U = \{u_1, u_2, \dots, u_n\}$ , 体积  $s_1, s_2, \dots, s_n$ , 价值  $v_1, v_2, \dots, v_n$ , 背包容量  $C$

**Output:**  $\sum_{u_i \in S} v_i$  的最大总价值, 且满足  $\sum_{u_i \in S} s_i \leq C$ , 其中  $S \subseteq U$

```

1 for  $i \leftarrow 0$  to  $n$  do
2   |  $V[i, 0] \leftarrow 0$ ;
3 end
4 for  $j \leftarrow 0$  to  $C$  do
5   |  $V[0, j] \leftarrow 0$ ;
6 end
7 for  $i \leftarrow 1$  to  $n$  do
8   for  $j \leftarrow 1$  to  $C$  do
9     // 如果是滚动数组的话, 这里需要是倒序的
10     $V[i, j] \leftarrow V[i-1, j]$ ;
11    if  $s_i \leq j$  then
12      |  $V[i, j] \leftarrow \max\{V[i, j], V[i-1, j-s_i] + v_i\}$ ;
13    end
14  end
15 end
16 return  $V[n, C]$ ;

```

**Algorithm 19:** KNAPSACK 算法

考虑反证, 假设  $(z_2, z_3, \dots, z_n)$  是上述子问题的最优解, 则:

$$\sum_{i=2}^n v_i z_i \geq \sum_{i=2}^n v_i y_i$$

$$w_1 y_1 + \sum_{i=2}^n w_i z_i \leq C$$

所以有

$$v_1 y_1 + \sum_{i=2}^n w_i z_i \geq \sum_{i=1}^n n v_i y_i$$

这说明  $(y_1, z_2, z_3, \dots, z_n)$  是比  $(y_1, y_2, \dots, y_n)$  更优的一个解, 矛盾!

背包问题的最优解能够在  $\Theta(nC)$  的时间内和  $\Theta(C)$  的空间内得到。

## 4.6 图像压缩问题

对于给定的  $n$  个像素点  $p_1, p_2, p_3, \dots, p_n$ , 将其分割为连续的  $m$  个段  $S_1, S_2, S_3, \dots, S_m$ 。设第  $i$  个像素段  $S_i$  中, 有  $N[i]$  个像素 ( $1 \leq i \leq m$ ), 每个像素用  $b[i]$  位表示 (比如 0-15 的数字, 要用 4 位表示)。

设  $h_i$  为分段  $S_i$  中最大灰度值所对应的二进制数的位数, 即

$$h_i = \left\lceil \log \left( \max_{t[i]+1 \leq k \leq t[i]+N[i]} p_k + 1 \right) \right\rceil$$

由于  $p_k \leq 255$ , 则  $h_i \leq b[i] \leq 8$ , 需要用 3 位来表示  $b[i]$ 。我们给定  $N[i]$  最多是 255, 这样需要 8 位来存储。像素段  $S_i$  需要的总内存空间是  $N[i] * b[i] + 11$ 。

像素序列  $\{p_1, p_2, p_3, \dots, p_n\}$  所需存储空间是:

$$\sum_{i=1}^m N[i] \times b[i] + 11m$$

假设图像压缩问题不满足最优子结构性质, 即存在某个最优分段方案, 其中至少一个子段的分割不是局部最优的。

考虑像素序列  $p_1$  到  $p_n$  的最优分割方案  $S_1, \dots, S_m$ , 设其中子段  $S_k$  ( $1 \leq k \leq m$ ) 不是该子区间的最优分割。则存在更优的分割  $S_k^*$  使得:

$$N[k] \times b[k] + 11 > N^*[k] \times b^*[k] + 11$$

将原方案中的  $S_k$  替换为  $S_k^*$ , 得到新方案的总存储空间:

$$\begin{aligned} \text{Total}^* &= \sum_{i \neq k} (N[i] \times b[i] + 11) + (N^*[k] \times b^*[k] + 11) \\ &< \sum_{i=1}^m (N[i] \times b[i] + 11) = \text{Total} \end{aligned}$$

这与原方案的最优性矛盾。因此, 最优分割方案中的每个子段必须自身是最优分割。

定义函数  $bm(i, j)$  表示从像素  $i$  到  $j$  中像素灰度值  $p_k$  最大值所需的最少二进制位数:

$$bm(i, j) = \left\lceil \log \left( \max_{i \leq k \leq j} \{p_k\} + 1 \right) \right\rceil \quad (1 \leq i \leq n)$$

根据动态规划原理, 存储长度满足:

$$S[i] = \min_{1 \leq k \leq \min\{i, 256\}} \{S[i-k] + k \times bm(i-k+1, i)\} + 11 \quad (1)$$

```

1 //s[i]记录1-i像素所需的最小空间, I[i]记录1-i像素的最优划分的最后一个段的大小,
2 //b[i]由p[i]得出, 它记录p[i]需要几个比特来存储。
3 void compress2(int* p, int n, int* s, int* I, int* b)
4 {
5     int header = 11; //一个段需要3位来存储每个像素点需要几个比特来存储, 8位来存储这个像
        素段有多少个像素点。
6     int Imax = 256; //N[i]的最大值
7     s[0] = 0;
8     //这个循环是为了计算s[i]数组, 用I[i]数组保存在哪里分段的信息, 其它数组、变量都是辅
        助。
9     for (int i = 1; i <= n; i++) {
10         //注意s[i]初始化为INT_MAX, 而不是初始化为0, 因为s[i]要寻找最小值
11         b[i] = length(p[i]);
12         int bmax = 0; //我们要算出在一个像素段中, 最大的那个像素点需要用几个比特来存
            储, 然后这个像素段中所有点都用那么多比特来存储。

```

```

13     s[i]=INT_MAX;//s[i]求的是最小值，所以初始化为大整数。
14     I[i];
15     for (int k = 1; k <= i && k <= Imax; k++) {
16         if (bmax < b[i - k + 1]) bmax = b[i - k + 1]; //如果像素段出现更大
           像素点，需要更大存储空间，刷新bmax
17         if (s[i] > s[i - k] + k * bmax) { //出现更小的s[i]则刷新
18             s[i] = s[i - k] + k * bmax;
19             I[i] = k; 记录分割位置：第1-i像素点的最后一个段有多少数字。
20         }
21     }
22     s[i] += header;
23 }
24 }

```

时间复杂度  $O(n)$

## 4.7 最大子段和

```

1  cin>>n;
2  for(int i=0;i<n;i++){
3      cin>>a;
4      f=max(f+a,a);
5      ans=max(f,ans);
6  }

```

## 4.8 多段图问题

- $P(i, j)$ : 从  $V_i$  中结点  $j$  到汇点  $t$  的最短路径
- $COST(i, j)$ :  $P(i, j)$  的路径成本
- $D(i, j)$ :  $V_i$  中结点  $j$  的最优决策（选择的后继节点）

$$COST(i, j) = \begin{cases} 0 & \text{当 } i = k \text{ (终止条件)} \\ \min_{\substack{l \in V_{i+1} \\ \langle j, l \rangle \in E}} \{c(j, l) + COST(i + 1, l)\} & \text{当 } i < k \text{ (递推规则)} \end{cases}$$

- $BP(i, j)$ : 从起点到阶段  $V_i$  节点  $j$  的最短路径
- $BCOST(i, j)$ : 该路径的累计成本
- $D(i, j)$ : 节点  $j$  选择的最优前驱节点

$$\text{BCOST}(i, j) = \begin{cases} 0 & \text{基础情况 (起点本身)} \\ \min_{\substack{l \in V_{i-1} \\ \text{边}(l, j) \in E}} [\text{BCOST}(i-1, l) + c(l, j)] & \text{递推情况} \end{cases}$$

- 将  $n$  个资源分配给  $r$  个项目
- 分配  $j$  个资源给项目  $i$  时, 获得净利  $N(i, j)$
- 目标: 找到使总净利最大的分配方案

#### 1. 阶段划分:

- 构造  $r+1$  段有向图 ( $r$  个项目对应  $r$  个决策阶段 +1 个终止阶段)
- 段  $i$  ( $1 \leq i \leq r$ ) 表示项目  $i$  的分配决策

#### 2. 节点设计:

- 源节点  $s = V(1, 0)$
- 中间节点  $V(i, j) = \begin{cases} \text{共分配 } j \text{ 个资源给项目 } 1, \dots, i-1, & 2 \leq i \leq r \\ j \in \{0, 1, \dots, n\} \end{cases}$
- 汇节点  $t = V(r+1, n)$

#### 3. 边构造规则:

- 边形式:  $\langle V(i, j), V(i+1, l) \rangle$ , 满足  $j \leq l$  且  $1 \leq i \leq r$
- 边权重:

$$w(\langle V(i, j), V(i+1, l) \rangle) = \begin{cases} N(i, l-j), & 1 \leq i < r \\ \max\{N(r, p) \mid 0 \leq p \leq n-j\}, & i = r \end{cases}$$

#### 4. 求解方法:

- 最优解对应  $s$  到  $t$  的**最大成本路径**
- 通过将边权重取负 ( $-N(i, k)$ ) 转换为多段图最短路径问题
- 动态规划递推式:

$$\text{COST}(i, j) = \max_{\substack{l \geq j \\ \langle V(i, j), V(i+1, l) \rangle \in E}} \{w(\langle V(i, j), V(i+1, l) \rangle) + \text{COST}(i+1, l)\}$$

### 4.9 最优二分检索树问题

通过动态规划选择树结构, 使得:

- 成功检索 (命中关键字) 和
- 不成功检索 (未命中) 的**加权平均比较次数最少**



**Input:** 成功概率数组  $p[1..n]$ , 失败概率数组  $q[0..n]$ , 关键字数量  $n$   
**Output:** 期望代价矩阵  $e[1..n+1][0..n]$ , 根节点矩阵  $root[1..n][1..n]$

```

1 for  $i \leftarrow 1$  to  $n+1$  do
2    $e[i][i-1] \leftarrow q[i-1]$ ;
3    $w[i][i-1] \leftarrow q[i-1]$ ;
4 end
5 for  $\ell \leftarrow 1$  to  $n$  do
6   for  $i \leftarrow 1$  to  $n-\ell+1$  do
7      $j \leftarrow i+\ell-1$ ;
8      $e[i][j] \leftarrow \infty$ ;
9      $w[i][j] \leftarrow w[i][j-1] + p[j] + q[j]$ ;
10    for  $r \leftarrow i$  to  $j$  do
11       $t \leftarrow e[i][r-1] + e[r+1][j] + w[i][j]$ ;
12      if  $t < e[i][j]$  then
13         $e[i][j] \leftarrow t$ ;
14         $root[i][j] \leftarrow r$ ;
15      end
16    end
17  end
18 end
19 return  $(e, root)$ ;

```

**Algorithm 20:** Optimal-BST

$$\min \left( \sum_{i=1}^n p_i \cdot (\text{depth}(a_i) + 1) + \sum_{j=0}^n q_j \cdot \text{depth}(E_j) \right)$$

$p_i$  : 关键字  $a_i$  的命中概率  
 $q_j$  : 区间  $E_j$  (失败检索区间) 的未命中概率  
 $\text{depth}(\cdot)$  : 结点在树中的深度

$$C(i, j) = \min_{i \leq k \leq j} \{C(i, k-1) + C(k+1, j) + W(i, j)\}$$

## 4.10 投资问题

给定  $m$  元资金和  $n$  个投资项目, 每个项目的效益函数为  $f_i(x)$ , 表示投入  $x$  元到第  $i$  个项目的收益。目标是通过资金分配最大化总效益:

$$\max \sum_{i=1}^n f_i(x_i) \quad \text{s.t.} \quad \sum_{i=1}^n x_i = m, \quad x_i \in \mathbb{N}$$

定义状态函数:

$F_k(x)$  = 将  $x$  元分配给前  $k$  个项目的最大效益

递推关系与边界条件:

$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\}, \quad k > 1$$

$$F_1(x) = f_1(x)$$

## 4.11 本章作业

### 4.11.1 改进 floyd 算法

请修改求解所有点对的 Floyd 算法, 使它可以检测到负权圈 (一个负权圈是它的总长度为负的圈)。

- **负权圈问题:** 当图中存在总权重为负的环路时, Floyd 算法会陷入无限更新状态。例如, 若路径  $i \rightarrow j$  包含负权圈, 每次绕行该圈都会使总路径权重减小, 导致算法无法收敛。

数学表现为:

$$\exists \text{环} C : \sum_{e \in C} w(e) < 0 \Rightarrow \lim_{k \rightarrow \infty} dist^{(k)}[i][j] = -\infty$$

- **最优子结构破坏:** 动态规划要求子问题的最优解能构成全局最优解。负权圈使得通过无限绕行可不断降低路径权重, 导致:

局部最优  $\nrightarrow$  全局最优

$$dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j]) \quad (\text{不收敛})$$

**改进思路:** 在算法完成后检查距离矩阵对角线元素, 满足:

$$\exists i \in V : dist[i][i] < 0 \Rightarrow \text{存在包含顶点 } i \text{ 的负权圈}$$

### 4.11.2 有向图的所有点对最短路径问题

设  $G$  是一个有向图, 其中每一条边有一个非负的长度, 如果两点之间无边, 令其长度为无穷。求图中每个顶点到其他顶点的最短路径。请证明该问题满足最优子结构性质。

假设最短路径问题不满足最优子结构性质, 则存在以下情形:

1. **假设:** 设  $p = \langle v_1, v_2, \dots, v_k \rangle$  是从  $v_1$  到  $v_k$  的最短路径, 但存在子路径  $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$  不是  $v_i$  到  $v_j$  的最短路径。
2. **构造矛盾:**
  - 存在更优路径  $p'_{ij}$  满足  $w(p'_{ij}) < w(p_{ij})$
  - 构造新路径  $p' = \langle v_1, \dots, v_{i-1} \rangle \circ p'_{ij} \circ \langle v_{j+1}, \dots, v_k \rangle$
  - 计算总权重:

$$w(p') = w(p) - w(p_{ij}) + w(p'_{ij}) < w(p)$$

3. **矛盾结论:** 这与  $p$  是最短路径的初始假设矛盾, 故原命题得证。

## 4.11.3 k-shattering 问题

**问题定义** 给定边带权值的完全二叉树  $T = (V, E, w)$ , 其中  $w : E \rightarrow \mathbb{R}^+$ 。一个  $k$ -shattering 集合是边集  $S \subseteq E$  满足:

- 删除  $S$  后,  $T$  被划分为  $m$  棵子树  $\{T_1, \dots, T_m\}$
- 每棵子树  $T_i$  的节点数  $|V(T_i)| \leq k$

求使  $\sum_{e \in S} w(e)$  最小的  $k$ -shattering 集合。

```

1 Function kShatteringDP( $T, w, k$ )
2    $n \leftarrow |V(T)|$ ;
3    $\text{dp}[1..n][1..k] \leftarrow \infty$ ;
4   for  $i \leftarrow n$  downto  $\lfloor n/2 \rfloor + 1$  do
5     for  $m \leftarrow 1$  to  $k$  do
6        $\text{dp}[i][m] \leftarrow 0$ ; // 叶子节点初始化
7     end
8   end
9   for  $i \leftarrow \lfloor n/2 \rfloor$  downto 1 do
10    for  $j \leftarrow 1$  to  $k$  do
11      if  $j = 1$  then
12         $\text{min\_left} \leftarrow \min_{1 \leq a \leq k} \text{dp}[2i][a]$ ;
13         $\text{min\_right} \leftarrow \min_{1 \leq b \leq k} \text{dp}[2i+1][b]$ ;
14         $\text{dp}[i][1] \leftarrow w(i, 2i) + w(i, 2i+1) + \text{min\_left} + \text{min\_right}$ ;
15      else
16         $\text{option1} \leftarrow w(i, 2i) + \min_{1 \leq a \leq k} \text{dp}[2i][a] + \text{dp}[2i+1][j-1]$ ;
17         $\text{option2} \leftarrow w(i, 2i+1) + \min_{1 \leq a \leq k} \text{dp}[2i+1][a] + \text{dp}[2i][j-1]$ ;
18         $\text{option3} \leftarrow \min_{1 \leq a \leq j-2} (\text{dp}[2i][a] + \text{dp}[2i+1][j-a-1])$ ;
19         $\text{dp}[i][j] \leftarrow \min(\text{option1}, \text{option2}, \text{option3})$ ;
20      end
21    end
22  end
23  return  $\min_{1 \leq j \leq k} \text{dp}[1][j]$ ;

```

**Algorithm 22:** 最小代价  $k$ -shattering 算法

- 状态定义:  $\text{dp}[i][m]$  表示以  $i$  为根的子树保留  $m$  个节点的最小代价
- 边界条件:

$$\text{dp}[i][m] = 0 \quad \forall i > \lfloor n/2 \rfloor$$

- 状态转移:

$$\text{dp}[i][j] = \begin{cases} w_{i,2i} + w_{i,2i+1} + \min_a \text{dp}[2i][a] + \min_b \text{dp}[2i+1][b] (\text{只有根}) & j = 1 \\ \min \begin{cases} w_{i,2i} + \min \text{dp}[2i][a] + \text{dp}[2i+1][j-1] (\text{断左}) \\ w_{i,2i+1} + \min \text{dp}[2i+1][a] + \text{dp}[2i][j-1] (\text{断右}) \\ \min_a (\text{dp}[2i][a] + \text{dp}[2i+1][j-a-1]) (\text{全留}) \end{cases} & j > 1 \end{cases}$$

- 时间复杂度:  $O(nk^2)$

最小代价:  $\min(16, 9) = 9$ , 对应删除边集  $\{\langle b, d \rangle, \langle b, e \rangle, \langle a, c \rangle\}$ 。

## 5 chapter5: 贪心算法

贪心算法很难有具体可行的方法对其适用性进行判定, 但是一般具有 2 个重要的性质: **贪心选择性**和**最优子结构性**。

所谓贪心选择性是指所求问题的整体最优解可以通过一系列局部最优的选择, 即贪心选择来达到。这意味着每一步的最优选择不会影响未来的选择, 即当前的选择只依赖于当前的状态, 而不依赖于未来的状态。

**对于一个约束优化问题:**

1. **可行解**: 如果决策序列是关于一个问题的完整解的形式, 且不破坏问题的约束条件, 则称它为该问题的可行解。
2. **部分可行解**: 如果一个决策序列是问题的完整解形式的一部分, 且满足约束条件, 则称它为部分可行解。
3. **不可行的(部分)解**: 不满足问题约束条件的完整或部分解形式的决策序列, 称为不可行的(部分)解。
4. **最优解**: 使目标函数值取最优值的可行解, 称为最优解。
5. **贪心解**: 通过贪心算法得到的可行解, 称为贪心解。
6. **贪心解性质**: 贪心解不一定是最优解。

## 5.1 贪心算法思想

**Input:** 集合  $A$ , 元素数量  $n$   
**Output:** 解集  $solution$

```

1 贪心算法一般框架
2 Algorithm GREEDY( $A, n$ )
3  $solution \leftarrow \emptyset$ 
4 for  $i \leftarrow 1$  to  $n$  do do
5      $x \leftarrow \text{SELECT}(A)$ 
6     if  $\text{FEASIBLE}(solution, x)$  then
7          $solution \leftarrow \text{UNION}(solution, x)$ 
8     end
9     end if
10 end
11 return ( $solution$ )

```

**Algorithm 23:** 贪心算法通用框架

1. **贪心选择性质:** 通过 SELECT 函数保证每一步选择都是当前最优, 且该选择最终能导向全局最优解 (需进行数学证明)
2. **最优子结构:** FEASIBLE 函数确保解集的逐步扩展不破坏问题结构。
3. **终止条件:** 循环  $n$  次确保覆盖所有候选元素, 最终解集  $solution$  必然完备。

贪心算法产生全局最优解的最重要因素是选择策略, 采用的贪心准则/贪心度量。

## 5.2 贪心算法举例

### 5.2.1 Dijkstra 算法

- $\lambda[v]$ : 算法中顶点  $v$  的当前距离标记 (对应伪代码中的  $dist[v]$ )
- $\delta[v]$ : 从源点 1 到  $v$  的真实最短路径长度
- $Y$ : 未处理的顶点集合 (对应优先队列  $Q$ )
- $X$ : 已处理的顶点集合 ( $X = V \setminus Y$ )

在 Dijkstra 算法中, 当顶点  $y$  在第 8 步 (ExtractMin 操作) 被选中且  $\lambda[y]$  有限时, 有:

$$\lambda[y] = \delta[y]$$

第一个离开  $Y$  的顶点是源点 1, 初始化时:

$$\lambda[1] = \delta[1] = 0 \quad (\text{显然成立})$$

假设对于所有在  $y$  之前离开  $Y$  的顶点  $x$ , 都有:

$$\lambda[x] = \delta[x]$$

下证对  $y$  也成立。

1. 路径存在性：由于  $\lambda[y]$  有限，必存在从 1 到  $y$  的路径  $P$ ，其长度为  $\lambda[y]$ 。
2. 关键路径构造：设  $\pi = 1 \rightarrow \dots \rightarrow x \rightarrow w \rightarrow \dots \rightarrow y$  是从 1 到  $y$  的真实最短路径（ $\pi$  具有子问题最优性，否则一定存在一个更短的子路径优化  $\pi$ ），其中：

- $x$  是路径  $\pi$  上最后一个在  $y$  之前离开  $Y$  的顶点
- $w$  是  $x$  的后继节点

3. 不等式链：

$$\begin{aligned}
 \lambda[y] &\leq \lambda[w] \quad (\text{因为 } y \text{ 先于 } w \text{ 被选中}) \\
 &\leq \lambda[x] + \text{length}(x, w) \quad (\text{松弛操作保证}) \\
 &= \delta[x] + \text{length}(x, w) \quad (\text{归纳假设}) \\
 &= \delta[w] \quad (\pi \text{ 的子路径最优性}) \\
 &\leq \delta[y] \quad (\text{非负权重下子路径} \leq \text{全路径})
 \end{aligned}$$

4. 等号成立：由于  $\lambda[y]$  是某条路径的实际长度，而  $\delta[y]$  是最短长度，故：

$$\lambda[y] \geq \delta[y]$$

结合上述不等式链，最终得：

$$\lambda[y] = \delta[y]$$

- 子路径不等式：保证  $\delta[w] \leq \delta[y]$ （否则  $x \rightarrow w \rightarrow y$  可能比全局最短路径更长）
- 松弛有效性：确保  $\lambda[w] \leq \lambda[x] + \text{length}(x, w)$  不会被负权边破坏
- 反例：若  $\text{length}(x, w) < 0$ ，可能导致  $\delta[w] > \delta[y]$ ，破坏归纳链条

朴素的数组实现的 dijkstra 算法的复杂度是  $O(n^2 + m)$ ，如果用堆进行优化，时间复杂度是  $O(n + m) \log n$  ( $n$  是顶点数量， $m$  是边数量)。如果是稠密图的话，会使得堆优化速度更慢，所以在稠密图（邻接矩阵存储）情况下，应该用前者。

### 5.2.2 Kruskal 算法

对于 Kruskal 算法，算法的时间复杂度取决于排序步骤，所以是  $O(m \log m)$ 。

- $G = (V, E)$ ：带权无向连通图
- $T_{\text{mst}}$ ：任意一棵最小生成树
- $A_k$ ：算法在第  $k$  步迭代后加入的边集（初始化  $A_0 = \emptyset$ ）

$\forall k \geq 0, A_k \subseteq T_{\text{mst}}$ （即  $A_k$  总是某个 MST 边的子集）

1. 基础步骤 ( $k = 0$ ):

$A_0 = \emptyset$ ，空集是任何集合的子集，显然  $A_0 \subseteq T_{\text{mst}}$ 。

## 2. 归纳步骤:

假设  $A_k \subseteq T_{\text{mst}}$ , 设  $e = (u, v)$  是第  $k+1$  步选择的边:

- 根据算法:  $e$  是当前权重最小且不产生环的边 (即  $e$  连接  $A_k$  中两个不同连通分量)
- 根据安全边引理:  $A_k \cup \{e\}$  是某个 MST 边的子集

$\therefore A_{k+1} = A_k \cup \{e\} \subseteq T'_{\text{mst}}$  ( $T'_{\text{mst}}$  为新的最小生成树)

## 3. 终止条件:

当  $|A_{\text{final}}| = |V| - 1$  时:

- $A_{\text{final}}$  无环 (算法保证)
- $A_{\text{final}}$  连通所有顶点 (否则存在连接不同分量的安全边, 算法不会终止)
- $A_{\text{final}} \subseteq T_{\text{mst}}$  且  $|A_{\text{final}}| = |T_{\text{mst}}|$

$\Rightarrow A_{\text{final}}$  即为一棵最小生成树

下面用反证法证明**安全边引理**

假设存在权重最小的跨分量边  $e \notin T_{\text{mst}}$  ( $T_{\text{mst}}$  是包含  $A$  的最小生成树)

1. 将  $e$  加入  $T_{\text{mst}}$ , 形成唯一环  $C$  (因树加边必成环)

2. 环  $C$  上必存在边  $e' = (x, y)$  满足:

- $e' \neq e$
- $e'$  连接  $A$  的不同连通分量 (因  $e$  连接两个分量)
- $w(e') \geq w(e)$  (因  $e$  是权重最小的跨分量边)

3. 构造新生成树:  $T_{\text{new}} = T_{\text{mst}} \cup \{e\} \setminus \{e'\}$

- 总权重:  $w(T_{\text{new}}) = w(T_{\text{mst}}) + w(e) - w(e') \leq w(T_{\text{mst}})$
- 当  $w(e) < w(e')$  时:  $w(T_{\text{new}}) < w(T_{\text{mst}})$  (与  $T_{\text{mst}}$  最小性矛盾)
- 当  $w(e) = w(e')$  时:  $T_{\text{new}}$  也是最小生成树, 且包含  $A \cup \{e\}$

## 5.2.3 Prim 算法

Prim 算法一般复杂度为  $\Theta(m + n^2)$ , 可以被堆优化为  $O(m \log n)$ .

给出一个含权无向图  $G = (V, E)$ , 其中  $|V| = n$ ,  $|E| = m$ :

- 算法 MST 可在  $O(m \log n)$  时间内找出最小生成树
- **稠密图特例**: 若存在  $\varepsilon > 0$  使得  $m \geq n^{1+\varepsilon}$ , 则时间复杂度可改进为  $O(\frac{m}{\varepsilon})$

设  $\varepsilon$  为固定常数, 其技术意义体现在:

$$\frac{m \log n}{m/\varepsilon} = \varepsilon \log n \Rightarrow \begin{cases} \text{当 } m = n^{1+\varepsilon} \text{ 时, 加速比为 } \Theta(\varepsilon \log n) \\ \varepsilon = \Omega(1) \text{ 时, } O(\frac{m}{\varepsilon}) \text{ 严格优于 } O(m \log n) \end{cases} \quad (2)$$

1. 稠密度控制:  $\varepsilon$  量化图的稠密程度

$$m = n^{1+\varepsilon} \implies \text{边数 } m \text{ 远超顶点数 } n$$

2. 复杂度改进: 当  $\varepsilon$  为常数时

$$O\left(\frac{m}{\varepsilon}\right) = O(m) \quad (\text{线性时间优于 } O(m \log n))$$

3. 实际意义: 对满足  $\exists c > 0, m > n^{1+c}$  的图, 算法有显著加速

该定理的正确性建立在图论(稀疏/稠密图分类)与数据结构(斐波那契堆优化)的交叉分析上, 其核心在于发现: 当边数足够多时 ( $m \gg n$ ), 可以通过特殊数据结构设计消除对数因子。

Prim 算法可以正确处理包含负权边的图的最小生成树 (MST) 问题, 其正确性不受边权值符号的影响。

**证明.** 设  $T_{\min}$  是  $G$  的最小生成树。将  $T$  中的  $n-1$  条边按权重从小到大排列为  $e_{k_1}, e_{k_2}, \dots, e_{k_{n-1}}$ , 将  $T_{\min}$  中的边同样排列为  $e_{g_1}, e_{g_2}, \dots, e_{g_{n-1}}$ 。从头开始比较:

设第一条属于  $T$  但不属于  $T_{\min}$  的边为  $e_{k_i} = (u_i, v_i)$ , 则在  $T_{\min}$  中存在一条属于  $T_{\min}$  但不属于  $T$  的边  $e_{g_j} = (u_i, v_*)$ 。根据 Prim 算法的性质 (2) 可知:

$$w(e_{k_i}) \leq w(e_{g_j})$$

在  $T_{\min}$  中删除  $e_{g_j}$  并加入  $e_{k_i}$  得到新树  $T_{\text{new}}$ , 满足:

$$\text{cost}(T_{\min}) \geq \text{cost}(T_{\text{new}})$$

由于  $T_{\min}$  是最小生成树, 又有:

$$\text{cost}(T_{\min}) \leq \text{cost}(T_{\text{new}})$$

故必然有:

$$\text{cost}(T_{\min}) = \text{cost}(T_{\text{new}})$$

即  $T_{\text{new}}$  也是最小生成树。

重复此替换过程直至  $T_{\text{new}} = T$ , 最终证明  $T$  是最小生成树。 □

## 5.2.4 活动选择问题

你有  $n$  个活动构成的集合  $S = \{a_1, a_2, \dots, a_n\}$ , 这些活动都需要占用会议室, 且每个时刻会议室只能给一场活动提供服务。每个活动  $a_i$  的开始时间 (start time) 为  $s_i$ , 结束时间 (finish time) 为  $f_i$ , 其中  $0 \leq s_i < f_i < \infty$ 。如果活动  $a_i$  占用的时间为左闭右开区间  $[s_i, f_i)$ 。如果活动  $a_i$  和  $a_j$  的时间区间  $[s_i, f_i)$  和  $[s_j, f_j)$  没有重叠, 那么活动  $a_i$  和  $a_j$  相互兼容。活动选择问题目标是从所有活动中选出一个总时间规模最大相互兼容的活动子集。假设这些活动已经按照结束时间从小到大排序, 即:  $f_1 \leq f_2 \leq f_3 \leq \dots \leq f_n$

如果给定活动序列没有按照结束时间从小到大排序, 先进行排序, 这步预处理非常重要, 没有这步预处理, 后面的贪心策略就无从开展。(其实也可以选择最晚开始的活动, 思路与最早结束大体一致, 不再额外展开)

下面开始使用贪心策略求解, 贪心策略每一步都要作出贪心选择 (greedy choice), 这样只有一个子问题需要处理。下面会分别讨论贪心的递归算法和迭代算法。



下面证明活动选择问题具有最优子结构。

定义  $S_{ij}$  为  $a_i$  结束后  $a_j$  开始前的活动集合，定义  $A_{ij}$  为  $S_{ij}$  中规模最大且相互兼容的活动子集。 $A_{ij}$  中包含活动  $a_k$ ，和两个子问题  $A_{ik}$  和  $A_{kj}$ 。定义  $A_{ik} = A_{ij} \cap S_{ik}$  且  $A_{kj} = A_{ij} \cap S_{kj}$ ，这样有  $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$ ，所以有  $|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$ 。

用剪切粘贴方法可以证明最优解  $A_{ij}$  一定包含两个子问题  $S_{ik}$  和  $S_{kj}$  的最优解。如果你能找到一个由  $S_{kj}$  相互兼容的活动组成的集合  $A'_{kj}$ ，其中  $|A'_{kj}| > |A_{kj}|$ ，那么你就可以用  $A'_{kj}$  替换  $A_{kj}$ 。此时就能构造出规模为

$$|A_{ik}| + |A'_{kj}| + 1 > |A_{ik}| + |A_{kj}| + 1 = |A_{ij}|$$

的相互兼容的活动集合，与  $A_{ij}$  是最优解矛盾。同理也可以用于证明子问题  $S_{ik}$ 。

定义  $S_{ij}$  的最优解的规模为  $c[i, j]$ ，运用动态规划方法可以写出递归式： $c[i, j] = c[i, k] + c[k, j] + 1$ ，即：

$$c[i, j] = \begin{cases} 0 & S_{ij} = \emptyset \\ \max_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & S_{ij} \neq \emptyset \end{cases}$$

虽然我们可以用动态规划方法求解，但是我们忽略了活动选择问题的一个重要特征，这个特征可以大大提高问题求解速度。

考虑任意非空子问题  $S_k$ ，如果  $a_m$  是  $S_k$  中结束时间最早的活动，那么  $a_m$  在  $S_k$  的某个最大兼容活动子集中。

下面证明活动选择问题具有贪心选择性质：证明：定义  $A_k$  是  $S_k$  的最大兼容活动子集，且  $a_j$  是  $A_k$  中结束时间最早的活动。

若  $a_j = a_m$ ，则已证明  $a_m \in A_k$ 。

若  $a_j \neq a_m$ ，令  $A'_k = (A_k - \{a_j\}) \cup \{a_m\}$ ，则  $|A_k| = |A'_k|$ 。又  $a_j$  是  $A_k$  中结束时间最早的活动， $a_m$  是  $A'_k$  中结束时间最早的活动， $f_m \leq f_j$ 。所以  $A'_k$  是  $S_k$  的最大兼容活动子集， $a_m \in A'_k$ 。

我们每次选择结束时间最早的且与当前活动兼容的活动，重复执行这个过程直到没有剩余活动可以选择。所选择的活动的结束时间必然是严格递增的。所以我们只需要按照结束时间单调递增的顺序处理所有活动，每个活动仅需要考察一次。

**Input:** $s[1..n]$ : 活动开始时间数组 $f[1..n]$ : 活动结束时间数组 (已按  $f_i$  升序排列) $k$ : 当前活动索引 (初始调用时  $k = 0$ ) $n$ : 总活动数**Output:** 最大兼容活动子集

```

1 function RECURSIVE-ACTIVITY-SELECTOR( $s, f, k, n$ )
2    $m \leftarrow k + 1$ 
3   while  $m \leq n$  and  $s[m] < f[k]$  do
4      $m \leftarrow m + 1$  // 在  $S_k$  中寻找第一个兼容活动
5   end
6   if  $m \leq n$  then
7     return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
8   end
9   else
10    return  $\emptyset$ 
11  end
12 // 初始调用方式
13 RECURSIVE-ACTIVITY-SELECTOR( $s, f, 0, n$ )

```

**Algorithm 25:** 递归式活动选择算法

当然也可以用动态规划的思想实现, 同时用二分的思路对其进行优化, 时间复杂度仍为  $O(n \log n)$ :

思考活动选择问题的一个变型: 每个活动  $a_i$  除了开始时间和结束时间外, 还有一个权值  $v_i$ 。目标不再是求规模最大的活动子集, 而是求权值总和最大的活动子集, 也就是选择一个活动集合  $A$ , 使得  $\sum_{a_k \in A} v_k$  最大化。设计一个多项式时间的算法求解该问题。

其实就只需要修改一下动态规划算法的比较项就可以了, 不再做更多展开。

**5.2.5 区间图着色问题**

假设有一组活动, 将它们安排到一些讲堂, 任意活动都可以在任意讲堂进行, 我们希望用最少的教室完成所有活动, 设计一个贪心算法求每个活动所在讲堂进行分配。

这个问题可以视为区间图着色问题 (interval-graph coloring problem)。区间图着色问题: 构造一个区间图, 顶点表示给定的活动, 边连接不兼容的活动。要求用最少的颜色对顶点进行着色, 相邻顶点的颜色不能相同。

一个朴素的方法是, 每当有新活动时, 就遍历所有讲堂, 查看是否有空闲讲堂。如果没有空闲讲堂, 就新开一个讲堂。为了方便选出空闲讲堂, 我们可以使用优先队列。

算法执行步骤如下:

按照开始时间对活动进行排序。初始化一个小顶堆, 按照结束时间排序, 堆顶活动为最早结束的活动。检查每一个活动: 如果堆为空, 开辟一个新讲堂, 选好讲堂后将活动加入堆中, 同时将该活动记录到所在讲堂举行的活动名单中。如果堆不为空, 和堆顶活动进行比较, 如果堆顶活动的结束时间小于或等于当前活动的开始时间, 将堆顶活动出队, 并将当前活动放在堆顶活动所在讲堂进行, 否则开辟一个新讲堂。选好讲堂后将活动加入堆中, 同时将该活动记录到所在讲堂举行的活动名单中。所有活动检查结

束后，输出所有讲堂举行的活动名单。

### 5.2.6 最小延迟调度问题

给定  $n$  个作业，每个作业  $J_i$  有一个处理时间  $t_i$  和截止时间  $d_i$ 。调度方案需为作业排列一个顺序，使得所有作业的延迟（完成时间减去截止时间）的最大值最小化。形式化定义为：

- 作业顺序  $\sigma = (J_{\sigma(1)}, \dots, J_{\sigma(n)})$  的完成时间  $C_{\sigma(i)} = \sum_{k=1}^i t_{\sigma(k)}$ 。
- 延迟  $L_{\sigma(i)} = \max(0, C_{\sigma(i)} - d_{\sigma(i)})$ 。
- 目标是最小化  $L_{\max} = \max_i L_{\sigma(i)}$ 。

**Input:** 任务集合  $S = \{J_1, J_2, \dots, J_n\}$ ，其中每个任务  $J_i$  有处理时间  $t_i$  和截止时间  $d_i$

**Output:** 任务完成时间数组  $f$

```

1 按照截止时间  $d_i$  对任务进行升序排序，使得  $d_1 \leq d_2 \leq \dots \leq d_n$ ;
2 初始化完成时间数组:  $f(1) \leftarrow 0$ ;
3 初始化计数器:  $i \leftarrow 2$ ;
4 while  $i \leq n$  do
5    $f(i) \leftarrow f(i-1) + t_{i-1}$ ;
6    $i \leftarrow i + 1$ ;
7 end
8 return  $f$ ;
```

#### Algorithm 28: 最小延迟调度算法

显然，所有无逆序且无空闲时间的调度具有相同的最大延迟。即对于满足  $d_{\sigma(1)} \leq \dots \leq d_{\sigma(n)}$  的调度  $\sigma$ ，其  $L_{\max}$  仅取决于任务顺序，与相同截止时间任务的内部排列无关。

下面证明：按  $d_i$  升序排列（EDD 策略）的调度  $\sigma^*$  是最优解。

若  $d_i > d_j$  但  $i$  排在  $j$  前，则称相邻任务对  $(J_i, J_j)$  为**逆序对**。

对任意含逆序对  $(J_i, J_j)$  的调度  $\sigma$ ：

$$\text{交换前延迟: } L_i = C + t_i - d_i, \quad L_j = C + t_i + t_j - d_j$$

$$\text{交换后延迟: } L'_j = C + t_j - d_j, \quad L'_i = C + t_j + t_i - d_i$$

其中  $C$  为  $J_i$  的开始时间。由于  $d_i > d_j$  且  $t_j > 0$ ：

$$\max(L'_j, L'_i) \leq \max(L_i, L_j)$$

通过有限次相邻交换可消除所有逆序对：

- 每次交换不增加  $L_{\max}$
- 最终得到无逆序的 EDD 调度
- 由引理 1 知此调度最优

### 5.2.7 分数背包问题

给出  $n$  个大小为  $s_1, s_2, \dots, s_n$ , 值为  $v_1, v_2, \dots, v_n$  的物品, 并设背包的容量为  $C$ 。要求找到非负实数  $x_1, x_2, \dots, x_n$ , 使得在满足不超过背包总容量的约束条件下, 装进背包物品部分总价值最大。

即, 最大化  $\sum_{i=1}^n x_i v_i$  且满足  $\sum_{i=1}^n x_i s_i \leq C$

选择单位大小的价值最大的物品即可, 不过多展开了 (过于简单)

### 5.2.8 带期限作业排序问题

问题: 设有一个单机系统、无其它资源限制且每个作业运行相等时间, 不妨假定每个作业运行 1 个单位时间。现有  $n$  个作业, 每个作业都有一个截止期限  $d_i > 0$ ,  $d_i$  为整数。如果作业能够在截止期限之内完成, 可获得  $p_i > 0$  的收益。问题要求得到一种作业调度方案, 该方案给出作业的一个子集和该作业子集的一种排列, 使得若按照这种排列次序调度作业运行, 该子集中的每个作业都能如期完成, 并且能够获得最大收益。

思路: 将输入的每个作业按收益从大到小进行排序, 用一个数组  $vis$  表示某时刻是否已经有作业正在运行,  $vis[i] = 1$ , 表示时间  $i$  被占用。然后从第 1 个作业开始往后搜索, 将该作业安排到  $vis[d]$  时间运行, 如果  $d$  已经有安排, 将从  $d - 1$  开始往前搜索, 直到找到一个未被占用的时间点。如果找不到空时间点, 则跳过此作业, 继续向后搜索直到所有作业都搜索完。

**Input:** 作业集合  $J = \{J_1, \dots, J_n\}$ , 其中每个作业  $J_i$  包含:

- 编号  $id_i$
- 收益  $p_i > 0$
- 截止时间  $d_i > 0$  (整数)

**Output:** 能在截止时间前完成的作业子集及其排列顺序

```

1 按照收益  $p_i$  对作业进行降序排序;
2 初始化时间槽数组  $vis[1..max\_d]$  全为 0;
3 初始化结果列表  $res \leftarrow \emptyset$ ;
4 foreach 作业  $J_i \in J$  (按已排序顺序) do
5      $found \leftarrow false$ ;
6     if  $vis[J_i.d] == 0$  then
7          $vis[J_i.d] \leftarrow 1$ ;
8          $res.push\_back(J_i.id)$ ;
9          $found \leftarrow true$ ;
10    end
11    else
12        for  $t \leftarrow J_i.d - 1$  downto 1 do
13            if  $vis[t] == 0$  then
14                 $vis[t] \leftarrow 1$ ;
15                 $res.push\_back(J_i.id)$ ;
16                 $found \leftarrow true$ ;
17                break;
18            end
19        end
20    end
21    if not found then
22        跳过作业  $J_i$ ;
23    end
24 end
25 return  $res$ ;

```

**Algorithm 29:** 最大收益作业调度算法

**定理 3.** 按效益值降序排列的贪心调度算法总能得到最优解。

**证明.** 采用交换论证法, 分三部分证明:

**1. 基本设定:**

- 设最优解作业集为  $I$ , 贪心解为  $J$
- 若  $I = J$  则得证, 故只需考虑  $I \neq J$  的情况
- 此时必有  $I \not\subseteq J$  (否则  $I$  非最优) 且  $J \not\subseteq I$  (否则  $J$  更优)

- 存在  $a \in J - I$  和  $b \in I - J$ , 且由贪心法性质有  $p_a \geq p_b$

## 2. 调度表调整:

1. 设  $S_J$  和  $S_I$  分别为  $J$  和  $I$  的可行调度表
2. 对公共作业  $i \in I \cap J$ , 调整调度表使得:
  - 若在  $S_J$  中  $i$  于  $[t, t+1]$  执行, 在  $S_I$  中于  $[t', t'+1]$  执行
  - 当  $t < t'$  时: 将  $S_J$  中  $[t', t'+1]$  的作业与  $i$  交换
  - 当  $t' < t$  时: 对  $S_I$  做对称调整
3. 最终得到新调度表  $S'_J$  和  $S'_I$ , 使公共作业执行时间一致

## 3. 最优性保持:

- 在调整后的  $S'_I$  中, 将作业  $b$  替换为  $a$ , 形成新解  $I' = (I - \{b\}) \cup \{a\}$
- 由于  $p_a \geq p_b$ , 有  $\sum_{i \in I'} p_i \geq \sum_{i \in I} p_i$
- 重复此过程可将  $I$  逐步转换为  $J$  而不降低总效益

因此  $J$  必为最优解。 □

**定理 4.** 作业集合  $J$  可行的充要条件: 存在按  $d_i$  非降序排列的调度序列  $\sigma$ , 使得  $\forall i \in J$  都在  $d_i$  前完成。

证明. 必要性由定义显然。充分性证明:

- 设  $\sigma$  为按  $d_i$  非降序排列的调度
- 若存在作业  $i_k$  在  $\sigma$  中第  $k$  个执行且完成时间  $k > d_{i_k}$
- 则前  $k$  个作业的  $d_{i_j} \leq d_{i_k} < k$ , 导致至少一个作业必然超期
- 矛盾, 故  $\sigma$  必为可行调度 □

## 5.2.9 黑白点最大匹配问题

**Input:** 黑点集合  $B = \{b_1, \dots, b_n\}$ , 其中  $b_i = (x_i^B, y_i^B)$   
 白点集合  $W = \{w_1, \dots, w_n\}$ , 其中  $w_j = (x_j^W, y_j^W)$   
**Output:** 最大匹配数  $count$

```

1 预处理阶段: 将黑点  $B$  按  $x^B$  升序排列 ( $x$  相同则按  $y^B$  升序);
2 将白点  $W$  按  $x^W$  降序排列 ( $x$  相同则按  $y^W$  降序);
3 匹配阶段:  $count \leftarrow 0$ ;
4  $i \leftarrow 1, j \leftarrow 1$ ;
5 while  $i \leq n$  且  $j \leq n$  do
6   if  $x_i^B \geq x_j^W$  且  $y_i^B \geq y_j^W$  then
7      $count \leftarrow count + 1$ ;
8      $i \leftarrow i + 1$ ; // 消耗当前黑点
9      $j \leftarrow j + 1$ ; // 消耗当前白点
10  end
11  else
12     $j \leftarrow j + 1$ ; // 跳过无法匹配的白点
13  end
14 end
15 return  $count$ ;

```

Algorithm 30: 黑白点最大匹配算法

证明. 我们通过以下步骤证明:

- 设最优匹配为  $M^*$ , 其匹配数为  $OPT$
- 设算法得到的匹配为  $M$ , 匹配数为  $|M|$
- 定义特征函数  $\phi(M)$  为匹配中黑点的字典序排列

对于任意最优匹配  $M^*$ , 存在匹配  $M'$  使得:

1.  $|M'| = |M^*|$
2.  $\phi(M')$  与算法输出  $M$  的字典序相同 (可以通过调整使得相同)

假设存在更优匹配  $M^*$ , 取第一个差异点:

1. 设算法第  $k$  步选择  $(b_i, w_j)$
2. 而  $M^*$  选择  $(b_i, w_{j'})$  或  $(b_{i'}, w_j)$  (如果不是则通过交换进行修正)
3. 情况 1:  $w_{j'} \neq w_j$

由于算法优先选最大的  $w_j$

$$\begin{aligned} \Rightarrow x_j^W &\geq x_{j'}^W \\ y_j^W &\geq y_{j'}^W \end{aligned}$$

可交换  $w_j$  和  $w_{j'}$  而不减少匹配数

4. 情况 2:  $b_{i'} \neq b_i$ 

算法选择最小的  $b_i$

$$\Rightarrow x_i^B \leq x_{i'}^B$$

$$y_i^B \leq y_{i'}^B$$

用  $b_i$  替换  $b_{i'}$  可保持可行性

上述交换均会使  $M^*$  的特征向量  $\phi(M^*)$  更接近算法结果  $M$ , 与  $M^*$  的最优性假设矛盾。  $\square$

## 5.2.10 过河问题

Oliver 和  $N - 1$  个朋友共  $N$  人需要从河东岸渡到西岸。他们只有一艘小船, 每次最多可载两人。当两个人  $i$  和  $j$  一起渡河时, 所需时间为  $\max(T_i, T_j)$ 。求所有人过河的最少总时间。

很明显的贪心, 主要讨论两种情况: 1、让速度最快的往返接送, 每次送两个人所用时间为两倍最快的人所用的时间加上两个最慢的人的所用的时间之和。即  $a[n] + a[1] + a[n - 1] + a[1]$ ;

2、先让两个最快的人过河, 再让第二快 (谁将船送回都不影响) 的人送回船, 让两个岁慢的人过河, 再将留在河对岸的速度最快的人把船送回。所用时间为  $a[2] + a[2] + a[n] + a[1]$ ;

## 5.2.11 树上的最小顶点覆盖

给出一种有效的贪心算法, 该算法能在线性时间内找到一棵树中的一个最优顶点覆盖。

表示方案一采用定长数组存储每个结点的孩子结点, 设给定一棵树  $T$ , 其根结点为  $T.root$ , 阶数为  $T.degree$ 。对于每个结点  $x$ , 具有颜色属性  $x.color$  和包含  $T.degree$  个孩子结点指针的数组  $x.children$ 。该方案借鉴红黑树思想进行结点着色, 规定每个结点非红即黑, 所有空结点默认为黑色, 且红色结点的所有孩子结点必须为黑色。在贪心算法实现中, 首先将所有孩子结点为空结点的非空结点标记为红色, 然后采用自底向上的遍历策略。具体实现采用方法一的后序遍历方式, 其着色规则为: 所有空结点保持黑色; 对于非空结点, 若其所有孩子结点均为黑色则将该结点着为红色, 否则着为黑色。通过后序遍历过程, 当所有结点完成着色时, 最终所有黑色结点的集合  $C$  即构成该树的最优顶点覆盖解。该方法的正确性依赖于树的后序遍历特性与颜色约束规则的结合, 确保在多项式时间内获得满足条件的顶点覆盖。

## 6 chapter6: 图的遍历

## 6.1 割点

## 6.2 强连通分量

## 6.3 A\* 算法

## 7 chapter7: NP 完全问题

在计算理论中, Church-Turing 论题奠定了可计算性的理论基础, 它明确指出一个问题的可计算性取决于其能否在图灵机上通过有限次计算获得正确结果。这一理论框架将人类面临的所有问题划分为可计算与不可计算两大类, 其中不可计算问题 (或称不可解问题) 具有本质上的局限性——无论投入多少



计算时间与资源，都无法通过计算机获得解决方案。值得注意的是，这里的“有限次计算”是一个相对宽松的条件，主要强调**计算过程必须具有终止性**。

Cook-Karp 论题在此基础上进一步细化了可计算问题的分类标准，引入了实际可计算性的重要概念。该论题将**多项式时间**计算作为实际可计算性的判定标准，即在图灵机上通过多项式时间步数能获得正确结果的问题才被视为实际可计算的。这一划分产生了实际可计算问题与难解问题的新分类：前者具有可行的算法解决方案，后者虽然理论上仍可通过计算机求解（如采用指数时间算法），但由于缺乏多项式时间算法而被认为在实际应用中难以处理。这两个论题共同构成了计算复杂性理论的重要基石，为计算机科学中问题可解性的研究提供了理论基础。

可计算的问题需要满足：

1. 必须是存在解的问题。（例如：是否有外星生命就不行）
2. 可以通过计算机求解的问题，可以用一系列机械的数学步骤（算法）来求解。（例如：证明一个整数  $n$  的阶乘的末尾数只能为 0,1,4,5,6,9 种的一个（费马小定理），这个问题是有解的，但目前没有找到有效的证明方法，也不属于可计算问题）
3. 一些需要人类创造力、想象力等方面的问题，也不属于可计算问题。

算法的复杂性是指解决特定问题时，具体算法执行所需的时间量度，这反映了算法本身的效率特性。与之相对，问题的复杂性则表征问题本身固有的难度级别，这是问题与生俱来的本质属性。

从根本上看，问题的复杂度是问题自身内在的、不可改变的特性；而算法的复杂度则受到多重现实条件的制约，既包括逻辑层面的算法设计能力，也包含物理层面的实现条件限制。这种区分揭示了计算理论中问题本质属性与解决方案特性之间的重要分野。

求解一个问题的好的算法的时复杂度函数应该恰好等于问题的计算复杂度。

算法最优性指的是在解决特定问题的所有可能算法中，效率最高的算法。这种最优性可以从两个重要维度进行衡量：最坏情况下最优和平均情况下最优。

**最坏情况下最优算法**是指在解决某问题的所有算法中，不存在其他算法在最坏情况下的时间复杂性比该算法更低。这意味着该算法在最不利输入情况下仍能保持最佳性能。

**平均情况下最优算法**则是指在解决某问题的所有算法中，不存在其他算法在平均情况下的时间复杂性比该算法更低，这表明该算法在典型输入情况下具有最优表现。这两种最优性标准为算法性能评估提供了重要理论框架。

## 7.1 计算复杂度的分析

### 7.1.1 平凡下界

有的问题必须扫描完所有输入才能得到解，算法总要得到所有输入，输出才能解决好，因此算法的输入规模/输出规模是它的平凡下界。

### 7.1.2 计数所需最少运算数

找最大问题：在  $n$  个不同的数中找最大的元素，基本元素是元素的比较，则对于任何算法在最坏情况下至少要做  $n - 1$  次比较。

### 7.1.3 决策树 (Decision Tree)

决策树是一种重要的算法分析工具，它以二叉树的形式直观地展现了算法执行过程。这种特殊的树结构中，每个内部节点都代表着算法执行过程中的一次关键运算操作，比如常见的比较运算；而叶子节点则对应着算法最终输出的结果。值得注意的是，在某些特定问题中，算法的输出结果也可能出现在内部节点上，而不仅限于叶子节点。

当我们分析一个具体算法的决策树时，可以清晰地看到算法处理输入实例的完整流程。算法执行总是从决策树的根节点开始，沿着特定的路径向下遍历。在这个过程中，每个节点都会执行一次基本操作，并根据操作结果（如小于、等于或大于）决定下一步的执行路径。这种执行过程实际上就对应着从根节点到某个叶子节点或内部节点的一条完整路径，完整地记录了算法处理该输入实例的整个过程。

从算法分析的角度来看，决策树为我们提供了深入理解算法行为的窗口。对于给定的算法和固定大小的输入规模  $n$ ，不同的输入实例会导致算法在决策树的不同节点终止。这些终止节点实际上将输入实例进行了分类，每个节点都对应着一类具有相同特征的输入。通过分析决策树中节点的总数，特别是叶子节点的数量，我们可以准确地把握算法需要处理的不同输入类别的数量，这对评估算法的复杂度具有重要意义。决策树的这种特性使其成为算法设计和分析中不可或缺的重要工具。

对于给定的  $n$ ，任何通过比较对  $n$  个元素排序的算法的决策树的深度至少是  $\lceil \log n! \rceil$ 。

$$\begin{aligned}\log n! &= \sum_{j=1}^n \log j \geq \int_1^n \log x \, dx = \log e \int_1^n \ln x \, dx \\ &= \log e (n \ln n - n + 1) \\ &= n \log n - n \log e + \log e \\ &\approx n \log n - 1.5n\end{aligned}$$

存在多项式时间算法的问题是易求解的 (tractable)，不存在多项式时间算法的问题是难求解的 (intractable)。

## 7.2 判定问题

对于判定问题，可以使用布尔值来表示问题的答案，并通过判断输出的是“Yes”还是“No”来确定问题的解。而对于优化问题，需要确定一个最优解；而对于计数问题需要计算问题实例的数目。**为了便于研究，计算复杂性理论中将问题限定为判定问题。**

而其他类的问题也都可以通过适当的变换**转化为判定问题**，且不改变问题的难易程度，从而利用计算复杂性理论进行分析和研究。

1. 寻找多数元素问题 (MAJORITY), 给定一个含有  $n$  个元素的数组，寻找并返回数组中的多数元素，如果没有多数元素，则返回 none.

可以转换为，多数元素判定 (MAJORITY-D): 给定一个含有  $n$  个元素的数组，问数组中是否存在多数元素。

2. 哈密顿回路搜索 (HCS): 给定一个无向图  $G$ , 搜索并返回其中一条哈密顿回路，如果没有哈密顿回路，则返回“No”。(哈密顿回路，是指经过图中每个顶点恰好一次的回路，如果图  $G$  中存在哈密顿回路，则称  $G$  为哈密顿图。

可以转化为，哈密顿图判定问题 (HC): 给定一个无向图  $G$ , 判断  $G$  是否是哈密顿图。

3. 最长公共子序列问题 (LCS): 给定取自相同字母集合的两个序列  $X$  和  $Y$ , 求  $X$  和  $Y$  的最长公共子序列。

可以转化为,  $K$  公共子序列问题 (KCS): 给定取自相同字母集合的两个序列  $X$  和  $Y$  以及正整数  $K$ , 问  $X$  和  $Y$  是否存在长度不小于  $K$  的公共子序列。

4. 货郎担问题 (TSP): 给定  $n$  个城市的集合, 城市  $i$  和城市  $j$  之间的距离为整数, 求一条经过每个城市恰好一次最后回到出发点的最短回路。

TSP-D: 给定  $n$  个城市的集合和一个正整数  $K$ , 城市  $i$  和城市  $j$  之间的距离为整数, 问是否存在经过每个城市恰好一次最后回到出发点, 且长度不超过  $K$  的回路。

## 7.3 P 类和 NP 类

### 7.3.1 图灵机模型

图灵机的逻辑结构包括:

- 一个无限长度的带子, 这个带子被分成了一个一个的单元格, 每个格子有包含一个来自字母表的符号 (含有空白)。
- 一个读写头, 可在纸带上左右移动, 并可以读、修改格子上的符号。
- 一个状态寄存器, 用来存储图灵机的当前状态。图灵机的所有可能状态数目是有限的, 并且有一个特殊的状态, 称为停机状态。
- 一个指令表 (控制单元), 根据机器当前所处的状态和带子上当前的符号, 指示机器进行特定的操作。比如: 擦除或者写入一个符号、向左或者向右移动磁头。并改变状态寄存器的值, 令机器进入一个新的状态或保持状态不变。

图灵机工作时, 大体重复如下过程:

1. 读写头读出当前格子的符号
2. 根据当前状态和读到的符号找到对应的控制指令, 并执行以下三个动作:
  - (a) 读写头在格子上擦除或写入一个数字或符号
  - (b) 图灵机状态转移到一个新的状态
  - (c) 读写头向左或向右移动一格

上述过程会一直重复, 直到机器到达一个终止状态, 或者无法继续执行为止。图灵机的状态转移方案可以是唯一的, 也可以不是唯一的。据此, 图灵机模型可以分为: 确定性图灵机 (Deterministic Turing Machines, DTM) 和非确定性图灵机 (Nondeterministic Turing Machines, NTM)

可以用 DTM 模型描述的算法, 称之为确定性算法。设  $A$  是求解问题  $\Pi$  的一个算法, 如果在求解问题  $\Pi$  的一个实例时, 在整个执行过程中, 每一步都只有一种选择, 则称  $A$  是确定性算法。因此如果对于同样的输入, 实例一遍又一遍地执行, 它的输出从不改变。如果  $A$  的时间是关于输入实例大小的多项式时间, 则称  $A$  是多项式时间确定性算法。

非确定性图灵机 (NTM) 是 DTM 的一种推广, NTM 和 DTM 的不同之处在于, 在计算的每一时刻, 根据当前状态和读写头所读的符号, NTM 存在多种状态转移方案, 机器可以任意选择其中一种方案继续运行, 直至停机。

可以用 NTM 模型描述的算法, 称之为不确定性算法。

不确定性算法的非形式化定义:

设  $A$  是求解问题  $\Pi$  的一个算法, 如果算法  $A$  以如下猜测 + 验证的方式工作, 称算法  $A$  为不确定性 (nondeterminism) 算法:

- 猜测阶段: 对问题的输入实例  $x$  产生一个任意字符串  $y$ , 在算法的每次运行,  $y$  可能不同, 因此猜测是以不确定的形式工作。
- 验证阶段: 在这个阶段, 用一个确定性算法验证两件事: 首先验证猜测的  $y$  是否是合适的形式, 若不是, 则算法停下并回答 no; 若是合适形式, 则继续检查它是否是  $x$  的解, 如果确实是  $x$  的解, 则停下并回答 yes, 否则停下并回答 no。

如果猜测和检查都在输入实例的多项式时间完成, 则称  $A$  为多项式时间不确定性算法。

注意: 若  $A$  输出 no, 并不意味着不存在一个满足要求的解, 也就是说, 并不意味着  $A$  不接受该输入, 因为猜测可能不正确。

### 7.3.2 P 类问题

在计算复杂性理论中, P 类问题是指那些判定问题, 其 yes/no 解可以通过确定性算法在多项式时间内求解。具体来说, 对于一个判定问题  $\Pi$ , 如果存在一个非负整数  $k$ , 使得对于输入规模为  $n$  的实例, 能够以  $O(n^k)$  的时间运行确定性算法得到答案, 则该问题属于 P(Polynomial) 类问题。

对于一个判定问题, 如果能构造出一个多项式求解算法, 则可证明该问题属于 P 类。显然, 所有易解问题都是 P 类问题。(例如: 排序问题、不相交集问题、最短路径问题、2 着色问题、2 可满足问题)

如果对于任意问题  $\Pi \in C$ ,  $\Pi$  的补也在  $C$  中, 我们说问题类  $C$  在补运算下是封闭的。比如 2 着色问题的补就是不 2 可着色。P 类问题在补运算下是封闭的。

### 7.3.3 NP 类问题

定义 1 (NP 类问题的验证定义): 判定问题类 NP 由这样的问题  $\Pi$  组成: 对于这些问题存在一个确定性算法  $A$ , 该算法在对  $\Pi$  的一个实例展示一个断言解时, 能在多项式时间内验证解的正确性。具体而言, 如果断言解导致答案是 yes, 就存在一种方法可以在多项式时间内验证这个解。

定义 2 (NP 类问题的非确定性计算定义): 判定问题类 NP 由这样的判定问题组成: 对于它们存在着多项式时间内运行的不确定性算法。形式化地说, 如果对于判定问题  $\Pi$ , 存在一个非负整数  $k$ , 对于输入规模为  $n$  的实例, 能够以  $O(n^k)$  的时间运行一个不确定性算法, 得到 yes/no 的答案, 则该判定问题  $\Pi$  是一个 NP (nondeterministic polynomial) 类问题。

NP 类问题是否对补运算封闭, 即  $NP? = co - NP$  尚未可知。

P 类问题是易求解的, NP 类问题是易验证的, 显然,  $P \subseteq NP$  目前仍然无法判断是否有或没有  $P = NP$ , 但是因为 P 在补运算下封闭, 所以有  $P \subseteq co - NP$ 。

## 7.4 多项式时间规约和 NP 完全性

NP 完全性 (NP-complete) 是 NP 类的一个特殊子类, 这类判定问题具有以下重要特性: 如果其中任何一个问题被证明存在多项式时间的确定性算法解, 那么 NP 类中的所有问题都将能够通过多项式时间的确定性算法求解。这意味着, 若存在这样的算法, 将直接导致  $NP = P$  的结论成立。

在计算复杂性理论中, 问题归约是一个核心概念。具体来说, 当一个问题 A 可以归约为问题 B 时, 这意味着我们可以利用解决 B 问题的方法来直接解决 A 问题, 或者说问题 A 能够被“转化”为问题 B 的形式进行处理。

从直观上看, 这种归约关系揭示了两个问题之间的复杂度比较: 若 A 可归约为 B, 则表明 B 的时间复杂度至少与 A 相当, 或者说 B 的求解难度不低于 A。换句话说, 问题 A 的难度不会超过问题 B 的难度, 因为只要能解决 B, 就必然能解决 A。

定义: 设判定问题  $\Pi_1 = \langle D_1, Y_1 \rangle$  和  $\Pi_2 = \langle D_2, Y_2 \rangle$ , 如果存在一个函数  $f: D_1 \rightarrow D_2$  满足如下条件:

- (1)  $f$  是多项式时间可计算的, 即存在计算  $f$  的多项式时间算法;
- (2) 对所有  $I \in D_1$ ,  $I \in Y_1 \Leftrightarrow f(I) \in Y_2$

则称  $f$  是  $\Pi_1$  到  $\Pi_2$  的多项式时间归约函数。计算  $f$  的多项式时间算法  $F$  称为归约算法。

定义: 设判定问题  $\Pi_1$  和  $\Pi_2$ , 如果存在  $\Pi_1$  到  $\Pi_2$  的多项式时间归约算法, 则称  $\Pi_1$  可多项式时间归约到  $\Pi_2$ , 记作  $\Pi_1 \propto_{\text{poly}} \Pi_2$ 。

证明. 哈密顿回路问题 (HC) 可多项式归约到货郎担问题 (TSP), 即  $HC \propto_{\text{poly}} TSP$ 。

### 1. 问题定义:

- 设  $HC = \langle D_1, Y_1 \rangle$ , 其中  $D_1$  为所有无向图的集合,  $Y_1$  为包含哈密顿回路的图
- 设  $TSP = \langle D_2, Y_2 \rangle$ , 其中  $D_2$  为所有带权完全图与界限值的组合,  $Y_2$  为存在长度不超过  $K$  的周游路线的实例

### 2. 构造归约函数: 定义函数 $f: D_1 \rightarrow D_2$ , 对任意 $I \in D_1$ (无向图 $G = \langle V, E \rangle$ ):

$$f(I) = (G', K)$$

其中:

- $G'$  为顶点集  $V$  上的完全图,  $E' = \{(u, v) \mid u, v \in V\}$
- 边权赋值规则:

$$d(u, v) = \begin{cases} 1, & \text{若 } (u, v) \in E \\ 2, & \text{若 } (u, v) \notin E \end{cases}$$

- 界限值  $K = |V|$

显然  $f$  是多项式时间可计算的。

### 3. 等价性证明: 对任意周游路线 (经过每个顶点恰好一次):

- 包含  $|V|$  条边, 每条边权值为 1 或 2
- 路线长度  $L$  满足  $|V| \leq L \leq 2|V|$

因此有:

$$\begin{aligned}
 f(I) \in Y_2 &\Leftrightarrow \exists \text{周游路线长度 } L \leq K = |V| \\
 &\Leftrightarrow L = |V| \quad (\text{因 } L \geq |V|) \\
 &\Leftrightarrow \text{路线中所有边权均为1} \\
 &\Leftrightarrow \text{该路线所有边都属于 } E \\
 &\Leftrightarrow \text{该路线是 } G \text{ 中的哈密顿回路} \\
 &\Leftrightarrow I \in Y_1
 \end{aligned}$$

综上,  $I \in Y_1 \Leftrightarrow f(I) \in Y_2$  且  $f$  为多项式时间可计算, 故  $HC \propto_{\text{poly}} TSP$  得证。  $\square$

定理: 如果  $\Pi_1 \propto_{\text{poly}} \Pi_2$ , 则  $\Pi_2 \in P$  蕴含  $\Pi_1 \in P$ ;  $\Pi_1$  是难解的蕴含  $\Pi_2$  是难解的。

也即表明:

定理: 如果  $\Pi_1 \propto_{\text{poly}} \Pi_2$ , 则相对于多项式时间, 如果  $\Pi_2$  是易解的, 则  $\Pi_1$  也是易解的, 即  $\Pi_2$  不会比  $\Pi_1$  更容易。或者反过来讲, 即  $\Pi_1$  不会比  $\Pi_2$  更难。

另外, 也容易证明多项式归约具有传递性:

定理: 设  $\Pi_1$ 、 $\Pi_2$  和  $\Pi_3$  是三个判定问题, 且有  $\Pi_1 \propto_{\text{poly}} \Pi_2$ ,  $\Pi_2 \propto_{\text{poly}} \Pi_3$ , 则  $\Pi_1 \propto_{\text{poly}} \Pi_3$ 。

## 7.5 NP 完全性

定义: 对于一个判定问题  $\Pi$ , 如果对所有  $\Pi' \in NP$ , 都有  $\Pi' \propto_{\text{poly}} \Pi$ , 则称  $\Pi$  是 NP 难的 (NP-hard, NPH)。

含义: NPH 问题不会比 NP 类中的任何问题容易。

定义: 对于一个判定问题  $\Pi \in NP$ , 如果对所有  $\Pi' \in NP$ , 都有  $\Pi' \propto_{\text{poly}} \Pi$ , 则称  $\Pi$  是 NP 完全的 (NP-complete, NPC)。

含义: NPC 问题是 NP 类中的 NPH 问题。由此也可以推断, NPC 问题是 NP 类中最难的问题。NP 完全性是判定 P 是否等于 NP 的关键。

NPH 问题和 NPC 问题的唯一差别是: NPC 问题必须是属于 NP 类的问题, 而 NPH 问题不一定在 NP 类中。

## 7.6 Cook-Levin 定理

库克-列文定理。

定理 (NP 困难的传递性): 如果存在  $\Pi' \in NPH$ , 使  $\Pi' \xrightarrow{\text{poly}} \Pi$ , 则  $\Pi$  是 NPH 的。

定理 (NP 完全性的传递性): 如果  $\Pi \in NP$ , 且存在  $\Pi' \in NPC$ , 使得  $\Pi' \xrightarrow{\text{poly}} \Pi$ , 则  $\Pi$  是 NPC 的。

上述定理表明, 如果要证明一个问题  $\Pi$  是属于 NPH (NPC) 的, 不再需要证明所有 NP 问题都多项式时间可归约到  $\Pi$ , 而是只需要证明存在一个 NPH (NPC) 的问题  $\Pi'$ ,  $\Pi'$  多项式时间可归约到  $\Pi$  即可。

因此, 要证明一个问题  $\Pi$  是属于 NPC 的, 现在只需要证明两点: (1)  $\Pi \in NP$ ; (2) 存在一个  $\Pi' \in NPC$ , 使得  $\Pi' \xrightarrow{\text{poly}} \Pi$ 。

可满足性问题是第一个被证明的 NPC 问题, SAT 问题是指: 是否存在一组对所有布尔变量的赋值, 使得整个合取范式为真。库克列文定理表明, SAT 问题是 NPC 的。

证明一个问题  $P_i$  是 NPC 问题需要 2 步进行:

1. 证明问题是 NP 类问题。
2. 证明存在一个 NPC 问题  $\Pi'$ ，使得其可以规约到问题  $\Pi$

已知 HC 问题是 NPC 问题，证明 TSP 问题也是 NPC 问题。

证明. (1)  $TSP \in NP$

可设计如下多项式不确定性运行算法：对任意给定的一个边带权的完全无向图  $G$  和正整数  $K$ ，随机猜测  $G$  中所有顶点的一个排列代表一条恰好经过每个城市一次的周游路线，然后检查这个排列相邻的两个顶点之间边以及排列的首尾顶点之间边的总值之和是否不超过  $K$ 。如果是，则回答“Yes”，否则回答“No”。当  $G$  中存在长度不超过  $K$  的周游路线时，总能猜对一个顶点的排列，它确实给出一条长度不超过  $K$  的周游路线，从而使算法回答“Yes”；如果  $G$  中不存在长度不超过  $K$  的周游路线，则猜测的任何排列表示的周游路线长度都会大于等于  $K$ ，从而算法总是回答“No”。显然，猜测一个排列和检查这个排列对应的周游路线的长度是否不超过  $K$  都可以在多项式时间内完成。因此，TSP 是 NP 类问题。任何一个构成长度不超过  $K$  的周游路线的顶点排列都是证据。

(2)  $HC \leq_{\text{poly}} TSP$ ，前面已证。 □

### 7.6.1 限制法

证明  $\Pi$  包含一个已知为 NPC 问题  $\Pi'$  作为它的一种特殊情况。

最大可满足性 (Max-SAT) 问题：任给关于  $n$  个变元  $x_1, x_2, \dots, x_n$  的  $m$  个简单析取式  $C_1, C_2, \dots, C_m$  以及正整数  $K$ ，问是否存在关于这些变元的赋值  $t$  使得  $C_1, C_2, \dots, C_m$  中至少有  $K$  个为真？

对比 Max-SAT 问题和 SAT 问题，很容易看出 SAT 是 Max-SAT 问题限定  $K = m$  的特殊情况。

当  $K = m$  时，赋值  $t$  使  $C_1, C_2, \dots, C_m$  中至少有  $K$  个为真就是使  $C_1, C_2, \dots, C_m$  全部为真，这当且仅当赋值  $t$  使  $F = C_1 \wedge C_2 \wedge \dots \wedge C_m$  为真。

因此当  $K = m$  时，Max-SAT 问题就退化为 SAT 问题。即 SAT 可以看作是 Max-SAT 的子问题。

### 7.6.2 局部替换法

一般来讲，如果已知  $\Pi \in NPC$ ， $\Pi'$  是  $\Pi$  在限定一定条件下的子问题，或者更一般情况， $\Pi'$  与  $\Pi$  的问题实例的构造有某方面的相似之处时，往往可以采用局部替换法来证明  $\Pi'$  的 NP 完全性。

局部替换法是选择已知 NP 完全问题的实例中的某些元素作为基本单元，然后把每个基本单元替换成指定结构，从而得到目标问题的对应实例。

证明三元可满足性问题 (3SAT) 是 NPC 问题。

显然 3SAT 问题是 SAT 问题的子问题。

对每一个  $I \in SAT$ ， $I$  是关于变元  $x_1, x_2, \dots, x_n$  的合取范式  $F = C_1 \wedge C_2 \wedge \dots \wedge C_m$ ，其中  $C_1, C_2, \dots, C_m$  是简单析取式。

设计转换函数  $f$ ，使得  $f(F) = F' = F'_1 \wedge F'_2 \wedge \dots \wedge F'_m$ ，其中  $F'_i$  是对应于  $C_i$  的三元合取范式，且  $C_i$  是可满足的当且仅当  $F'_i$  是可满足的。这样构造的  $F'$  显然满足  $F$  是可满足的当且仅当  $F'$  是可满足的。

根据  $C_i$  的四种可能情形，三元合取范式  $F'_i$  分别构造如下：

1.  $C_i$  仅包含单个文字，设  $C_i = z_1$ ，其中  $z_1$  为文字，即某变元或者变元的否定（下同）。

引入两个新的变元  $y_{i1}$  和  $y_{i2}$ , 令

$$F'_i = (z_1 \vee y_{i1} \vee y_{i2}) \wedge (z_1 \vee \neg y_{i1} \vee y_{i2}) \wedge (z_1 \vee y_{i1} \vee \neg y_{i2}) \wedge (z_1 \vee \neg y_{i1} \vee \neg y_{i2})$$

显然, 无论怎么赋值,  $y_{i1} \vee y_{i2}$ ,  $\neg y_{i1} \vee y_{i2}$ ,  $y_{i1} \vee \neg y_{i2}$  和  $\neg y_{i1} \vee \neg y_{i2}$  不可能同时为真, 因此  $F'_i$  为真当且仅当  $z_1$  为真, 也即  $F'_i$  为真当且仅当  $C_i$  为真。

2.  $C_i$  是两个文字的析取式, 设  $C_i = z_1 \vee z_2$ , 其中  $z_1, z_2$  均为文字。引入一个新的变元  $y_i$ , 令

$$F'_i = (z_1 \vee z_2 \vee y_i) \wedge (z_1 \vee z_2 \vee \neg y_i)$$

显然, 无论怎么赋值,  $y_i$  和  $\neg y_i$  不可能同时为真, 因此  $F'_i$  为真当且仅当  $z_1 \vee z_2$  为真, 也即  $F'_i$  为真当且仅当  $C_i$  为真。

3.  $C_i$  是三个文字的析取式, 设  $C_i = z_1 \vee z_2 \vee z_3$ , 其中  $z_1, z_2, z_3$  均为文字。直接令  $F'_i = C_i = z_1 \vee z_2 \vee z_3$ , 显然  $F'_i$  为真当且仅当  $C_i$  为真。

4.  $C_i$  是三个以上文字的析取式, 设  $C_i = z_1 \vee z_2 \vee \cdots \vee z_k$ ,  $k \geq 4$ , 其中  $z_1, z_2, \dots, z_k$  均为文字。

引入  $k-3$  个新的变元  $y_{i1}, y_{i2}, \dots, y_{i(k-3)}$ , 令

$$F'_i = (z_1 \vee z_2 \vee y_{i1}) \wedge (\neg y_{i1} \vee z_3 \vee y_{i2}) \wedge (\neg y_{i2} \vee z_4 \vee y_{i3}) \wedge \cdots \wedge (\neg y_{i(k-4)} \vee z_{k-2} \vee y_{i(k-3)}) \wedge (\neg y_{i(k-3)} \vee z_{k-1} \vee z_k)$$

设由真值指派  $t$  满足  $C_i$ , 则一定存在一个  $j (1 \leq j \leq k)$  使得  $t(z_j) = 1$ 。将  $t$  扩展到对所有新变元的赋值上:

- (a) 如果  $j = 1$  或者  $2$  时, 令  $t(y_{is}) = 0, 1 \leq s \leq k-3$ ;
- (b) 如果  $j = k-1$  或者  $k$  时, 令  $t(y_{is}) = 1, 1 \leq s \leq k-3$ ;
- (c) 如果  $3 \leq j \leq k-2$  时, 令

$$t(y_{is}) = \begin{cases} 1, & 1 \leq s \leq j-2 \\ 0, & j-1 \leq s \leq k-3 \end{cases}$$

不难验证, 扩展后的真值指派  $t$  也满足  $F'_i$ 。因此, 如果  $t(C_i) = 1$ , 则  $t(F'_i) = 1$ 。

其实证明到这里就足够了, 但是为了加强问题, 我们进一步证明等价性。

5. 反过来, 设  $t(F'_i) = 1$ 。如果  $t(y_{i1}) = 0$ , 则必有  $t(z_1 \vee z_2) = 1$ ; 如果  $t(y_{i(k-3)}) = 1$ , 则必有  $t(z_{k-1} \vee z_k) = 1$ ; 否则必有  $s (1 \leq s \leq k-4)$ , 使得  $t(y_{is}) = 1$  且  $t(y_{i(s+1)}) = 0$ , 从而  $t(z_{s+2}) = 1$  (反证可得)。因此, 如果  $t(F'_i) = 1$ , 则必存在  $j (1 \leq j \leq k)$  使得  $t(z_j) = 1$ , 从而如果  $t(C_i) = 1$ 。因此,  $F'_i$  为真当且仅当  $C_i$  为真成立。

综合四种情况可得,  $F$  是可满足的当且仅当  $F'$  是可满足的。另外,  $F'_i$  为三元合取范式, 所包含的简单析取式的个数不会超过  $C_i$  中文字的 4 倍, 每个析取式由 3 个文字组成。因此,  $f(F) = F'$  可在  $|F|$  的多项式时间内构造, 也就函数  $f$  是多项式时间可计算的。因此  $f$  是 SAT 到 3SAT 的多项式归约函数。



### 7.6.3 构件设计法

如果限制法和局部替换法都不起效果，可以考虑构件设计法。基本思想是用目标问题的实例设计某些构件，能够把这些构件组合起来“实现”已知的 NPC 问题实例。

对于下面 3 个问题，容易证明他们是 NP 的。下面会逐一证明 SAT 问题可以规约到他们任意之一。

- 顶点覆盖：给出一个无向图  $G = (V, E)$  和一个正整数  $k$ ，是否存在大小为  $k$  的子集  $C \subseteq V$ ，使  $E$  中的每条边至少和  $C$  中的一个顶点关联？
- 独立集：给出一个无向图  $G = (V, E)$  和一个正整数  $k$ ，是否存在  $k$  个顶点的子集  $S \subseteq V$ ，使得对于每一对顶点  $u, w \in S$ ， $(u, w) \notin E$ ？
- 团集：给出一个无向图  $G = (V, E)$  和一个正整数  $k$ ， $G$  包含一个大小为  $k$  的团集吗？（注意一个  $G$  中大小为  $k$  的团集是  $G$  中  $k$  个顶点的一个完全子图。）

**顶点覆盖问题是 NPC 问题** 给定一个有  $m$  个子句和  $n$  个布尔变元  $x_1, x_2, \dots, x_n$  的可满足性实例  $f = C_1 \wedge C_2 \wedge \dots \wedge C_m$ ，按如下方式构造图  $G$ ：

1. 对于  $f$  中的每一个布尔变元  $x_i$ ， $G$  包含一对顶点  $x_i$  和  $\neg x_i$ ，它们有一条边相连；
2. 对于每个子句  $C_j$  包含的  $n_j$  个文字， $G$  包含一个大小为  $n_j$  的团集  $C_j$ ；
3. 对于在  $C_j$  中的每个顶点  $w$ ，有一条边连接  $w$  到 1 中构造的顶点对  $(x_i, \neg x_i)$  中它相应的文字。这些边称为连通边；
4. 令  $k = n + \sum_{j=1}^m (n_j - 1)$ 。

显然，上述构造图  $G$  的方法可在多项式时间内完成。

引理： $f$  是可满足的当且仅当构造的图  $G$  有一个大小为  $k$  的顶点覆盖集。

证明：

( $\Rightarrow$  方向)

1. 考察  $f$  中每个文字的赋值：如果  $x_i$  为真，则将步骤 (1) 构造的对应顶点  $x_i$  加到顶点覆盖集中；否则，将  $\neg x_i$  对应顶点加入到顶点覆盖集中；因此在 (1) 中所有边也已经被覆盖。
2. 由于  $f$  是可满足的，因此每个子句中至少有 1 个文字赋值为真。在步骤 (2) 构造的每个子句对应的团集  $C_j$  中与赋值为真的文字对应的顶点  $u$ ，同 (1) 中对应的顶点  $v$  之间的连通边  $(u, v)$  均已经被覆盖（因为  $v$  已经在覆盖集中）。
3. 将每个团集  $C_j$  中除了顶点  $u$  之外的其余  $n_j - 1$  个顶点也加入顶点覆盖集中，显然这些顶点能覆盖团集中所有边。
4. 显然，顶点覆盖集的大小为  $k$ 。因此，如果  $f$  是可满足的，则图  $G$  有一个大小为  $k$  的顶点覆盖。

( $\Leftarrow$  方向)

1. 假设图  $G$  有一个大小为  $k$  的顶点覆盖集。
2. 显然在步骤 (1) 构造的每个边  $(x_i, \neg x_i)$  中至少有 1 个顶点必须在覆盖集中。因此可知已有  $n$  个顶点在覆盖集中，同时，这些顶点为端点的连通边也由这些顶点所覆盖了。

3. 考虑剩余的  $(n_1 - 1) + (n_2 - 1) + \cdots + (n_m - 1)$  个顶点。不难看出, 对于一个大小为  $n_j$  的团集, 顶点覆盖集的大小为  $n_j - 1$ 。因此, 对每个团集, 由于连通边除了与连通边有关的顶点外, 其余顶点都包含在顶点覆盖中。
4. 对于  $f$  中的每个变元  $x_i$ , 如果它对应的顶点在覆盖集中, 则令  $x_i = \text{true}$ , 否则, 令  $x_i = \text{false}$ 。这样, 在每个子句对应的团集中, 一定有一个顶点连接  $x_i$  或  $\neg x_i$  对应赋值为真的顶点, 才能使连通边被覆盖。
5. 因此, 每个子句中至少有 1 个文字的赋值为真, 即  $f$  是可满足的。

**独立集问题是 NPC 问题** 引理: 设  $G = (V, E)$  是连通无向图, 那么  $S \subseteq V$  是一个独立集当且仅当  $V - S$  是  $G$  的一个顶点覆盖。

证明: 设  $e = (u, v)$  是  $G$  中的任意边:

1.  $S$  是独立集  $\Leftrightarrow u$  和  $v$  不同时在  $S$  中
2.  $\Leftrightarrow u$  或  $v$  至少有一个在  $V - S$  中
3.  $\Leftrightarrow V - S$  覆盖了边  $e$
4. 由于  $e$  是任意边, 因此  $V - S$  是  $G$  的顶点覆盖

**团集问题是 NPC 问题** 给定一个有  $m$  个子句和  $n$  个布尔变元  $x_1, x_2, \dots, x_n$  的可满足性实例  $f = C_1 \wedge C_2 \wedge \cdots \wedge C_m$ , 构造一个图  $G = (V, E)$ , 其中:  $V$  是  $2n$  个文字的所有出现的集合 (注意: 一个文字是一个布尔变元或它的否定)  $E = \{(x_i, x_j) | x_i \text{ 和 } x_j \text{ 位于两个不同的子句, 并且 } x_i \neq \neg x_j\}$

图  $G$  的每个顶点对应于  $f$  中的每个文字, 多次出现的重复表示。若图  $G$  中两个顶点对应的文字不互补, 且不出现在同一子句中, 则将其连线。(节点  $a-b$  连线意味着文字  $a$  和  $b$  可同时为真)

引理:  $f$  是可满足的当且仅当  $G$  有一个大小为  $m$  的团集。

证明: 设  $f$  有  $m$  个子句, 则有:  $f$  可满足  $\Leftrightarrow m$  个子句同时为真  $\Leftrightarrow$  每个子句至少 1 个文字为真  $\Leftrightarrow G$  中有  $m$  个顶点之间彼此相连  $\Leftrightarrow G$  中有  $m$  个顶点的团集

## 7.7 本章作业

### 7.7.1 平面加权图的最小生成树问题的计算复杂度

设平面图  $G = (V, E)$ ,  $E$  中的边  $(i, j)$  的权值为两个点  $i$  和  $j$  之间的距离。  $|V| = n$ 。计算  $G$  的一棵最小生成树。

证明, 求解该问题的算法类的时间复杂度下界为  $\Omega(n \log n)$ 。

规约证明. 将排序问题规约到最小生成树问题:

1. **输入转换:** 给定待排序的  $n$  个实数  $\{x_1, x_2, \dots, x_n\}$ , 构造平面图  $G = (V, E)$ :

- 顶点集  $V = \{v_1, \dots, v_n\}$ , 其中每个  $v_i$  对应实数  $x_i$
- 边集  $E = \{(v_i, v_j) | \forall i \neq j\}$  (完全图)
- 边权定义为  $w(v_i, v_j) = |x_i - x_j|$

2. **MST 的性质**: 在如此构造的图中, 最小生成树必定是:

$$\text{MST} = \{(v_{(1)}, v_{(2)}), (v_{(2)}, v_{(3)}), \dots, (v_{(n-1)}, v_{(n)})\}$$

其中  $v_{(i)}$  对应排序后的第  $i$  小元素 (即  $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$ )。

3. **规约的正确性**:

1. 从 MST 的边可以立即读出排序结果: 遍历 MST 边集即可得到有序序列
2. 该转换过程的时间复杂度为  $O(n^2)$  (构造完全图)

4. **复杂度下界**: 由于比较排序问题的下界为  $\Omega(n \log n)$ , 而 MST 问题可以解排序问题, 因此:

$$\text{MST 问题的复杂度下界} \geq \text{排序问题的下界} = \Omega(n \log n)$$

特别地, 对于平面图, 该下界仍然成立 (因为规约构造的图可以是平面的)。□

### 7.7.2 元素唯一性问题

设  $S$  是由  $n$  个数构成的数组, 判断  $S$  中的元素是否唯一。如果唯一, 输出 "Yes"; 否则输出 "No"。

证明该判定问题的计算复杂度是  $\Theta(n \log n)$ 。

给定  $n$  维空间  $E^n$  中的集合:

$$W = \{(x_1, x_2, \dots, x_n) \in \mathbb{R}^n \mid x_i \neq x_j, \forall i \neq j\}$$

需先证明  $W$  可分解为  $n!$  个互不相交的连通分量  $W_\pi$ 。

对每个排列  $\pi \in S_n$  ( $S_n$  为  $n$  阶对称群), 定义:

$$W_\pi = \{(x_1, \dots, x_n) \mid x_{\pi(1)} < x_{\pi(2)} < \dots < x_{\pi(n)}\}$$

每个  $W_\pi$  是连通的, 因为:

1. 作为开凸集: 由线性不等式组定义的解空间是凸的
2. 任意两点可通过直线路径连接:  $\forall \mathbf{a}, \mathbf{b} \in W_\pi$ , 有  $t\mathbf{a} + (1-t)\mathbf{b} \in W_\pi$  ( $0 \leq t \leq 1$ )

对任意两个不同排列  $\pi \neq \sigma$ , 必有:

$$W_\pi \cap W_\sigma = \emptyset$$

因为:

- 存在至少一对指标  $i, j$  使得  $\pi^{-1}(i) < \pi^{-1}(j)$  但  $\sigma^{-1}(i) > \sigma^{-1}(j)$
- 在  $W_\pi$  中要求  $x_i < x_j$ , 而在  $W_\sigma$  中要求  $x_i > x_j$

所有  $W_\pi$  的并集等于  $W$ :

$$\bigcup_{\pi \in S_n} W_\pi = W$$

因为:

- 对任意  $\mathbf{x} \in W$ , 可通过排序得到唯一排列  $\pi$  使  $x_{\pi(1)} < \dots < x_{\pi(n)}$

- 故  $\mathbf{x} \in W_\pi$  且不属于其他  $W_\sigma$

在  $\mathbb{R}^n$  空间中:

- 每个超平面  $x_i = x_j$  将空间分成两个半空间
- $W_\pi$  对应所有  $\binom{n}{2}$  个不等式  $x_{\pi(i)} < x_{\pi(j)}$  ( $i < j$ ) 的交集
- 这些不等式定义的区域是  $\mathbb{R}^n$  中的开凸多面体
- 不同的严格全序排列共有  $n!$  种
- 每个排列对应唯一的坐标顺序关系
- 因此连通分量数量为  $|S_n| = n!$

集合  $W$  被超平面  $\{x_i = x_j\}_{i \neq j}$  划分为  $n!$  个:

- 互不相交的
- 连通的
- 开凸的

分量, 每个分量对应一个唯一的排列顺序。

### 7.7.3 2SAT 问题

设计一个多项式时间确定算法求解 2 可满足性问题, 从而证明 2 可满足性问题是 P 类问题。

#### 1. 构建蕴含图:

- 对于每个变量  $x$ , 创建两个顶点:  $x$  和  $\neg x$ 。
- 对于每个子句  $(a \vee b)$ , 添加两条有向边:

$$- \neg a \rightarrow b$$

$$- \neg b \rightarrow a$$

#### 2. 计算强连通分量 (SCC):

- 使用 Kosaraju 算法或 Tarjan 算法找到图中的所有强连通分量。
- 强连通分量是指图中任意两个顶点相互可达的最大子图。

#### 3. 检查可满足性:

- 对于每个变量  $x$ , 检查  $x$  和  $\neg x$  是否在同一个 SCC 中。
- 如果存在这样的  $x$ , 则公式不可满足, 返回“不可满足”。
- 否则, 公式可满足, 返回“可满足”。

#### 4. 构造满足的赋值 (可选):

- 对 SCC 进行拓扑排序, 得到顺序  $S_1, S_2, \dots, S_k$ 。

- 初始化所有变量为未赋值。
- 按  $i = k$  downto 1 的顺序处理每个 SCC:
  - 如果  $S_i$  中的任何文字未被赋值:
    - \* 将  $S_i$  中的所有文字赋值为真。
    - \* 将它们的否定赋值为假。
- 返回得到的赋值。

#### 7.7.4 2 着色问题

设计一个多项式时间的确定算法求解 2 着色问题，从而证明 2 着色问题是 P 类问题。

**Input:** 无向图  $G = (V, E)$

**Output:** 若  $G$  是二分图返回着色方案，否则返回”不可着色”

```

1 Algorithm TWO-COLORING( $G$ )
2 foreach  $v \in V$  do
3   |  $color[v] \leftarrow$  未定义
4 end
5  $queue \leftarrow$  空队列
6 任选顶点  $s \in V$ 
7  $color[s] \leftarrow$  红色
8  $queue.enqueue(s)$ 
9 while  $queue$  非空 do
10  |  $u \leftarrow queue.dequeue()$ 
11  | foreach  $v \in Adj[u]$  do
12  |   | if  $color[v] =$  未定义 then
13  |   |   |  $color[v] \leftarrow$  与 $color[u]$ 相反的颜色
14  |   |   |  $queue.enqueue(v)$ 
15  |   | end
16  |   | else if  $color[v] = color[u]$  then
17  |   |   | return ”不可着色”
18  |   | end
19  | end
20 end
21 return  $color$ 

```

**Algorithm 33:** 2-着色问题的 BFS 解法

**正确性证明** 算法基于以下观察：

- 图是二分图当且仅当可以用两种颜色着色且相邻节点颜色不同
- 使用 BFS 遍历时，若发现相邻节点颜色相同则立即终止
- 颜色冲突检测保证了算法的正确性

### 时间复杂度分析

- 初始化阶段:  $O(|V|)$
- BFS 遍历:  $O(|V| + |E|)$
- 总时间复杂度:  $O(|V| + |E|)$

由于:

- 算法在多项式时间  $O(|V| + |E|)$  内终止
- 使用确定性图遍历 (BFS/DFS)
- 不需要猜测和验证步骤

因此 2-着色问题属于 P 类问题。

#### 7.7.5 规约问题的传递性证明

对判定问题  $\Pi_1$  和  $\Pi_2$ , 证明如下结论:

如果  $\Pi_1 \propto_{poly} \Pi_2$ , 则  $\Pi_2 \in P$  蕴含  $\Pi_1 \in P$ ;  $\Pi_1$  是难解的蕴含  $\Pi_2$  是难解的。

证明. 设  $\Pi_1$  和  $\Pi_2$  为形式语言, 证明分为两部分:

##### 第一部分: 正向蕴含

1. 已知  $\Pi_1 \propto_{poly} \Pi_2$ , 即存在多项式时间可计算函数  $f$ , 使得

$$\forall x \in \Sigma^*, x \in \Pi_1 \Leftrightarrow f(x) \in \Pi_2$$

2. 设  $\Pi_2 \in P$ , 则存在多项式时间算法  $A_2$  判定  $\Pi_2$
3. 构造  $\Pi_1$  的判定算法  $A_1$ :
  - 输入  $x$ , 计算  $f(x)$  (耗时  $p(|x|)$ ,  $p$  为多项式)
  - 运行  $A_2(f(x))$  (耗时  $q(|f(x)|)$ ,  $q$  为多项式)
  - 输出  $A_2$  的结果
4. 总时间复杂度为  $p(|x|) + q(|f(x)|)$ , 仍为多项式时间

##### 第二部分: 逆否命题

1. 原命题的逆否命题为: 若  $\Pi_2 \notin \text{难解类}$ , 则  $\Pi_1 \notin \text{难解类}$
2. 假设  $\Pi_2$  不是难解的 (即  $\Pi_2 \in P$ )
3. 由第一部分结论直接可得  $\Pi_1 \in P$ , 故  $\Pi_1$  不是难解的
4. 特别地, 当 "难解类" 取  $P$  的补集时:

$$\Pi_2 \notin \bar{P} \Rightarrow \Pi_1 \notin \bar{P}$$

即  $\Pi_2 \in P \Rightarrow \Pi_1 \in P$

综上得证: 多项式时间规约具有保持问题复杂度的性质。 □

### 7.7.6 集合覆盖问题是 NPC 问题

集合覆盖问题：给出集合  $S = s_1, s_2, \dots, s_n$ ，和它的子集簇  $C = c_1, c_2, \dots, c_m$ ，其中  $\forall c_i, c_j \subseteq S$ ，给定正整数  $k$ ，是否存在  $C$  的大小不超过  $k$  的子集  $C'$ ， $C'$  中的集合的并是  $S$ ？

顶点覆盖问题：给出一个无向图  $G = (V, E)$ ，和一个正整数  $k$ ，是否存在大小不超过  $k$  的顶点子集  $C \subseteq V$ ，使  $E$  中的每条边至少和  $C$  中的一个顶点关联？

请证明：顶点覆盖问题  $\leq_{poly}$  集合覆盖问题。

通过将顶点覆盖问题（Vertex Cover）规约到集合覆盖问题（Set Cover），证明集合覆盖问题属于 NPC 类。

**规约构造** 给定顶点覆盖实例  $\langle G = (V, E), k \rangle$ ，构造集合覆盖实例  $\langle S, C, k' \rangle$  如下：

1. 令全集  $S = E$ ，即所有边构成的集合
2. 对每个顶点  $v \in V$ ，创建子集  $s_v = \{e \in E \mid \text{边 } e \text{ 与顶点 } v \text{ 关联}\}$ ，构成子集簇  $C = \{s_v \mid v \in V\}$
3. 保持参数不变，设  $k' = k$

#### 正确性证明

##### 1. 正向证明（Vertex Cover $\Rightarrow$ Set Cover）：

- 设存在大小为  $j \leq k$  的顶点覆盖  $V' = \{v_1, \dots, v_j\}$
- 构造对应的子集簇  $C' = \{s_{v_1}, \dots, s_{v_j}\}$
- 对任意边  $e = (u, w) \in E$ ：
  - 由顶点覆盖性质， $u$  或  $w$  至少一个在  $V'$  中
  - 若  $u \in V'$ ，则  $e \in s_u \in C'$
- 故  $C'$  覆盖  $S$ ，且  $|C'| = j \leq k$

##### 2. 反向证明（Set Cover $\Rightarrow$ Vertex Cover）：

- 设存在大小为  $j \leq k$  的集合覆盖  $C' = \{s_{v_1}, \dots, s_{v_j}\}$
- 构造对应的顶点集  $V' = \{v_1, \dots, v_j\}$
- 对任意边  $e = (u, w) \in E$ ：
  - 存在  $s_v \in C'$  包含  $e$
  - 故  $e$  至少与  $v \in V'$  关联
- 故  $V'$  是顶点覆盖，且  $|V'| = j \leq k$

##### 3. 时间复杂度分析：

- 构造全集  $S$ ： $O(|E|)$
- 构建子集簇  $C$ ： $O(|V| \cdot \deg(v)) = O(|V||E|)$
- 总时间复杂度为多项式时间  $O(|V||E|)$

**结论** 通过上述多项式时间规约，且已知顶点覆盖问题是 NP 完全问题，可证明集合覆盖问题属于 NPC 类。

### 7.7.7 0-1 背包问题

0-1 背包问题的动态规划算法是一个多项式时间算法吗？解释你的答案。

0-1 背包问题的动态规划算法不是一个多项式时间算法。该算法的运行时间为  $O(nW)$ ，其中  $n$  是商品数量， $W$  是背包能容纳的最大商品总重量。 $W$  的二进制编码长度为  $l = \lceil \lg W \rceil$ ，该算法的运行时间为  $O(n2^l)$ ，随着  $l$  的增加， $O(n2^l)$  呈指数级增长。我们通常称 0-1 背包问题为一个伪多项式时间算法<sup>+</sup>(pseudo-polynomial time algorithm)。

### 7.7.8 子例程时间影响

证明：若一个算法对多项式时间子例程的调用次数为常数，并且每次执行的额外工作量也需要多项式时间，则该算法的运行需要多项式时间。若多项式时间子程序的调用次数修改为多项式，则该算法的运行可能需要指数时间。

首先证明第一个命题，假设输入规模为  $n$ ，调用  $c$  次多项式时间子例程，其中  $c$  为常数，因为每次迭代的输出输入为下一次迭代的输入。设第  $i$  次迭代多项式时间子例程的运行时间为  $O(n^{k_i})$ ，其中  $k_i$  为常数，多项式时间的额外工作量为  $O(n^{l_i})$ ，其中  $l_i$  为常数，第一次迭代的运行时间为  $T_1(n) = O(n^{k_1}) + O(n^{l_1}) = O(n^{\max\{k_1, l_1\}})$  为多项式时间，输出规模为  $n_1 = O(T_1(n)) = O(n^{\max\{k_1, l_1\}})$ ，第二次迭代的运行时间为

$$T_2(n) = O(n_1^{\max\{k_2, l_2\}}) = O((n^{\max\{k_1, l_1\}})^{\max\{k_2, l_2\}}) = O(n^{\max\{k_1, l_1\} \max\{k_2, l_2\}})$$

为多项式时间，输出规模为  $n_2 = O(T_2(n)) = O(n^{\max\{k_1, l_1\} \max\{k_2, l_2\}})$ ，采用归纳法，第  $i$  次迭代的运行时间  $T_i(n) = O(n^{\prod_{j=1}^i \max\{k_j, l_j\}})$  为多项式时间，输出规模为

$$n_i = O(T_i(n)) = O(n^{\prod_{j=1}^i \max\{k_j, l_j\}})$$

总的运行时间为

$$\sum_{i=1}^c T_i(n) \leq c \max\{T_1(n), T_2(n), \dots, T_c(n)\}$$

为多项式时间。

然后证明第二个命题，举例如下，假设开始输入规模为 1，每次迭代输出规模为输入规模的两倍， $n$  次迭代后输出规模为  $2^n$ ，输出规模与运行时间多项式相关，该算法的运行时间为指数时间。证明完毕。

## 8 chapter8: 回溯法

### 8.1 回溯法基本原理

一种更好的搜索算法，将搜索空间尽量减小，分支限界法也是这样的原理。

回溯法求解分 3 个步骤：

1. 定义一个解空间，他包含问题的解

问题的解一般可表示为向量（也称元组）形式  $X = (x_1, x_2, \dots, x_n)$ ，其中分量  $x_i$  的取值范围为有穷且可枚举的集合  $S_i$ 。



每个  $x_i$  的所有可能取值的组合构成的集合, 称为问题的解空间。其中每一个取值组合对应问题实例的一个可能解。(注意好组合问题与排序问题的不同)

对 0/1 背包问题, 对  $n$  个物品, 定义解的形式为  $n$  元组  $X = (x_1, x_2, \dots, x_n)$ , 其中每个分量  $x_i$  的取值范围  $S_i = \{0, 1\}$ 。所有 0 和 1 构成的  $n$  元组构成了问题的解空间, 由  $2^n$  种可能的解组成。

例如, 当  $n=3$  时, 0/1 背包问题的解空间如下:  $(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,1,1)$

解空间树通常有两种类型: 子集树和排序树。当所给的问题是从  $n$  个元素的集合  $S$  中找出满足某种性质的子集时, 相应的解空间树称为子集树。当所给的问题是确定  $n$  个元素满足某种性质的排列时, 相应的解空间树称为排列树。

- 活结点: 如果已生成一个结点而它的儿子结点还没有全部生成, 则这个结点叫做活结点; 当前正在生成其儿子结点的活结点叫 E-结点 (正在扩展的结点)。
- 死结点: 不再进一步扩展或者其儿子结点已经全部生成的已生成结点是死结点。

## 2. 用易于搜索的方式组织解空间

## 3. 深搜解空间, 获得问题的解

回溯法原理: 当从状态  $s_i$  搜索到状态  $s_{i+1}$  后, 如果  $s_{i+1}$  变为死结点, 则从状态  $s_{i+1}$  回退到  $s_i$ , 再从  $s_i$  找其他可能的路径, 所以回溯法体现出走不通就退回再走的思路。通常采用两种策略来避免无效的搜索, 提高回溯法的搜索效率:

1. 使用约束函数在扩展顶点处剪去不满足约束的子树;
2. 如果是优化问题, 用限界函数剪去不能得到最优解的子树。

设  $P(x_1, x_2, \dots, x_i)$  为真表示向量  $\langle x_1, x_2, \dots, x_i \rangle$  满足某个性质 (例如,  $n$  皇后问题中前  $i$  个皇后放置在彼此不能攻击的位置)。多米诺性质:

$$P(x_1, x_2, \dots, x_{k+1}) \rightarrow P(x_1, x_2, \dots, x_k) \quad 0 < k < n$$

要使回溯法得到正确应用, 性质  $P$  必须满足多米诺性质。强调解的前缀继承性 (回溯法的核心) 而回溯法的本质就是约束穷举。

```

Input: 问题的输入
Output: 解向量  $x[1..n]$  或 "no solution"

1 for  $k \leftarrow 1$  to  $n$  do
2   |  $x[k] \leftarrow k$  ;                                // 初始排列
3 end
4  $flag \leftarrow false$ ;
5 advance(1);
6 if  $flag$  then
7   | return  $x[1..n]$ 
8 else
9   | return "no solution"
10 end
11 Procedure  $advance(i)$ 
12   if  $i > n$  then
13     |  $flag \leftarrow true$  ;                          // 搜索到叶子结点, 找到可行解
14     | return
15   end
16   for  $j \leftarrow i$  to  $n$  do
17     |  $swap(x[i], x[j])$  ;                              // 生成候选排列
18     | if  $legal(i)$  then
19       |  $advance(i + 1)$  ;                             // 递归进入下一层
20     | end
21     |  $swap(x[i], x[j])$  ;                              // 回溯恢复状态
22   end

```

Algorithm 34: BACKTRACK (基于排列树的递归框架)

## 8.2 回溯法应用

### 8.2.1 图的三着色问题

给出一个无向图  $G = (V, E)$ , 需要用三种颜色之一为  $V$  中的每个顶点着色 (三种颜色编号分别为 1, 2 和 3), 使得没有两个邻接的顶点着同样的颜色。

解的表示: 设图的顶点数为  $n$ , 用  $n$  元组  $(c_1, c_2, \dots, c_n)$  来表示顶点的一种着色方案,  $c_i \in \{1, 2, 3\}$ ,  $1 \leq i \leq n$ 。一个  $n$  个顶点的图共有  $3^n$  种可能的着色 (合法的和非法的)。

例如,  $(1, 2, 2, 3, 1)$  表示一个有 5 个顶点的图的一种着色方案, 一共有  $3^5$  种可能的着色 (合法的和非法的)。

所有着色方案的集合可以用一棵完全三叉树表示, 树中从根到叶子的每一条路径代表一种颜色指派。

```

Input: 无向图  $G = (V, E)$ , 顶点数  $n = |V|$ 
Output: 顶点 3 着色方案  $c[1..n]$  ( $c[i] \in \{1, 2, 3\}$ ) 或 "no solution"
1  for  $k \leftarrow 1$  to  $n$  do
2     $c[k] \leftarrow 0$ ;                                // 初始化颜色数组
3  end
4   $flag \leftarrow false$ ;                                // 解存在标志位
5   $k \leftarrow 1$ ;                                // 从第一个顶点开始
6  while  $k \geq 1$  do
7    while  $c[k] \leq 2$  do
8       $c[k] \leftarrow c[k] + 1$ ;                        // 尝试下一种颜色
9      if  $\forall (i, j) \in E, c[i] \neq c[j]$  then
10        $flag \leftarrow true$ 
11       break;                                // 找到合法解则退出循环
12     end
13     else if  $\forall i < k, (i, k) \in E \Rightarrow c[i] \neq c[k]$  then
14        $k \leftarrow k + 1$ ;                                // 当前是部分解则前进
15     end
16   end
17   if  $c[k] > 2$  then
18      $c[k] \leftarrow 0$ 
19      $k \leftarrow k - 1$ ;                                // 回溯到前一个顶点
20   end
21 end
22 if  $flag$  then
23   return  $c[1..n]$ ;                                // 输出合法着色方案
24 else
25   return "no solution";                                // 无解提示
26 end

```

**Algorithm 35: COLORITER** (迭代图着色算法)

对于这两种算法的时间复杂性, 我们注意到在最坏情况下生成  $O(3^n)$  个节点。对于每个生成的节点, 如果当前着色是合法的、部分解, 或者二者都不是, 那么需要  $O(n)$  的工作来检查。

因此, 在最坏情况下, 全部的运行时间是  $O(n3^n)$ 。

空间复杂度是  $\Theta(n)$ 。

### 8.2.2 流水作业调度问题

有  $n$  个作业 (编号为  $1 \sim n$ ) 要在由两台机器  $M_1$  和  $M_2$  组成的流水线上完成加工。每个作业加工的顺序都是先在  $M_1$  上加工, 然后在  $M_2$  上加工。 $M_1$  和  $M_2$  加工作业  $i$  所需的时间分别为  $a_i$  和  $b_i$  ( $1 \leq i \leq n$ )。

流水作业调度问题要求确定这  $n$  个作业的最优加工顺序, 使得从第一个作业在机器  $M_1$  上开始加工, 到最后一个作业在机器  $M_2$  上加工完成所需的时间最少。可以假定任何作业一旦开始加工, 就不允许被

中断，直到该作业被完成，即非优先调度。

```

Input:  $a[1..n], b[1..n]$ :  $n$  个作业在  $M_1$  和  $M_2$  上的处理时间
Output:  $bestx[1..n]$ : 最优调度排列,  $bestf$ : 最优调度总时间

1 for  $i \leftarrow 1$  to  $n$  do
2    $x[i] \leftarrow i$ ; // 初始化当前调度排列
3    $f2[i] \leftarrow 0$ ; // 初始化  $M_2$  完成时间数组
4 end
5  $bestf \leftarrow +\infty$ ; // 初始最优值为正无穷
6  $f1 \leftarrow 0$ ; //  $M_1$  累计完成时间

7 Procedure  $jobschedule(i)$ 
8   if  $i > n$  then
9     if  $f2[n] < bestf$  then
10       $bestf \leftarrow f2[n]$ ; // 更新最优值
11      for  $j \leftarrow 1$  to  $n$  do
12         $bestx[j] \leftarrow x[j]$ ; // 复制解向量
13      end
14    end
15    return
16  end
17  for  $j \leftarrow i$  to  $n$  do
18     $swap(x[i], x[j])$ ; // 生成候选排列
19     $f1 \leftarrow f1 + a[x[i]]$ ; // 更新  $M_1$  完成时间
20     $f2[i] \leftarrow \max(f1, f2[i-1]) + b[x[i]]$ ; // 计算  $M_2$  完成时间
21    if  $ubound(i) < bestf$  then
22       $jobschedule(i+1)$ ; // 递归搜索
23    end
24     $f1 \leftarrow f1 - a[x[i]]$ ; // 回溯恢复状态
25     $swap(x[i], x[j])$ ; // 恢复排列顺序
26  end

27 Function  $ubound(i)$ 
28    $tot-sum \leftarrow 0$ ; // 初始化未选择作业总时间
29   for  $j \leftarrow i+1$  to  $n$  do
30      $tot-sum \leftarrow tot-sum + b[x[j]]$ ; // 累计  $M_2$  处理时间
31   end
32   return  $f2[i] + tot-sum$ ; // 返回下界估计值

33  $jobschedule(1)$ ; // 从第 1 层开始搜索
34 return  $bestx[1..n], bestf$ 

```

**Algorithm 36:** JobScheduleREC (流水作业调度递归算法)

## 9 chapter9: 分支限界法

### 9.1 分支-限界法基本原理

- **分支-限界法定义**: 生成当前 E-结点 (扩展结点) 的全部子结点 (分支), 利用剪枝函数对不能生成含答案结点的子树剪枝 (限界) 的状态空间树检索方法。即, 每当访问一个结点 (E-结点) 时, 会生成这个结点的所有子结点 (广度优先搜索)
- **核心要素**: 多个活节点的组织, E-节点的检索, 剪枝函数的设计
- **FIFO 检索策略**: 类似于 BFS 的状态空间检索, 活结点采用普通队列组织 (first in first out), 选择方式太过死板, 不利于答案的得到。
- **LIFO 检索策略**: 类似于 D-检索的状态空间检索, 活结点表采用栈组织 (last in first out)
- **最优检索策略**: 最小耗费 (最大收益) 的方式检索, 活结点表采用优先级队列组织

目标状态是否可由初始状态到达的判定方法: 对于 15-迷问题 (4\*4 棋盘内通过合法移动恢复成目标排列)

给 4\*4 棋盘的方格编号 1 ~ 16 位置  $i$  表示目标排列中放  $i$  号牌的方格位置, 位置 16 是空格的位置。

$POSITION(i)$ : 编号  $i$  的牌在初始状态的位置号 ( $1 \leq i < 16$ )  $POSITION(16)$ : 空格的位置号,  $LESS(i)$ : 满足  $\{j | j < i \wedge POSITION(j) > POSITION(i)\}$  的数目。

若初始状态空格横纵坐标和为奇数,  $X = 1$ ; 否则  $X = 0$ 。

当且仅当  $\sum_{i=1}^{16} LESS(i) + X$  为偶数时, 目标状态可由初始状态到达。

- **活结点管理**: 使用优先队列 (堆结构) 组织活结点, 每个结点按”成本估计值”排序, 总是扩展当前成本最小的结点
- **成本计算**: 对于已部分解  $x[1..k]$ , 计算已发生成本 (从根到当前结点的实际代价) 和预估成本 (启发式函数估算完成解所需的最小代价), 总成本 = 已发生成本 + 预估成本 (结点的成本估计函数:  $\hat{c}(X) = f(h(X)) + \hat{g}(X)$ ). 如果  $X$  是一个答案结点或是一个叶子结点, 则有  $c(X) = \hat{c}(X)$ 
  - $\hat{g}(X)$  是由  $X$  到达一个答案结点所需做的附加工作的估计
  - $h(X)$  是由根结点到结点  $X$  的成本
  - $f(\cdot)$  是一个非降函数。 $f$  函数强使算法优先选择更靠近答案结点但又离根较近的结点。它也能调整  $h(X)$  和  $\hat{g}(X)$  在  $\hat{c}(X)$  中的影响比例。

- **搜索过程**:

1. 初始化优先队列 (加入根结点)
2. 循环直到队列为空:
  - 取出队首 (成本最小的 E-结点)
  - 生成所有子结点
  - 对每个子结点计算成本估计值, 检查剪枝条件 (若成本 > 当前最优解则丢弃), 存活结点入队

– 若到达叶结点则更新最优解

LC-检索的两种特殊情况:

1. 当  $f(h(x)) =$  结点  $X$  的级数, 且  $\hat{g}(X) = 0$  时 (忽略未来成本, 只考虑历史成本, 因而先考虑浅层结点): 若结点的  $\hat{c}$  值相同, 选取最先生成的活结点. 此时 LC-检索等价于 BFS 检索 (队列实现的宽度优先搜索)
2. 当  $f(h(x)) = 0$ , 且满足  $\forall Y \in \text{children}(X), \hat{g}(X) \geq \hat{g}(Y)$  时 (忽略历史成本, 只考虑未来成本, 因而先考虑深层结点): 若结点的  $\hat{c}$  值相同, 选取最晚生成的活结点. 此时 LC-检索等价于 D-检索 (堆栈实现活结点表)

成本函数  $c(\cdot)$ : 从根出发到最近目标结点路径上的每个结点赋予这条路径的长度作为它们的成本。

估计成本函数  $\hat{c}(\cdot)$ :  $\hat{c}(X) = h(X) + \hat{g}(X)$

- $h(X)$  是由根到结点  $X$  的路径的长度
- $\hat{g}(X)$  是对以  $X$  为根的子树中由  $X$  到目标状态的一条路径长度的估计。它至少应是能把状态  $X$  转换成目标状态所需的最小移动数
- $\hat{g}(X) =$  不在其目标位置的非空白牌数目
- 可以看出  $\hat{c}(X)$  是  $c(X)$  的下界

状态空间树  $T$ ，结点的代价函数  $\hat{c}$   
 可行解函数  $cost$ ， $\varepsilon > 0$ ，下界函数  $lb$   
**Input:** 假设：  $T$  至少包含一个可行解结点，且满足  $\forall X, lb(X) \leq c(X) \leq \hat{c}(X)$   
 $\varepsilon$  足够小，使得  $\forall X, Y, lb(X) < lb(Y) \Rightarrow lb(X) - \varepsilon < lb(X) < lb(Y)$   
**Output:** 函数值最大的解

```

1  $E \leftarrow T$ ;
2  $PARENT(E) \leftarrow 0$ ;
3 if  $T$  是可行解结点 then
4    $L \leftarrow \max(cost(T), lb(T) - \varepsilon)$ ;
5    $ans \leftarrow T$ ;
6 else
7    $L \leftarrow lb(T) - \varepsilon$ ;
8    $ans \leftarrow 0$ ;
9 end
10 初始化活结点表为空;
11 while true do
12   for  $E$  的每个儿子  $X$  do
13     if  $c(X) > L$  then
14        $ADD(X)$ ;
15        $PARENT(X) \leftarrow E$ ;
16       if  $X$  是可行解结点 且  $cost(X) > L$  then
17          $L \leftarrow \max(cost(X), lb(X) - \varepsilon)$ ;
18          $ans \leftarrow X$ ;
19       end
20       else if  $lb(X) - \varepsilon > L$  then
21          $L \leftarrow lb(X) - \varepsilon$ ;
22       end
23     end
24   end
25   if 活结点表为空 或 下一个  $E$  结点满足  $\hat{c}(E) \leq L$  then
26     输出从  $ans$  到  $T$  的路径;
27     输出最大值  $L$ ;
28     return;
29   end
30    $LARGEST(E)$  ; // 选择  $\hat{c}$  值最大的活结点作为  $E$ 
31 end

```

**Algorithm 37:** UCBB (基于代价的搜索算法)

$\varepsilon$  确保  $lb(X) < lb(Y) \Rightarrow lb(X) - \varepsilon < lb(Y)$ ，消除了边界值相等时的判断歧义。

## 9.2 分支限界法的应用

分支限界法的核心在于 **cost 函数设计**，通过评估部分解的潜在最优性实现高效剪枝。其通用模板为：

1. 初始化优先队列（按 cost 排序）
2. 扩展当前最优部分解的子节点
3. 计算子节点的 cost 值并剪枝

### 9.2.1 Cost 函数设计方法

对于线性松弛方法中的 Cost 函数：

$$C = V_{\text{selected}} + \sum \left( v_i \times \min \left( 1, \frac{W_{\text{remain}}}{w_i} \right) \right)$$

其中：

- $V_{\text{selected}}$  = 当前已选物品的总价值
- $W_{\text{remain}}$  = 背包/容器的剩余可用容量
- $v_i$  = 第  $i$  个物品的价值
- $w_i$  = 第  $i$  个物品的重量/体积

公式含义：

1.  $\min \left( 1, \frac{W_{\text{remain}}}{w_i} \right)$  计算每个物品能放入剩余容量的最大比例
  - 当  $\frac{W_{\text{remain}}}{w_i} \geq 1$  时，可取完整物品（比例 =1）
  - 当  $\frac{W_{\text{remain}}}{w_i} < 1$  时，只能取部分物品（比例 <1）
2.  $v_i \times \min(\dots)$  表示该物品的实际贡献价值
3. 求和项  $\sum$  计算所有未选物品的潜在最大价值

典型应用场景：

- 背包问题（Knapsack Problem）的线性规划松弛
- 资源分配问题中计算理论上界
- 分支定界法中的界限估计



## 9.2.2 关键实现技术

```

1 Function BranchAndBound(problem)
2    $Q \leftarrow \text{InitializePriorityQueue}();$ 
3   Insert( $Q$ , root_node);
4   while  $Q \neq \emptyset$  do
5      $N \leftarrow \text{ExtractMin}(Q);$ 
6     if IsCompleteSolution( $N$ ) then
7       | UpdateGlobalBest( $N$ );
8     else
9       foreach child node  $N_i$  of  $N$  do
10        |  $\text{cost}(N_i) \leftarrow \text{CalculateCost}(N_i);$  // 核心计算步骤
11        | if  $\text{cost}(N_i)$  better than current bound then
12          | Insert( $Q$ ,  $N_i$ );
13        | end
14      end
15    end
16  end
17  return global_best;

```

**Algorithm 38:** 分支限界法通用模板

采用 MST 下界作为 cost 函数：

$$\text{cost}(\pi) = \sum_{i=1}^k d_{\pi(i)} + \text{MST}(\text{未访问城市})$$

其中  $\pi$  为已访问路径， $d_{\pi(i)}$  为路径长度。

使用线性松弛上界：

$$\text{cost}(S) = \sum_{i \in S} v_i + \sum_{j \notin S} v_j \cdot \min \left( 1, \frac{W - \sum_{i \in S} w_i}{w_j} \right)$$

## 9.2.3 TSP 问题中的 MST 下界

$$C_{\text{MST}} = L_{\text{current}} + W_{\text{MST}}$$

- 变量说明：

- $L_{\text{current}}$ ：当前已构建路径的总长度
- $W_{\text{MST}}$ ：剩余未访问节点最小生成树 (MST) 的边权和

- 计算原理：

1. 通过 Prim 或 Kruskal 算法计算剩余节点的 MST
2. MST 权值和  $W_{\text{MST}}$  是连接剩余节点的最小代价
3. 最终路径长度  $\geq L_{\text{current}} + W_{\text{MST}}$

- 示例:

```

1  def tsp_lower_bound(current_path, remaining_nodes):
2      mst = compute_mst(remaining_nodes) # 计算剩余节点MST
3      return current_path.length + mst.total_weight

```

#### 9.2.4 背包问题中的线性松弛

$$C_{LP} = V_{\text{selected}} + \sum_{i \in \text{剩余物品}} \left( v_i \times \min \left( 1, \frac{W_{\text{remain}}}{w_i} \right) \right)$$

- 变量说明:

- $V_{\text{selected}}$ : 已选物品总价值
- $W_{\text{remain}}$ : 背包剩余容量
- $v_i/w_i$ : 物品的价值-重量比

- 计算原理:

1. 对剩余物品按  $v_i/w_i$  降序排序
2. 尽可能装入高价值密度物品（可分割）
3. min 函数确保不超过物品本身价值

- 分支定界法中的应用:

$$\text{全局上界} = \max_{\text{所有节点}} C_{LP}$$

- 算法实现:

```

1  def knapsack_upper_bound(selected, remaining):
2      sorted_items = sorted(remaining, key=lambda x: x.v/x.w, reverse=True)
3      bound = selected.total_value
4      for item in sorted_items:
5          if W_remain <= 0: break
6          take = min(1, W_remain/item.w)
7          bound += take * item.v
8          W_remain -= take * item.w
9      return bound

```

## 10 chapter10: 随机算法

### 10.1 导论

1. 与确定性算法相比，随机算法所需运行时间或空间通常较小；

2. 随机算法易于理解和实现。

1. **Las Vegas 算法**<sup>2</sup>: 总是给出正确的解, 或者无解

通常需要分析**算法的平均运行时间**, 运行时间本身是一个随机变量。

**期望运行时间是输入规模的多项式且总能给出正确答案的随机算法称为有效的拉斯维加斯算法**, 也称为 **ZPP 类算法** (zero-error probabilistic polynomial time) 算法, 即零错误概率多项式时间随机算法。

2. **Monte Carlo 算法**: 总是给出解, 偶尔可能是错误解, 可以通过多次运行原算法, 且满足每次运行时随机选择都相互独立, 使产生错误解的概率减到任意小。

通常需要分析**算法的出错概率**, 总是在**多项式时间内运行且出错概率不超过 1/3**的随机算法称为**有效的 \* 蒙塔卡洛型随机算法**, 也称为 **BPP 类算法** (bounded-error probabilistic polynomial time), 即错误概率有限的多项式时间随机算法。

错误类型:

- **弃真型错误**: 在求解一个判定问题时把本应该接受的输入误判为拒绝。
- **取伪型错误**: 就是把本应该拒绝的输入误判为接受。

根据是否同时出现两类错误, 蒙特卡洛型算法又分为:

- (a) **单侧错误的蒙特卡洛型算法**: 在所有输入上, 要么犯弃真的错误, 要么犯取伪的错误, 不同时出现两种不同的错误。
  - 有效的**弃真型**单侧错误随机算法称为 **RP 类算法** (Randomized Polynomial-time)
  - 有效的**取伪型**单侧错误随机算法称为 **coRP 类算法** (Complementary Randomized Polynomial-time, RP 的补类)
- (b) **双侧错误的蒙特卡洛型算法**: 在所有输入上可以同时出现两种不同的错误。

对于随机算法 A, 其在大小为 n 的一个实例 I 上的运行时间可能因不同次执行而异。因此, 更自然的性能度量是 **A 在实例 I 上的期望运行时间**, 也即算法 A 反复求解实例 I 所花费的平均时间。

- 对于确定算法, 求解期望运行时间时, 概率分布定义在**输入**上
- 对于随机算法, 求解期望运行时间时, 概率分布定义在**算法**上

**期望时间**: 对于大小为 n 的特定输入 I, 算法的期望时间就是在随机数的不同序列上的时间的平均:

$$ET(A, I) = \sum_{\text{random sequence } R} \text{probability}(R) * \text{time}(A, I, R)$$

**最坏情况下的期望时间**:

$$W\_ET(A) = \max_{\text{input } I \text{ with size } n} (\text{random sequence } R \text{ probability}(R) * \text{time}(A, I, R))$$

<sup>2</sup>Las Vegas 和 Monte Carlo 不是一种算法的名称, 而是对一类随机算法的特性的概括。不太可能为 **NP 完全问题**设计出多项式时间的随机算法

## 10.2 随机算法举例

这是一个很不错的讲解栏目：<https://www.luogu.com.cn/article/wegx023s>

### 10.2.1 随机快速排序

其实是在快速排序的基础上，增加了**随机选取**的步骤。

该算法在最坏情况的运行时间仍然是  $\Theta(n^2)$ ，但这个最坏情况不是由于输入形式，而是由于随机数生成器每次非常不幸地选择了不好的划分元素，这种情况不大可能发生。算法的期望运行时间为  $\Theta(n \log n)$ ，是一个 ZPP 类算法

### 10.2.2 随机选择

在最坏情况下，随机数生成器在第  $j$  次迭代中选择第  $j$  小的元素，因此只丢弃一个元素。由于将  $A$  划分为  $A_1$ 、 $A_2$  和  $A_3$  所需的比较次数正好为  $n$ ，因此算法在最坏情况下进行的元素比较总数为

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$$

设  $C(n)$  为对输入数组有  $n$  个元素时进行元素比较运算的**期望次数**。由于随机选择的  $v$  可以等概率假设为  $1, 2, \dots, n$  中的任何一个，因此根据  $v < k$  或  $v > k$ ，有两种情况需要考虑。

- $v < k$ : 剩余元素的数量为  $n - v$  ( $v$  右侧的元素数量为  $n - v$ )
- $v > k$ : 剩余元素的数量为  $v - 1$  ( $v$  左侧的元素数量为  $v - 1$ )

因而有：

$$C(n) = n + \frac{1}{n} \sum_{v=1}^n \begin{cases} C(v-1) & \text{if } v < k \\ C(n-v) & \text{if } v > k \end{cases}$$

由数学归纳法，假设  $C(m) \leq 4m$  对所有  $m < n$  成立，下证  $C(n) \leq 4n$ ：

$$\begin{aligned} C(n) &\leq n + \frac{1}{n} \sum_{v=1}^n 4 \cdot \max(n-v, v-1) \\ &\leq n + \frac{8}{n} \cdot \sum_{v=1}^{\frac{n}{2}} (n-v) \\ &\leq n + \frac{8}{n} \cdot \frac{3n^2}{8} \\ &= 4n \end{aligned}$$

又由于每个元素至少被检查一次，因此  $C(n) \geq n$ ，因此算法的期望运行时间是  $\Theta(n)$ ，是 ZPP 类算法。

### 10.2.3 多选问题

设有  $n$  个元素的数组  $A$ ，并且设  $K[1..r]$  ( $1 \leq r \leq n$ ) 是一个由 1 到  $n$  之间的  $r$  个正数构成的有序数组，即一个排序数组。

多选问题就是对于所有  $i$ ， $1 \leq i \leq r$ ，选择  $A$  中的第  $K[i]$  小的元素。假定  $A[1..n]$  中所有元素各不相同。如果  $r = 1$ ，该问题即为选择问题。如果  $r = n$ ，该问题即为排列问题。

设  $A[1..n] = [a_1, a_2, \dots, a_n]$ ;  $K[1..r] = [k_1, k_2, \dots, k_r]$ , 设中间排位为  $k = k_{\frac{r}{2}}$ , 可使用 SELECT 算法发现第  $k$  小的元素  $a$ . 接着根据  $a$  把  $A$  划分成两个子序列  $A_1$  和  $A_2$ , 其中  $A_1$  中元素个数大于等于  $a$ ,  $A_2$  中元素个数小于等于  $a$ . 令  $K_1 = [k_1, k_2, \dots, k_{\frac{r}{2}}]$ ,  $K_2 = [k_{\frac{r}{2}+1}, k_{\frac{r}{2}+2}, \dots, k_r]$ . 递归调用, 一个关于  $A_1$  和  $K_1$  的递归调用, 一个关于  $A_2$  和  $K_2$  的递归调用。

Multiselect 算法的时间复杂度是  $\Theta(n \log r)$ . 这是因为调用 SELECT 算法的复杂度是  $\Theta(n)$ , 所以有递推式

$$T(n, r) = \Theta(n) + T(n_1, \frac{r}{2}) + T(n_2, \frac{r}{2}), n_1 + n_2 \leq n$$

递归深度是  $\log r$ , 所以时间复杂度是  $\Theta(n \log r)$ .

将多选问题要寻找的元素称为靶标。随机一致地挑选一个元素  $a$  (而不是通过 SELECT 精心选择), 将数组  $A$  关于  $a$  分割成两部分: 一部分元素小于  $a$ , 另一部分元素大于  $a$  处理逻辑:

如果两部分均含有靶标, 则递归调用算法继续求解; 否则, 丢弃不含有靶标的部分, 对含有靶标部分继续求解

该算法以概率  $1 - O(\frac{1}{n})$  的时间复杂度为  $O(n \log r)$ .  $r$  个靶标的分配可以看作  $r$  次独立伯努利试验, 其分布近似二项分布  $B(r, 1/2)$ . 根据 Hoeffding 不等式, 靶标分配的偏离概率满足:

$$P\left(\left|\text{左侧靶标数} - \frac{r}{2}\right| \geq \epsilon r\right) \leq 2 \exp(-2\epsilon^2 r)$$

当  $r \geq c \log n$  时, 此概率上界为  $O(1/n)$ 。

#### 10.2.4 随机取样

首先将所有  $n$  个元素标记为未选中。接下来, 重复以下步骤, 直到选择了  $m$  个元素:

1. 生成一个介于 1 和  $n$  之间的随机数  $r$
2. 如果  $r$  被标记为未选中, 则将其标记为选中并将其添加到样本中

算法的期望运行时间  $T(n) = \Theta(n)$  如果  $m > \frac{n}{2}$ , 那么应该考虑反面, 改为随机选择  $n - m$  个整数丢弃, 将剩下的整数添加到样本中。

设第  $k$  次选择时已选中  $k$  个元素, 则:

- 成功选中新元素的概率  $p_k = \frac{n-k}{n}$
- 失败 (重复选中) 的概率  $q_k = 1 - p_k = \frac{k}{n}$

根据几何分布, 第  $k+1$  个新元素的期望尝试次数为:

$$E_k = \frac{1}{p_k} = \frac{n}{n-k}$$

每次尝试的时间成本为  $\Theta(1)$  (仅需检查布尔数组  $S$ )。

$$\begin{aligned} T(n) &= \sum_{k=0}^{m-1} E_k \cdot \Theta(1) \\ &= \sum_{k=0}^{m-1} \frac{n}{n-k} \\ &= n \left( \frac{1}{n} + \frac{1}{n-1} + \dots + \frac{1}{n-m+1} \right) \\ &= n \cdot (H_n - H_{n-m}) \quad (\text{调和数展开}) \end{aligned}$$

利用调和数近似  $H_n \approx \ln n + \gamma$ :

$$T(n) \approx n(\ln n - \ln(n-m)) = n \ln \left( \frac{n}{n-m} \right)$$

- 当  $m = O(n)$  时 (如  $m = \sqrt{n}$ ):  $T(n) \approx n \cdot \frac{m}{n} = \Theta(m)(\ln x \approx x - 1)$
- 当  $m = \Theta(n)$  时 (如  $m = n/2$ ):  $T(n) = \Theta(n)$

因此对于任意  $m < n$ , 最坏情况下  $T(n) = \Theta(n)$ 。

### 10.2.5 素数测试

易知除 2, 3 以外的所有素数都是  $6k \pm 1$  的形式, 因此可以用  $6k \pm 1 \leq \sqrt{n}$  来对  $n$  是否是素数进行测试。

1. **随机抽取一个数**  $a$  ( $1 \leq a < n$ );
2. **检查等式**: 验证涉及  $a$  和  $n$  的特定数论等式 (与所选测试方法对应)。  
若等式不成立, 则:
  - $n$  是合数 (Composite)
  - $a$  称为合数的**证据** (Witness)
  - 测试终止
3. **重复迭代**: 返回步骤 1, 直到达到预设置信度。

现在我们先介绍幂运算: 如果每次乘法花费 1 个时间单元, 算法的运行时间是  $\Theta(\log n)$ 。但如果是任意大的整数, 两数相乘的时间是  $O(\log^2 n)$ , 则算法的运行时间是  $O(\log^3 n)$ 。

利用费马定理: 如果  $n$  是素数, 那么对于所有的  $a \not\equiv 0 \pmod{n}$ , 都有  $a^{n-1} \equiv 1 \pmod{n}$ , 如果通过  $a$  的测试, 则成  $n$  是基 2 的伪素数, 例如 341。而对于小于 100000 的数中, 只有 78 个数计算出错, 这是因为费马定理的逆并不正确。一类被称为**卡迈克尔数**的合数  $n$ , 它对于所有相对于  $n$  互素的正整数  $a$  均满足费马定理, 他们在小于  $1e8$  的范围内只有 255 个。

引理: 如果合数  $n$  不是一个卡迈克尔数, 那么算法 (随机选择  $a$  并验证费马定理) 将检测出  $n$  为合数的概率至少是  $\frac{1}{2}$ 。

若  $n$  是合数但非卡迈克尔数, 则存在至少一个  $b \perp n$  使得  $b^{n-1} \not\equiv 1 \pmod{n}$ , 则定义集合

$$W = \{a \in \{1, 2, \dots, n-1\} | a^{n-1} \not\equiv 1 \pmod{n}\}$$

而对于  $W$  在整数集  $Z_n^*$  上的补集  $G$ , 易知  $G$  是  $Z_n^*$  的一个乘法子群, 那么由拉格朗日定理知  $|G|$  整除  $|Z_n^*| = \phi(n)$ , 所以  $|G| \leq \frac{1}{2}\phi(n)$ , 所以  $|W| \geq \frac{1}{2}\phi(n)$ , 引理得证。

再考虑 Miller-Rabin 算法, 设  $n$  为奇素数, 则存在  $q, m$  使得:

$$n-1 = 2^q m (q \leq 1)$$

构造序列:

$$a^m \pmod{n}, a^{2m} \pmod{n}, a^{4m} \pmod{n}, \dots, a^{2^{q-1}m} \pmod{n}$$

由费马定理知, 最后一项  $a^{n-1} \equiv 1(\text{mod } n)$ . 而如果  $x^2 \equiv 1(\text{mod } n)$  的根是  $x = \pm 1$  称为平凡根, 其他根是不平凡的, 如果存在非平凡根, 那么一定是合数。

所以检测该序列中某项是 1, 那么前一项必须是 1 或者  $n-1$  否则  $n$  是合数。

每次测试出错 (把合数判定为素数) 概率至多是  $\frac{1}{2}$ , 这样我们重复运行  $k$  次, 就可以将出错的概率降低到  $2^{-k}$ , 那么取  $k = \lceil \log n \rceil$ , 失败的概率不超过  $2^{-\lceil \log n \rceil} \leq \frac{1}{n}$ , 而这就是 **Miller-Rabin 算法**。

### 10.2.6 验证串的相等性

对于一个比特串  $w$ , 设  $I(w)$  是  $w$  表示的一个整数, 那么可以选一个素数  $p$ , 这样得到一个指纹函数:

$$I_p(x) = I(x)(\text{mod } p)$$

对于  $I_p(x) = I_p(y)$ , 而  $x \neq y$  的现象, 我们称之为**假匹配**。每次验证时重新选择  $p$ , 并重复选择  $p$ , 可以增强  $x = y$  的可信度。

我们知道, 设  $\pi(n)$  是小于  $n$  的不同素数的个数, 那么  $\pi(n)$  趋近于  $\frac{n}{\ln n}$ . 对于两个长度为  $n$  的串  $x, y$ , 如果发生假匹配, 那么  $p | (I(x) - I(y))$ , 所以其概率为:

$$\frac{|\{p \leq M | p \text{ is prime and } p | (I(x) - I(y))\}|}{\pi(M)} \leq \frac{\pi(n)}{\pi(M)}$$

**关键观察:** 假匹配发生时, 素数  $p$  必须满足  $p | (I(x) - I(y))$ 。由于  $x$  和  $y$  是长度为  $n$  的比特串,  $I(x)$  和  $I(y)$  的绝对值不超过  $2^n - 1$ , 因此

$$|I(x) - I(y)| \leq 2^n - 1.$$

(这个结论需要在  $n > 11$  后成立, 这是由切比雪夫函数完成估计的, 最终  $n$  以内的素数的乘积的增长速率约为  $e^{0.9n}$ , 是高于  $2^n - 1$  这个速率的, 所以该估计是正确的)

而如果取  $M = n^2$ , 重复执行  $k$  次算法, 这样最后概率最大是  $(\frac{2}{n})^k$ .

### 10.2.7 模式匹配

确定模式串是否出现在文本串中 (假设都是 0,1 串)。利用随机算法比较模式指纹  $I_p(Y)$  和文本块  $I_p(X(j))$ , 让文本块的指纹要易于计算, 最好还可以通过  $X(j)$  计算出  $X(j+1)$

仿照上一题思想, 我们把这里的指纹就假定为选定的串对应的数的大小, 那么有:

$$I_p(X(j+1)) = (2I_p(X(j)) - 2^m x_j + x_{j+m})(\text{mod } p)$$

易知该算法的时间复杂度是  $O(m+n)$

### 10.2.8 货郎担问题

假设有一个旅行商人要拜访  $n$  个城市, 路径选择需满足以下限制:

1. 每个城市只能拜访一次;
2. 最终必须回到出发城市。

优化目标：选择总路程最短的路径

**Input:** 城市数  $n$ ，每两城市间距离矩阵  $D$

**Output:** 最优路径  $bestPath$ ，路径总长  $bestDistance$

```

1  $bestPath \leftarrow []$  // 初始化最优路径为空
2  $bestDistance \leftarrow \infty$  // 初始化最短距离为无穷大
3  $path \leftarrow$  随机排列 $(1, 2, \dots, n)$  // 生成初始随机路径
4  $distance \leftarrow$  计算路径总长度( $path, D$ ) // 计算初始路径长度
5 for  $j \leftarrow 1$  to  $iteration$  do
6   if  $distance < bestDistance$  then
7      $bestPath \leftarrow path$  // 更新最优路径
8      $bestDistance \leftarrow distance$  // 更新最短距离
9   end
10   $path \leftarrow$  生成新随机路径() // 重新采样路径
11   $distance \leftarrow$  计算路径总长度( $path, D$ ) // 重新计算长度
12 end
13 return  $bestPath, bestDistance$  // 返回找到的最优解

```

**Algorithm 47:** 随机采样求解 TSP 算法 (RandomTSP)

### 10.3 随机游走算法

常用有限马尔科夫链来分析随机游走算法。

马尔可夫特性：随机过程在给定  $t$  时刻的状态为  $X_t$ ，该过程的后续状态及其出现的概率与  $t$  之前的状态无关。也就是说，过程当前的状态包括了过程所有的历史信息，该过程的进一步发展完全由当前状态所决定，与当前状态之前的历史无关，这种性质也称为**无后效性**或**无记忆性**。

马尔科夫不等式：设  $X$  为只取非负的随机变量，数学期望  $E(X) = \mu$ ，对于任意  $\varepsilon > 0$ ， $P(X \geq \varepsilon) \leq \frac{\mu}{\varepsilon}$

1. **Integral method** (for continuous case):

$$E(X) = \int_0^{\infty} xf(x)dx \geq \int_{\varepsilon}^{\infty} xf(x)dx \geq \int_{\varepsilon}^{\infty} \varepsilon f(x)dx \geq \varepsilon P(X \geq \varepsilon)$$

2. **Indicator method** (general): Define  $Y = \varepsilon \cdot \mathbf{1}_{\{X \geq \varepsilon\}}$ , then:

$$E(Y) = \varepsilon P(X \geq \varepsilon) \leq E(X)$$

设一个离散的随机序列  $\{X_0, X_1, \dots\}$ ，其中每个  $X_i$  都从有限集中取值，若它满足

$$P(X_{i+1} = x_{i+1} | X_i = x_i, X_{i-1} = x_{i-1}, \dots, X_0 = x_0) = P(X_{i+1} = x_{i+1} | X_i = x_i) = p_{x_i, x_{i+1}}$$

这称之为**有限马尔科夫链**

对于有限马氏链，不妨设  $X_t$  取值的状态空间为  $\{0, 1, \dots, n\}$ ，于是  $p_{i,j}$ （从状态  $i$  转移到状态  $j$  的概率）可组成一个  $n+1$  阶方阵

$$P = [p_{i,j}],$$

称为**一步转移矩阵**，每一行元素之和等于 1。



设  $p_i(t)$  表示在时刻  $t$  处于状态  $i$  的概率, 则

$$p_i(t) = p_0(t-1)p_{0i} + p_1(t-1)p_{1i} + \cdots + p_{n-1}(t-1)p_{(n-1)i} + p_n(t-1)p_{ni}$$

设  $\mathbf{p}(t)$  表示向量  $(p_0(t), p_1(t), \dots, p_n(t))$ , 有

$$\mathbf{p}(t) = \mathbf{p}(t-1)P$$

对于任意  $m$ , 定义  $m$  步转移矩阵

$$P^{(m)} = [p_{i,j}^{(m)}],$$

其中

$$p_{i,j}^{(m)} = P(X_{t+m} = j \mid X_t = i).$$

因此,

$$P^{(m)} = P^m, \mathbf{p}(t+m) = \mathbf{p}(t)P^m.$$

## 10.4 二元布尔可满足问题

**Input:** 整数  $n$  (变元数量), 整数  $m$  (子句数量), 合取范式  $F$  (每个子句至多两个文字)

**Output:** 可满足赋值, 或宣布不可满足

```

1 随机初始化赋值  $\sigma$ : 对每个变元  $x_i$  随机赋 True 或 False;
2 for  $k \leftarrow 1$  to  $2mn^2$  do
3   if  $\sigma$  满足  $F$  then
4     return  $\sigma$ ;                                // 找到可满足解
5   end
6   随机选择  $F$  中一个不满足的子句  $C$ ;
7   从  $C$  中随机选择一个文字  $l$ ;
8   翻转  $l$  对应变元的赋值;                      // 将 True 改为 False 或反之
9 end
10 return "不可满足";                             // 超过最大迭代次数未找到解

```

### Algorithm 48: 2-SAT 随机局部搜索算法

**定理:** 若输入公式是不可满足的, 则上述随机游走算法正确宣布不可满足; 若输入公式是可满足的, 则上述算法以  $1 - \frac{1}{2^m}$  概率找到可满足赋值。

定义马尔可夫链  $\{Y_0, Y_1, Y_2, \dots\}$  如下:

$$Y_0 = X_0$$

$$P(Y_{t+1} = 1 \mid Y_t = 0) = 1$$

$$P(Y_{t+1} = j + 1 \mid Y_t = j) = \frac{1}{2}$$

$$P(Y_{t+1} = j - 1 \mid Y_t = j) = \frac{1}{2} \quad (j > 0)$$

显然有:

$$P(Y_{t+1} = j + 1 \mid Y_t = j) = \frac{1}{2} \geq P(X_{t+1} = j + 1 \mid X_t = j)$$

$$P(Y_{t+1} = j - 1 \mid Y_t = j) = \frac{1}{2} \geq P(X_{t+1} = j - 1 \mid X_t = j)$$

从任意状态出发,  $Y_t$  比  $X_t$  花费更长的期望时间进入状态  $n$ 。因此  $Y_t$  进入状态  $n$  的时间作为算法期望运行时间的上界。

设  $h_j$  表示从状态  $j$  到状态  $n$  的期望运行时间, 则有:

$$\begin{aligned} h_n &= 0 \\ h_0 &= h_1 + 1 \\ h_j &= \frac{h_{j+1} + 1}{2} + \frac{h_{j-1} + 1}{2}, \quad 0 < j < n \end{aligned}$$

由归纳法可证, 对于所有  $j$ , 有:

$$h_j \leq n^2 - j^2 \leq n^2$$

因此有:

$$E[X \text{ 从 } X_0 \text{ 到达 } n \text{ 的时间}] \leq E[Y \text{ 从 } Y_0 \text{ 到达 } n \text{ 的时间}] \leq n^2$$

取  $\varepsilon = 2n^2$ , 根据马尔可夫不等式, 则有:

$$P(X \text{ 从 } X_0 \text{ 到达 } n \text{ 的时间} \geq 2n^2) \leq \frac{n^2}{2n^2} = \frac{1}{2}$$

证明:

- **不可满足情形:** 显然, 算法无法找到可满足赋值, 因此正确宣布不可满足。
  - **可满足情形:** 算法有可能找不到可满足赋值而出错, 下面证出错概率为  $\frac{1}{2^m}$ 。
1. **汉明距离定义:** 设  $\sigma^*$  为最优赋值, 当前赋值  $\sigma$  与  $\sigma^*$  的汉明距离  $d(\sigma, \sigma^*)$  表示两者取值不同的变量个数。
  2. **改进概率:** 对每个不满足子句, 至少以  $\frac{1}{2}$  概率减少  $d(\sigma, \sigma^*)$  (因子句最多 2 个文字)。
  3. **收敛分析:** 经过  $2n^2m$  步迭代, 未到达  $\sigma^*$  的概率:

$$P(\text{失败}) \leq \left(1 - \frac{1}{2n}\right)^{2n^2m} \approx e^{-nm} \leq \frac{1}{2^m}$$

## 10.5 本章作业

### 10.5.1 优惠券收集

有  $n$  种优惠券, 每种优惠券数目不限。每次实验随机抽取一张优惠券, 优惠券来自  $n$  种中任何一种的概率均等。计算需要多少次实验才能使得收集的优惠券包含  $n$  种中每一种至少一张。

设  $X_i$  表示在已收集  $i-1$  种优惠券后, 首次抽到第  $i$  种新优惠券所需的次数 ( $i = 1, 2, \dots, n$ )。

在已收集  $k$  种优惠券时, 抽到新优惠券的概率为  $p_k = \frac{n-k}{n}$ , 因此  $X_{k+1}$  服从参数为  $p_k$  的几何分布, 其期望为:

$$E(X_{k+1}) = \frac{1}{p_k} = \frac{n}{n-k}$$

集齐所有  $n$  种优惠券的总次数  $T$  为各阶段次数之和:

$$T = X_1 + X_2 + \dots + X_n$$

总期望为:

$$E(T) = \sum_{k=0}^{n-1} \frac{n}{n-k} = n \sum_{k=1}^n \frac{1}{k} = nH_n$$

当  $n$  较大时,  $H_n \approx \ln n + \gamma$ 。因此:

$$E(T) \approx n \ln n + \gamma n$$

集齐  $n$  种优惠券的平均实验次数为:

$$E(T) = nH_n = n \left( 1 + \frac{1}{2} + \cdots + \frac{1}{n} \right)$$

对于大  $n$ , 可近似为  $n \ln n + \gamma n$ , 其中  $\gamma \approx 0.577$ ,

### 10.5.2 用公平硬币随机序列生成

假定你有一枚公平硬币, 请设计一个有效的随机算法用来生成整数  $1, 2, \dots, n$  的一个随机排列。并分析你的算法的时间复杂性。

**Input:** 整数  $n$  (排列长度)

**Output:** 数组  $A[1..n]$  包含  $1$  到  $n$  的随机排列

```

1 初始化数组  $A$ :  $A[i] \leftarrow i$  对  $i = 1$  到  $n$ ;
2 for  $i \leftarrow n$  downto  $2$  do
3   生成  $\lceil \log_2 i \rceil$  次公平硬币投掷, 其中正面朝上为  $1$ , 反面为  $0$  来得到随机数的二进制串, 得到
   随机数  $j \in [0, 2^{\lceil \log_2 i \rceil} - 1]$ ;
4   while  $j \geq i$  do
5     重新生成  $j$ ; // 拒绝采样保证均匀性
6   end
7   交换  $A[i]$  与  $A[j+1]$ ;
8 end
9 return  $A$ 
```

#### Algorithm 49: Fisher-Yates 随机排列算法 (基于公平硬币)

考虑  $i$  次枚举后才找到

$$\sum_{i=1}^{\infty} q^i = \frac{q}{1-q} = 1-p \quad q = 1 - \frac{1}{2-p}$$

在最优情况下, 所有的  $j$  都被接受, 最优时间复杂度是

$$\sum_{i=2}^n \lceil \log_2 i \rceil \triangleq C_{opt}(n)$$

有:

$$C_{opt}(n) \leq \sum_{i=2}^n \log_2 n = O(n \log n)$$

也有: (此处省略分部积分的步骤)

$$C_{opt}(n) \geq \sum_{i=2}^n \log_2 i = \sum_{i=2}^n \int_{i-1}^i \log_2 x dx \geq \sum_{i=2}^n \int_{i-1}^i \log_2 x dx = \int_1^n \log_2 x dx = O(n \log n)$$

因此由夹逼定理有:  $C_{opt}(n) = O(n \log n)$

下面考虑最坏情况  $C_{wor}(n)$

对于  $i$  情况下单次  $j$  的取值, 拒绝概率为:

$$P_{reject}(i) = \frac{2^{\lceil \log_2 i \rceil} - i}{2^{\lceil \log_2 i \rceil}} = 1 - \frac{i}{2^{\lceil \log_2 i \rceil}} \triangleq p_i$$

考虑  $i$  情况下  $k$  次  $j$  的取值被拒绝的概率是  $p_i^k$ , 每次被拒绝都会多进行  $\lceil \log_2 i \rceil$  次硬币投掷实验, 因此  $k$  次取值后产生的次数为:  $p_i^k \cdot k \lceil \log_2 i \rceil$ , 因此  $i$  情况下的总投掷次数为: (跳过等差乘等比级数的数学推导)

$$w_i = \sum_{k=1}^{\infty} p_i^k \cdot k \lceil \log_2 i \rceil = \frac{p_i(1+p_i)}{(1-p_i)^2} \lceil \log_2 i \rceil = \frac{(2^{\lceil \log_2 i \rceil} - i)(2^{\lceil \log_2 i \rceil + 1} - i)}{i^2} \leq \frac{i \cdot 3i}{i^2} = 3$$

因为拒绝而多出的总次数为:

$$C_{penalty}(n) = \sum_{i=2}^n w_i \leq 3n = O(n)$$

所以最坏时间复杂度为:

$$C_{wor}(n) = C_{penalty}(n) + C_{opt}(n) = O(n \log n)$$

综上, 该算法的时间复杂度是  $O(n \log n)$  (当然可以改成如果  $j > i$ , 就  $j \% i = i$ , 这样也就省了这步计算了)

### 10.5.3 矩阵互逆

给定两个  $n \times n$  方阵  $A$  和  $B$ , 判定它们是否互逆 (即  $AB = I$ )。算法基于蒙特卡罗方法, 通过随机采样验证  $AB$  矩阵元素的性质:

- 对角线元素应为 1 (允许误差  $\epsilon$ )
- 非对角线元素应为 0 (允许误差  $\epsilon$ )

**Input:** 矩阵  $A, B$ , 误差阈值  $\epsilon$ , 采样次数  $T$

**Output:** YES 或 NO

```

1 for  $t \leftarrow 1$  to  $T$  do
2    $J \leftarrow \text{rand}(1, n)$  // 随机选行
3    $K \leftarrow \text{rand}(1, n)$  // 随机选列
4    $\text{ans} \leftarrow 0$  for  $l \leftarrow 1$  to  $n$  do
5      $\text{ans} \leftarrow \text{ans} + A[J][l] \times B[l][K]$ 
6   end
7   if ( $J = K$  且  $|\text{ans} - 1| > \epsilon$ ) 或 ( $J \neq K$  且  $|\text{ans}| > \epsilon$ ) then
8     return NO
9   end
10 end
11 return YES

```

- 时间复杂度:  $O(T \cdot n)$

- 每次采样需要  $O(n)$  时间计算矩阵元素
- 共进行  $T$  次采样
- 空间复杂度:  $O(1)$  (仅需存储临时变量)

#### 10.5.4 三元可满足问题

请设计随机游走算法求解三元可满足问题，并分析算法的期望运行时间。

**Input:** 3-SAT 公式  $F$ , 最大迭代次数  $T$

**Output:** 满足  $F$  的赋值 (若找到), 否则返回“无解”

```

1 随机初始化变量赋值  $\mathbf{x} = (x_1, \dots, x_n)$ ;
2 for  $t \leftarrow 1$  to  $T$  do
3   if  $\mathbf{x}$  满足  $F$  then
4     return  $\mathbf{x}$ ;
5   end
6   随机选择一个不满足的子句  $C$ ;
7   随机选择  $C$  中的一个变量  $x_j$ ;
8   翻转  $x_j$  的值 ( $\text{True} \leftrightarrow \text{False}$ );
9 end
10 return “无解”;
```

- 递推关系 (3 文字子句):

$$h_j = \frac{2}{3}h_{j+1} + \frac{1}{3}h_{j-1} + 1$$

- 特征根:

$$r_1 = 1, r_2 = -\frac{1}{2} \implies h_j \approx 3^j$$

- 指数下界:

$$E[T] \geq 3^n - 1 = \Omega(3^n)$$

#### 10.5.5 拉斯维加斯型与蒙特卡洛型的期望运行时间推导关系

设  $A$  是一个蒙特卡洛随机算法, 满足:

- 期望运行时间不超过  $T(n)$
- 返回正确解的概率为  $p(n)$
- 任何解的正确性可在  $T'(n)$  时间内验证

我们构造拉斯维加斯算法  $A'$  如下:

1. 运行  $A$  (耗时  $\leq T(n)$ )
2. 验证解的正确性 (耗时  $\leq T'(n)$ )
3. 若正确则返回解, 否则重复验证

$$E[\text{时间}] \leq \frac{T(n) + T'(n)}{p(n)}$$

期望时间公式成立因为：

$$E[\text{总时间}] = \sum_{k=1}^{\infty} [(1 - p(n))^{k-1} p(n) \cdot k(T(n) + T'(n))] = \frac{T(n) + T'(n)}{p(n)}$$

该转换在保证结果正确性的前提下，仅增加了多项式时间开销

## 11 chapter11: 近似算法

### 11.1 导入

#### 11.1.1 组合优化问题

组合优化问题  $\Pi$  不是最小化问题就是最大化问题。它由三个部分组成：

1. 一个实例集合  $D_{\Pi}$ ；
  2. 对于每个实例  $I \in D_{\Pi}$ ，存在  $I$  的一个候选解的有限集合  $S_{\Pi}(I)$ ；
  3. 实例  $I \in D_{\Pi}$  的每个解  $\sigma \in S_{\Pi}(I)$ ，存在一个值  $f_{\Pi}(\sigma)$ ，称为  $\sigma$  的解值。
- 如果  $\Pi$  是最小化问题，那么实例  $I \in D_{\Pi}$  的最优解  $\sigma^*$  满足：

$$\forall \sigma \in S_{\Pi}(I), f_{\Pi}(\sigma^*) \leq f_{\Pi}(\sigma)$$

- 最大化问题的最优解类似定义：

$$\forall \sigma \in S_{\Pi}(I), f_{\Pi}(\sigma^*) \geq f_{\Pi}(\sigma)$$

- 最优解值记为  $OPT(I) = f_{\Pi}(\sigma^*)$ 。

#### 11.1.2 近似算法

最优化问题  $\Pi$  的一个近似算法  $A$  是一个（**多项式时间**）算法，给定一个实例  $I \in D_{\Pi}$ ，算法输出某个解  $\sigma \in S_{\Pi}(I)$ 。算法  $A$  得到的解也称为近似解，值  $f_{\Pi}(\sigma)$  记为  $A(I)$ 。

数学表达：

- 问题实例：  $I \in D_{\Pi}$
- 可行解集：  $S_{\Pi}(I)$
- 近似解：  $\sigma \in S_{\Pi}(I)$
- 解值表示：  $A(I) = f_{\Pi}(\sigma)$

### 11.1.3 装箱问题

- **问题定义**: 给定一个大小在 0 和 1 之间的物品集合, 将这些物品装到容量为 1 的箱子中, 要求使用箱子数目最少 (最小化问题)。
- **实例集**  $D_{\Pi}$ : 由所有实例  $I = \{s_1, s_2, \dots, s_n\}$  组成, 其中  $\forall j (1 \leq j \leq n)$ , 有  $s_j \in (0, 1]$ 。
- **解集合**  $S_{\Pi}(I)$ : 由子集集合  $\sigma = \{B_1, B_2, \dots, B_k\}$  构成, 满足:
  1.  $\sigma$  是  $I$  的不相交划分
  2.  $\forall j (1 \leq j \leq k), \sum_{s \in B_j} s \leq 1$
- **解值**:  $f_{\Pi}(\sigma) = |\sigma| = k$  (使用的箱子数量)。
- 最优解是符合条件的具有最小子集数的  $\sigma$ 。
- 令  $A$  是为每一物品分配一个箱子的 (平凡) 算法。根据定义 ( $A$  的时间复杂度是多项式级别的),  $A$  是一个近似算法。但显然,  $A$  不是一个好的近似算法。

### 11.1.4 差界

- 对于问题的所有实例  $I$ , 希望近似算法  $A$  满足:

$$|A(I) - OPT(I)| \leq K \quad (K \text{ 为常数})$$

- **定义**: 若对每个实例  $I$  都有

$$OPT(I) - K \leq A(I) \leq OPT(I) + K \iff |A(I) - OPT(I)| \leq K$$

则称算法  $A$  具有**有界误差**或**差界**  $K$

- 这也称为近似算法的**绝对性能保证** (absolute performance guarantee)
- 适用于绝对误差敏感的场景 (如资源分配问题中固定容量的偏差)
- 只有很少的 NP 困难问题已知差界

### 11.1.5 相对性能界

- **定义**: 设  $A$  是求解优化问题  $\Pi$  的近似算法,  $I$  是  $\Pi$  的实例

$$R_A(I) = \max \left( \frac{A(I)}{OPT(I)}, \frac{OPT(I)}{A(I)} \right) \geq 1$$

- 称为解  $A(I)$  的**近似度** (近似比)
- 提供了算法解质量的统一测度, 也称**相对近似度**
- **绝对近似度**:

$$R_A = \inf \{r \geq 1 \mid R_A(I) \leq r, \forall I \in D_{\Pi}\}$$

表示所有实例上的最大近似度

- **渐进近似度**:

$$R_A^{\infty} = \inf \{r \geq 1 \mid \exists N, R_A(I) \leq r, \forall I \in D_{\Pi}, |I| \geq N\}$$

表示足够大实例上的近似度

### 11.1.6 近似方案

- **定义**：最优化问题的**近似方案**是算法簇  $\{A_\epsilon\}_{\epsilon>0}$ ，满足

$$R_{A_\epsilon} \leq 1 + \epsilon$$

- 近似方案是近似度收敛于 1 的一系列算法
- 可视为算法  $A$ ，输入实例  $I$  和误差界  $\epsilon$ ，满足

$$R_A(I, \epsilon) \leq 1 + \epsilon$$

- **多项式近似方案 (PAS)**：近似方案  $\{A_\epsilon\}_{\epsilon>0}$  中，每个  $A_\epsilon$  在  $|I|$  的多项式时间内运行。注意：PAS 的运行时间对  $|I|$  是多项式，但对  $1/\epsilon$  可能非多项式
- **完全多项式近似方案 (FPAS)** 是近似方案  $\{A_\epsilon\}_{\epsilon>0}$ ，每个  $A_\epsilon$  在  $|I|$  和  $1/\epsilon$  的多项式时间内运行
- **伪多项式时间算法**：在  $L$  的多项式时间内运行的算法，其中  $L$  是输入实例的最大数值。示例：0/1 背包问题的动态规划算法（运行时间  $O(nC)$ ）是伪多项式时间算法。注：若算法在  $\log L$  的多项式时间内运行，则是严格多项式时间算法
- **FPAS 构造方法**：对存在伪多项式时间算法  $A$  的 NP 难问题：
  1. 对实例  $I$  的数值进行标度变换和舍入，得到  $I'$
  2. 将  $A$  应用于  $I'$  得到近似解

### 11.1.7 可近似性分类

#### 1. 完全可近似的 (Fully Approximable)：

- 定义：对任意小的  $\epsilon > 0$ ，存在  $(1 + \epsilon)$ -近似算法
- 典型问题：0/1 背包问题

#### 2. 可近似的 (Approximable)：

- 定义：存在具有常数比的近似算法（如 2-近似）
- 典型问题：最小顶点覆盖问题、多机调度问题

#### 3. 不可近似的 (Inapproximable)：

- 定义：除非  $P=NP$ ，否则不存在具有常数比的近似算法
- 典型问题：货郎问题 (TSP)



## 11.2 可近似算法举例

### 11.2.1 装箱问题

会有四种启发式方法, 最先匹配 (FF, 分配第一个能装的箱子), 最佳匹配 (BF, 分配能装的箱子里面剩余空间最小的), 递减最先匹配 (FFD), 递减最佳匹配 (BFD)。下面我们对 FF 进行近似度分析:

假定: 箱子容量为 1 个单位体积, 物品大小  $s_i \leq 1$  ( $i = 1, 2, \dots, n$ )。

定义:

- $FF(I)$  表示在实例  $I$  下, 使用 FF 算法装入物品时所用的箱子数目
- $OPT(I)$  表示最优装入时所用的箱子数目

FF 装箱规则说明:

- 根据 FF 装箱规则, **至多有一个非空箱子**所装的物品体积  $< \frac{1}{2}$
- 若存在两个这样的箱子  $b_i, b_j$  ( $i < j$ ), 由于它们装的物品体积均  $< \frac{1}{2}$ , 根据算法规则,  $b_j$  中的物品应被装入  $b_i$  而非新箱子

**定理 5.** 给定若干个物品, 判断它们是否可由两个箱子装下是 NP-完全的。

**证明.** 可由划分问题 (Partition problem) 规约。划分问题是 Karp 的 21 个 NP-完全问题之一, 该问题表述为: 给定若干个正整数, 问可否将其划分成和相等的两个部分; 换言之, 给定  $c_1, c_2, \dots, c_n \in \mathbb{Z}^+$ , 划分问题问是否存在一个  $S \subseteq \{1, 2, \dots, n\}$  使得

$$\sum_{i \in S} c_i = \sum_{i \notin S} c_i$$

根据一个划分问题实例, 我们可以构造一个装箱问题, 令物品的尺寸为:

$$a_i = \frac{2c_i}{\sum_{j=1}^n c_j}, \quad i = 1, 2, \dots, n$$

同时我们假设这些  $a_i \in (0, 1]$ 。由于  $\sum_{i=1}^n a_i = 2$ , 如果这些物品能用两个箱子装下, 那么这两个箱子都正好装满, 这也同时解答了划分问题。这样我们就把一个已知的 NP-完全问题规约为了两个箱子的装箱问题。而显然一个给定的装箱问题的解可在多项式时间内验证, 因此该问题是 NP-完全的。□

要证明问题  $L$  是 NP-完全的, 只需证明: 存在性: 已知的某个 NP-完全问题 (如划分问题) 可规约到  $L$  无关性: 不要求  $L$  的所有实例都满足某种特性, 只需特定构造的实例能覆盖原问题的解空间

我们之前衡量近似算法采用的是绝对近似比, 定义为最坏情况下算法的求解值与问题的最优值之间的比率。对任意实例  $I$ , 用  $A(I)$  表示针对实例  $I$  运行算法  $A$  后所用箱子数;  $OPT(I)$  表示装完实例  $I$  中所有物品所用的最少箱子数。若存在常数  $\alpha \geq 1$ , 对任意实例  $I$  有:

$$A(I) \leq \alpha \cdot OPT(I)$$

则算法  $A$  的近似比至多是  $\alpha$ 。

**定理 6.** 除非  $P = NP$ , 否则装箱问题不存在多项式时间算法具有小于  $\frac{3}{2}$  的绝对近似比。

**证明 (反证法).** 假设存在近似比小于  $\frac{3}{2}$  的多项式时间近似算法  $A$ , 分类讨论:

- 当  $OPT(I) = 2$  时:

$$A(I) < \frac{3}{2} \times 2 = 3 \implies A(I) = 2 = OPT(I)$$

- 当  $OPT(I) \geq 3$  时:

$$A(I) \geq 3$$

此时算法  $A$  可精确判定“是否能用两个箱子装下”的问题，而该问题是  $NP$ -完全的。若  $P \neq NP$ ，则导致矛盾。  $\square$

虽然  $OPT(I) = 2$  的情况直接揭示了矛盾的核心（算法  $A$  被迫精确求解  $NP$ -完全问题），但必须验证  $OPT(I) \geq 3$  时的行为，才能确保反证假设的全局一致性。这是复杂性理论证明中“规约 + 反证”方法的严谨性体现。

**定义 4** (渐进近似比). 对任意常数  $\alpha \geq 1$ ，任意实例  $I$ ，若存在常数  $k$  满足：

$$A(I) \leq \alpha \cdot OPT(I) + k \quad (AAR) \tag{3}$$

则称所有满足上式的  $\alpha$  的下确界为算法  $A$  的**渐进近似比**。其中：

- $\alpha$  决定当  $OPT(I)$  充分大时  $A(I)$  与  $OPT(I)$  的比值
- $k$  可以是：

– 固定常数，或

–  $o(OPT(I))$  (满足  $\lim_{OPT(I) \rightarrow \infty} \frac{k}{OPT(I)} = 0$ )

- 当  $k = 0$  时，渐进近似比退化为**绝对近似比**

**定理 7.** 对于任意的实例  $I$ ，有：

$$FF(I) \leq \frac{17}{10}OPT(I) + \frac{4}{5}$$

(常数  $ppt$  中为 2,  $BF$  同为  $\frac{17}{10}$ )

后续再做展开：<https://zhuanlan.zhihu.com/p/685478438>

**定理 8.** 对于任意的实例  $I$ ，有：

$$FFD(I) \leq \frac{11}{9}OPT(I) + 4$$

( $BFD$  同为  $\frac{11}{9}$ )

### 11.2.2 ETSP 问题

给定一个平面上  $n$  个点的集合  $S$ ，找出一个在这些点上的最短长度的游程  $t$ 。这里一个游程是恰好访问每个点一次的一条回路。

ETSP 问题是 TSP 问题的特殊情况，点之间的距离是欧式距离，满足三角不等关系。对于最近邻 (NN) 法：采用贪心法，设  $p_1$  是任意一个起始点，下一个访问距离  $p_1$  最近的点，比如  $p_2$ ，再继续访问距离  $p_2$  最近的点，直到回到  $p_1$ 。

**定理 9.** 对于  $ESTP$  问题所有满足三角不等式的  $n$  个城市的实例  $I$ , 有:

$$NN(I) \leq \frac{1}{2}(\lceil \log_2 n \rceil + 1)OPT(I).$$

而且, 对于每一个充分大的  $n$ , 存在满足三角不等式的  $n$  个城市的实例  $I$  使得:

$$NN(I) > \frac{1}{3}(\log_2(n+1) + \frac{4}{3})OPT(I).$$

很明显这里对于  $NN$  的近似比:

$$R_{NN}(I) = \frac{NN(I)}{OPT(I)} = O(\log n)$$

显然  $R_{NN} = \infty$ . 所以  $NN$  算法不是常数近似比的算法, 不能实际应用。

### 11.2.3 最小顶点覆盖问题

开始时初始化顶点集合  $C$  为空集, 然后重复执行以下操作——任意选取图中的一条边  $(u, v)$ , 将这两个顶点  $u$  和  $v$  加入集合  $C$  中, 同时从图中删除这两个顶点及其所有关联边; 持续进行这样的操作直到图中所有的边都被删除为止, 最终得到的集合  $C$  就是所求的顶点覆盖。

算法挑选的边构成图的极大匹配  $M$  (即不存在其他匹配  $M'$  满足  $M \subset M'$ ); 由于覆盖  $M$  中的边至少需要  $|M|$  个顶点, 因此最小顶点覆盖的大小至少为  $|M|$ , 而  $MVC$  算法实际得到的覆盖大小为  $2|M|$ , 由此可推导出近似比  $R_{MVC} \leq 2$ .  $MVC$  是 2-可近似算法。

设  $G = (V, E)$ , 其中顶点集  $V = \{u, v\}$ , 边集  $E = \{(u, v)\}$ 。近似顶点覆盖算法  $APPROX - VERTEX - COVER$  返回的解只能是  $\{u, v\}$ , 而最优解为  $\{u\}$  或  $\{v\}$ , 此时该算法返回顶点覆盖的规模恰为最优覆盖的两倍。

考虑以下启发式算法来解决顶点覆盖问题: 重复选择度数最高的顶点, 然后删除其所有关联边。举一个例子来说明这种启发式算法的近似比不为 2。(提示: 考虑一个二分图, 左边顶点集合中的点的度数相同, 右边顶点集合中的点的度数不相同。)

**解答:**

构造二分图  $G = (V, E)$ , 其中:

$$V = L \cup R$$

$$L = \{l_1, l_2, l_3, l_4, l_5\}$$

$$R = \{r_1, r_2, \dots, r_{11}\}$$

边集  $E$  包含以下边:

$$\begin{aligned} &\{(l_1, r_1), (l_1, r_3), (l_1, r_4), (l_1, r_6), (l_1, r_7), \\ &(l_2, r_1), (l_2, r_2), (l_2, r_4), (l_2, r_6), (l_2, r_8), \\ &(l_3, r_1), (l_3, r_2), (l_3, r_3), (l_3, r_5), (l_3, r_9), \\ &(l_4, r_1), (l_4, r_2), (l_4, r_3), (l_4, r_5), (l_4, r_{10}), \\ &(l_5, r_1), (l_5, r_2), (l_5, r_3), (l_5, r_4), (l_5, r_{11})\} \end{aligned}$$

### 11.2.4 多机调度问题

给定有穷作业集  $A$  和  $m$  台相同的机器, 作业  $a$  的处理时间为正整数  $t(a)$ , 每一项作业可以在任一机器上处理。如何把作业分配给机器才能使完成所有作业的时间最短? 即如何把  $A$  划分成  $m$  个不相交的子集  $A_i$  使得  $\max \{ \sum_{a \in A_i} t(a) \mid i = 1, 2, \dots, m \}$  最小。

贪心法 G-MPS: 按输入的顺序分配作业; 把每一项作业分配给当前负载最小的机器; 如果当前负载最小的机器有 2 台或 2 台以上, 则分配给其中的任意一台 (比如标号最小的一台)。

**定理 10.** 对于多机调度问题的每一个有  $m$  台机器的实例  $I$ , 贪心算法 G-MPS 满足:

$$G-MPS(I) \leq \left(2 - \frac{1}{m}\right) OPT(I)$$

其中  $OPT(I)$  表示最优调度长度。

证明. 首先建立两个基本事实:

1.  $OPT(I) \geq \frac{1}{m} \sum_{a \in A} t(a)$  ( $m$  台机器的最大负载不小于平均负载)
2.  $OPT(I) \geq \max_{a \in A} t(a)$  (最大负载不小于任一作业的处理时间)

设机器  $M_j$  的负载最大, 即  $G-MPS(I) = t(M_j)$ 。令  $b$  是最后被分配给  $M_j$  的作业。根据贪心算法规则, 在分配  $b$  时  $M_j$  的负载最小, 因此有:

$$t(M_j) - t(b) \leq \frac{1}{m} \left( \sum_{a \in A} t(a) - t(b) \right)$$

由此可得:

$$\begin{aligned} G-MPS(I) &= t(M_j) \\ &\leq \frac{1}{m} \left( \sum_{a \in A} t(a) - t(b) \right) + t(b) \\ &= \frac{1}{m} \sum_{a \in A} t(a) + \left(1 - \frac{1}{m}\right) t(b) \\ &\leq OPT(I) + \left(1 - \frac{1}{m}\right) OPT(I) \quad (\text{由事实 2}) \\ &= \left(2 - \frac{1}{m}\right) OPT(I) \end{aligned}$$

□

给定  $m$  台机器和  $m(m-1)+1$  项作业, 其中前  $m(m-1)$  项作业的处理时间都为 1, 最后一项作业的处理时间为  $m$ 。算法方案是将前  $m(m-1)$  项作业平均分配给  $m$  台机器, 每台分配  $m-1$  项, 最后一项任意分配给一台机器, 此时  $G-MPS(I) = 2m-1$ 。最优分配方案则是将前  $m(m-1)$  项作业平均分配给  $m-1$  台机器, 每台分配  $m$  项, 最后一项分配给剩下的机器, 此时  $OPT(I) = m$ 。由此可得  $G-MPS$  是 2-近似算法。

递减贪心法 DG-MPS: 首先按处理时间从大到小重新排列作业 (处理时间长的作业优先处理), 然后运用 G-MPS。

时间复杂度分析: DG-MPS 增加排序时间  $O(n \log n)$ , 仍然是多项式时间。

**定理 11.** 对于多机调度问题的每一个有  $m$  台机器的实例  $I$ ，递减贪心算法  $DG-MPS$  满足：

$$DG-MPS(I) \leq \left( \frac{3}{2} - \frac{1}{2m} \right) OPT(I)$$

其中  $OPT(I)$  表示最优调度长度。

证明. 设作业按处理时间从大到小排列为  $a_1, a_2, \dots, a_n$ ，考虑负载最大的机器  $M_j$  和最后分配给  $M_j$  的作业  $a_i$ ，存在两种情况：

1. **情况 1:**  $M_j$  只有一个作业

- 此时  $i = 1$ ，即  $a_i$  是处理时间最大的作业
- 显然  $DG-MPS(I) = t(a_1) = OPT(I)$

2. **情况 2:**  $M_j$  有至少两个作业

- 则  $i \geq m + 1$ （因为前  $m$  个作业已分配给不同机器）
- 由最优解性质可得：

$$OPT(I) \geq t(a_m) + t(a_{m+1}) \geq 2t(a_{m+1}) \geq 2t(a_i)$$

- 对  $DG-MPS$  的负载进行分析：

$$\begin{aligned} DG-MPS(I) &= t(M_j) \\ &\leq \frac{1}{m} \left( \sum_{k=1}^n t(a_k) - t(a_i) \right) + t(a_i) \\ &= \frac{1}{m} \sum_{k=1}^n t(a_k) + \left( 1 - \frac{1}{m} \right) t(a_i) \\ &\leq OPT(I) + \left( 1 - \frac{1}{m} \right) \cdot \frac{1}{2} OPT(I) \\ &= \left( \frac{3}{2} - \frac{1}{2m} \right) OPT(I) \end{aligned}$$

□

### 11.3 不可近似问题

**定理 12.** 对于不要求满足三角不等式的货郎问题，不存在常数近似比的近似算法，除非  $P = NP$ 。

证明. 假设存在这样的近似算法  $A$ ，其近似比  $r \leq K$  ( $K$  为常数)。我们将通过构造性证明这与  $NP$  完全问题（哈密顿回路问题）的关系。

#### 构造方法

对任意给定的图  $G = \langle V, E \rangle$ ，构造货郎问题实例  $I_G$  如下：

- 城市集合为  $V$ ，其中  $|V| = n$
- 距离函数定义为：

$$d(u, v) = \begin{cases} 1, & \text{若 } (u, v) \in E \\ Kn, & \text{否则} \end{cases}$$

### 关键性质

1. 若  $G$  有哈密顿回路:

$$\text{OPT}(I_G) = n, \quad A(I_G) \leq r\text{OPT}(I_G) \leq Kn$$

2. 若  $G$  无哈密顿回路:

$$\text{OPT}(I_G) > Kn, \quad A(I_G) \geq \text{OPT}(I_G) > Kn$$

基于算法  $A$  可设计如下判定算法:

1. 构造  $I_G$  (耗时  $O(n^2)$ )
2. 对  $I_G$  运行  $A$  (多项式时间)
3. 若  $A(I_G) \leq Kn$  输出"Yes", 否则输出"No"

该算法可在多项式时间内判定哈密顿回路问题 (HC)。由于 HC 是  $NP$  完全的, 故  $P = NP$ , 与普遍接受的复杂性假设矛盾。□

### 11.4 完全可近似问题

例如对于 01 背包问题: 如果用贪心算法 (G-KK), 不断尝试单位价值最大的物品:

**定理 13.** 对于 0-1 背包问题的任何实例  $I$ , 贪心算法  $G$  和最优解  $\text{OPT}(I)$  满足:

$$\text{OPT}(I) < 2G - KK(I)$$

其中  $KK(I)$  表示背包的剩余容量。

**证明.** 设物品按单位重量价值  $\frac{v_i}{w_i}$  从大到小排列为  $u_1, u_2, \dots, u_n$ ,  $u_k$  是贪心算法  $G$  中第一件未装入背包的物品。根据贪心策略的特性:

$$\begin{aligned} \text{OPT}(I) &< G - KK(I) + v_k \quad (\text{最优解最多比贪心解多放入 } u_k \text{ 的价值}) \\ &\leq G - KK(I) + v_{\max} \quad (v_k \leq \max_{1 \leq i \leq n} v_i) \\ &\leq 2G - KK(I) \quad (\text{因 } v_{\max} \leq G) \end{aligned}$$

其中关键不等式成立因为:

- 贪心解  $G$  至少包含一个最大价值物品 (否则可以替换)
- $KK(I)$  表示贪心解装入后的剩余容量

□

**Input:** 参数  $k \geq 1$ , 实例  $I$   
**Output:** 近似解  $A_\varepsilon(I)$

```

1  $\varepsilon \leftarrow \frac{1}{k}$ ;
2 将物品按  $\frac{v_i}{w_i}$  降序排列:  $\frac{v_1}{w_1} \geq \dots \geq \frac{v_n}{w_n}$ ;
3 for  $j \leftarrow 0$  to  $k$  do
4   枚举所有  $j$  件物品的子集  $S \subseteq \{1, \dots, n\}$ ;
5   if  $\sum_{i \in S} w_i \leq C$  then
6     用 G-KK 算法装入剩余物品  $\{1, \dots, n\} \setminus S$ ;
7     记录当前解  $A_j(I)$ ;
8   end
9 end
10 输出  $\max\{A_0(I), \dots, A_k(I)\}$ ;

```

**Algorithm 50:** PTAS 算法

**定理 14.** 对任意  $k \geq 1$ , PTAS 算法满足:

- 时间复杂度:  $O(kn^{k+1})$
- 近似比:  $R_k = \frac{OPT(I)}{A_\varepsilon(I)} < 1 + \frac{1}{k} = 1 + \varepsilon$

**证明. 时间复杂度分析:**

- 子集枚举量:  $\sum_{j=0}^k \binom{n}{j} = O(kn^k)$
- 每次 G-KK 计算:  $O(n)$
- 总时间:  $O(kn^{k+1})$

**近似比证明:** 设最优解  $X$ , 分两种情况:

1. 若  $|X| \leq k$ : 算法必得  $X$  (因枚举所有  $\leq k$  的子集)
2. 若  $|X| > k$ :
  - 取  $Y \subset X$  为前  $k$  个最大价值物品
  - 设  $Z = X \setminus Y = \{u_{k+1}, \dots, u_r\}$  (保持  $\frac{v_i}{w_i}$  降序)
  - 设  $u_m$  为  $Z$  中第一个未被算法装入的物品
  - 定义  $W$  为算法在  $u_m$  前装入的非  $\{u_1, \dots, u_m\}$  物品

关键推导:

$$\begin{aligned}
C' &= C - \sum_{j=1}^k w_j - \sum_{j=k+1}^{m-1} w_j \\
C'' &= C' - \sum_{u_j \in W} w_j < w_m \quad (\text{因 } u_m \text{ 未装入}) \\
OPT(I) &\leq \sum_{j=1}^k v_j + \sum_{j=k+1}^{m-1} v_j + C' \frac{v_m}{w_m} \\
&< \sum_{j=1}^k v_j + \sum_{j=k+1}^{m-1} v_j + \sum_{u_j \in W} v_j + v_m \quad (\text{因 } \frac{v_j}{w_j} \geq \frac{v_m}{w_m}) \\
&< A_\varepsilon(I) + \frac{OPT(I)}{k+1} \quad (\text{因 } (k+1)v_m \leq OPT(I)) \\
&\Rightarrow R_k < 1 + \frac{1}{k} = 1 + \varepsilon
\end{aligned}$$

□

- **PTAS (Polynomial-Time Approximation Scheme):** 对任意小正数  $\varepsilon$ , 在输入规模  $n$  的多项式时间内找到  $(1 + \varepsilon)$ -近似解。运行时间关于  $n$  的多项式, 但可能关于  $1/\varepsilon$  指数增长。
- **FPTAS (Fully Polynomial-Time Approximation Scheme):** 运行时间同时是  $n$  和  $1/\varepsilon$  的多项式, 具有实际可行性。

对于存在伪多项式时间算法的 NP 难问题, 构造 FPTAS 的一般方法:

1. 设原问题实例  $I$  的输入值为  $x_1, \dots, x_k$
2. 伪多项式算法  $A$  运行时间为  $T(n, \sum x_i)$
3. 进行标度变换:  $x'_i = \lfloor \frac{x_i}{f(\varepsilon)} \rfloor$
4. 得到新实例  $I'$
5. 用算法  $A$  求解  $I'$  得解  $S'$
6. 证明  $S'$  是  $I$  的  $(1 + \varepsilon)$ -近似解



```

Input: 整数  $k > 0$ , 实例  $I = (C, \{(s_i, v_i)\}_{i=1}^n)$ 
Output: 近似解  $A_\varepsilon(I)$ 
1  $K \leftarrow \frac{C}{2(k+1)n}$ ; // 尺度因子
2 ;
3  $C' \leftarrow \lfloor C/K \rfloor$ ;
4 for  $i = 1$  to  $n$  do
5    $s'_i \leftarrow \lfloor s_i/K \rfloor$ ;
6    $v'_i \leftarrow \lfloor v_i/K \rfloor$ ;
7 end
8 创建新实例  $I' = (C', \{(s'_i, v'_i)\}_{i=1}^n)$ ;
9  $S \leftarrow \text{KNAPSACK}(I')$ ; // 应用伪多项式算法
10 ;
11 return  $S$ 

```

**Algorithm 51:** FPTAS for 0-1 Knapsack

性质:

- 构成算法簇: 对每个  $k$ ,  $\varepsilon = 1/k$ , 记为  $A_\varepsilon$
- 时间复杂度:  $\Theta(n^2/\varepsilon)$

**定理 15.** 对每个整数  $k > 1$ ,  $\varepsilon = 1/k$  和任意实例  $I$ :

$$\text{OPT}(I) < (1 + \varepsilon)A_\varepsilon(I)$$

时间复杂度为  $\Theta(n^2/\varepsilon)$ 。

证明. 时间复杂度分析:

$$\begin{aligned}
 T(n) &= \Theta(nC'/K) \\
 &= \Theta\left(n \cdot \frac{C}{K} \cdot \frac{1}{K}\right) \\
 &= \Theta\left(n \cdot \frac{2(k+1)n}{K}\right) \\
 &= \Theta(kn^2) = \Theta(n^2/\varepsilon)
 \end{aligned}$$

**近似比证明:** 设  $I'$  为标度变换后的实例:

- $\text{OPT}(I') \geq \sum_{i \in S^*} \lfloor v_i/K \rfloor$  ( $S^*$  是最优解)
- 有不等式链:

$$\text{OPT}(I) - K \cdot \text{OPT}(I') \leq Kn \quad (\text{最优解至多 } n \text{ 项})$$

$$\text{OPT}(I) - A_\varepsilon(I) \leq Kn \quad (\text{近似解定义})$$

$$\begin{aligned}
 A_\varepsilon(I) &\geq \text{OPT}(I) - Kn \\
 &= \text{OPT}(I) - \frac{C}{2(k+1)}
 \end{aligned}$$

- 假设  $\text{OPT}(I) \geq C/2$  (否则易得最优解), 则:

$$\begin{aligned}\frac{A_\varepsilon(I)}{\text{OPT}(I)} &\geq 1 - \frac{1}{2(k+1)\text{OPT}(I)/C} \\ &> 1 - \frac{1}{k+1} = \frac{k}{k+1} \\ \Rightarrow R_{A_\varepsilon} &< \frac{k+1}{k} = 1 + \frac{1}{k} = 1 + \varepsilon\end{aligned}$$

□

## 11.5 课后作业

### 11.5.1 独立集问题

给出一个独立集问题的近似算法, 即找到最大的顶点子集, 子集中顶点互不相连。并请证明或否定该近似算法的近似度是有界的。

使用贪心算法:

1. 初始化独立集  $S = \emptyset$ 。
2. 从图中选择一个度数最小的顶点  $v$ , 将其加入  $S$ 。
3. 将  $v$  及其所有邻居从图中删除。
4. 重复步骤 2-3, 直到图中没有剩余顶点。

每次选择一个顶点加入  $S$  时, 最多会”排除”  $\Delta + 1$  个顶点 (包括该顶点及其邻居)。因此, 贪心算法得到的独立集大小至少为  $\frac{\text{OPT}}{\Delta+1}$ , 其中  $\text{OPT}$  是最优解的大小。

### 11.5.2 最大子集和问题

最大子集和问题: 给定含有  $n$  个正整数的集合  $S = \{s_1, s_2, \dots, s_n\}$  和一个正整数  $C$ , 要求找出  $S$  的一个子集, 使得它们的和最大且不超过  $C$ 。请给出求解该问题的完全多项式近似方案。要求分析算法时间复杂度和近似比。

运用动态规划思想:

1. 设定近似参数  $\varepsilon \in (0, 1)$ , 计算缩放因子  $k = \frac{\varepsilon \cdot \max(S)}{n}$ 。
2. 对集合  $S$  中每个元素  $s_i$ , 计算缩放后的值  $s'_i = \lfloor \frac{s_i}{k} \rfloor$ 。
3. 使用动态规划求解缩放后的问题:
  - 定义  $dp[i][j]$  表示前  $i$  个元素能否组成和  $j$
  - 初始化  $dp[0][0] = \text{True}$
  - 状态转移:  $dp[i][j] = dp[i-1][j] \vee dp[i-1][j - s'_i]$
4. 在满足  $\sum_{i \in T} s'_i \leq \lfloor \frac{C}{k} \rfloor \triangleq J_{\max}$  的所有子集  $T$  中, 利用  $dp[n][..]$  来找到使  $\sum_{i \in T} s_i$  最大的解。

而对于该算法的时间复杂度与近似比:

- 参数缩放阶段:

- 计算最大值  $\max(S)$ :  $O(n)$
- 计算缩放因子  $k = \frac{\varepsilon \cdot \max(S)}{n}$ :  $O(1)$
- 缩放所有元素  $s'_i = \lfloor \frac{s_i}{k} \rfloor$ :  $O(n)$

- 动态规划阶段:

- 最大缩放和计算:

$$J_{\max} = \left\lfloor \frac{C}{k} \right\rfloor = O\left(\frac{nC}{\varepsilon \max(S)}\right)$$

- 状态转移次数:

$$O(n \cdot J_{\max}) = O\left(n \cdot \frac{nC}{\varepsilon \max(S)}\right)$$

- 总时间复杂度:

$$T(n, \varepsilon) = O\left(\frac{n^2 C}{\varepsilon \max(S)}\right)$$

注意到  $\max(S) \leq C$ , 因此:

$$T(n, \varepsilon) \leq O\left(\frac{n^2}{\varepsilon}\right)$$

显然, 这是一个完全多项式近似算法。

- 缩放误差分析:

$$\begin{aligned} \forall s_i : k \cdot s'_i &\leq s_i \leq k(s'_i + 1) \\ \sum_{i \in T} s_i - k \sum_{i \in T} s'_i &\leq nk = \varepsilon \max(S) \end{aligned}$$

- 近似比推导:

$$ALG \geq k \cdot OPT' \geq OPT - nk = OPT - \varepsilon \max(S)$$

$$\begin{aligned} R_{ALG} &= \max\left(\frac{ALG}{OPT}, \frac{OPT}{ALG}\right) \\ &= \frac{OPT}{ALG} \\ &\leq \frac{OPT}{OPT - \varepsilon \max(S)} \\ &= \frac{1}{1 - \frac{\varepsilon \max(S)}{OPT}} \end{aligned}$$

若  $\max(S) \leq OPT$ , 则易知  $R_{ALG} \leq 1 + \varepsilon$ , 若  $\max(S) > OPT$ , 则对该算法进行修正, 将所有满足  $s_i > C$  的  $s_i$  均删除, 再次进行计算, 即可转化为上一种情况。

其中  $ALG$  为该算法得到的解,  $OPT$  为最优解,  $OPT'$  为缩放后问题的最优解

### 11.5.3 无向图最大割集问题

设无向图  $G = (V, E)$ ,  $V_1 \cup V_2 = V, V_1 \cap V_2 = \emptyset$ . 若边  $(u, v) \in E$  且  $u \in V_1$ , 则称  $(u, v)$  为割边; 否则称为非割边. 求最大割集问题: 任给无向图  $G = (V, E)$ , 求  $G$  的边数最多的割集.

求最大割集的局部改进算法 MCUT 思想如下:

初始令  $V_1 = V, V_2 = \emptyset$ . 重复以下操作:

1. 若存在顶点  $u$ , 在  $u$  关联的边中非割边数  $>$  割边数

- 若  $u \in V_1$ , 则将  $u$  移至  $V_2$
- 若  $u \in V_2$ , 则将  $u$  移至  $V_1$

2. 直到不存在这样的顶点时停止

最终得到的  $(V_1, V_2)$  即为解. 对任意实例满足:

$$OPT(I) \leq 2 \cdot MCUT(I)$$

其中  $OPT(I)$  表示最优解的边数.

设  $C$  为算法得到的割边数  $MCUT(I)$ , 设顶点  $u$  的割边与非割边数分别是  $c(u)$  和  $nc(u)$ , 那么:

每条割边连接  $V_1$  和  $V_2$ , 因此  $\sum_{u \in V} c(u) = 2C$ . 因为  $c(u) \geq nc(u)$ , 所以对于每个顶点  $u$ , 其度数  $d(u) = c(u) + nc(u) \leq 2 \times c(u)$ , 那么  $c(u) \geq \frac{d(u)}{2}$ , 所以  $2C = \sum_{u \in V} c(u) \geq \frac{1}{2} \sum_{u \in V} d(u) = |E|$ , 所以  $OPT(I) \leq |E| \leq 2C = 2 \cdot MCUT(I)$

### 11.5.4 平面 TSP 问题的最小权匹配

对 TSP 问题所有满足三角不等关系的实例  $I$ , 已知有最小权匹配 (MM) 近似算法求解, 且  $MM(I) < \frac{3}{2}OPT(I)$ , 请给出具有该近似比的紧实例.

这个题肯定需要让顶点数  $n$  趋于无穷, 使得  $length(T) \rightarrow OPT(I), length(M) \rightarrow \frac{1}{2}OPT$  即可, 这里我们必须约定我们找到的最小生成树满足某些特殊的性质, 使得  $n$  个点中度数为奇数的点的数量是接近  $n$  的, 这样就可以使得  $length(M) \rightarrow \frac{1}{2}OPT$ . 而且依靠这个最小生成树得到的欧拉图, 必须是能够变成哈密尔顿回路的, 即不存在割点.

按照以上原则进行构造即可.

- **PTAS (Polynomial-Time Approximation Scheme):** 对任意小正数  $\epsilon$ , 在输入规模  $n$  的多项式时间内找到  $(1 + \epsilon)$ -近似解. 运行时间关于  $n$  的多项式, 但可能关于  $1/\epsilon$  指数增长.
- **FPTAS (Fully Polynomial-Time Approximation Scheme):** 运行时间同时是  $n$  和  $1/\epsilon$  的多项式, 具有实际可行性.

对于存在伪多项式时间算法的 NP 难问题, 构造 FPTAS 的一般方法:

1. 设原问题实例  $I$  的输入值为  $x_1, \dots, x_k$
2. 伪多项式算法  $A$  运行时间为  $T(n, \sum x_i)$
3. 进行标度变换:  $x'_i = \lfloor \frac{x_i}{f(\epsilon)} \rfloor$
4. 得到新实例  $I'$



- 假设  $\text{OPT}(I) \geq C/2$  (否则易得最优解), 则:

$$\begin{aligned}\frac{A_\epsilon(I)}{\text{OPT}(I)} &\geq 1 - \frac{1}{2(k+1)\text{OPT}(I)/C} \\ &> 1 - \frac{1}{k+1} = \frac{k}{k+1} \\ \Rightarrow R_{A_\epsilon} &< \frac{k+1}{k} = 1 + \frac{1}{k} = 1 + \epsilon\end{aligned}$$

□

### 11.5.5 最大团问题

给定近似参数  $\epsilon$ , 假设存在一个近似比为常数  $\rho$  的算法  $A$  用于求解最大团问题。对于图  $G = (V, E)$ , 设其最大团大小为  $m$ , 我们通过以下步骤构造新的近似算法:

**Input:** 图  $G = (V, E)$ , 近似参数  $\epsilon$   
**Output:** 大小为  $n^{1/k}$  的团  $C$

- 1 计算  $k = \lceil 1/\log_c(1 + \epsilon) \rceil$ ; //  $c$  为算法  $A$  的近似比
- 2 构造图  $G^{(k)}$ ; // 通过  $G$  的  $k$ -幂运算
- 3 使用算法  $A$  在  $G^{(k)}$  中找到大小为  $n$  的团
- 4 将  $G^{(k)}$  的团映射回  $G$ , 得到团  $C$
- 5 **return**  $C$

**Algorithm 53:** 完全多项式时间近似模式  $A'$

$$\begin{aligned}\frac{m}{n^{1/k}} &\leq c^{1/k} \\ \text{令 } k &= \frac{1}{\log_c(1 + \epsilon)} \\ \Rightarrow \frac{m}{n^{1/k}} &\leq 1 + \epsilon \\ \Rightarrow n^{1/k} &\geq \frac{m}{1 + \epsilon}\end{aligned}$$

$$\begin{aligned}k &= \frac{1}{\log_c(1 + \epsilon)} = \frac{\ln c}{\ln(1 + \epsilon)} \\ &\geq \frac{\ln c}{\epsilon} \quad (\text{因为 } \ln(1 + \epsilon) \leq \epsilon)\end{aligned}$$

因此算法  $A'$  满足:

- 近似比:  $1 + \epsilon$
- 时间复杂度: 关于  $|V|$  和  $1/\epsilon$  的多项式

上述构造证明了最大团问题存在完全多项式时间近似模式 (FPTAS), 其中:

$$\text{输出团大小} \geq \frac{m}{1 + \epsilon}$$

且运行时间关于输入规模和  $1/\epsilon$  均为多项式级别。

### 11.5.6 证明 TSP 是不可近似问题

反证法. 假设存在常数  $\rho \geq 1$ , 使得一般旅行商问题 (TSP) 存在多项式时间  $\rho$ -近似算法  $A$ 。我们将通过以下步骤导出矛盾:

**步骤 1: 问题转化** 给定哈密顿环问题实例  $G = (V, E)$ , 构造对应的 TSP 实例:

- 完全图  $G' = (V, E')$ , 其中  $E' = \{(u, v) : u, v \in V \text{ 且 } u \neq v\}$
- 代价函数定义为:

$$c(u, v) = \begin{cases} 1 & \text{若 } (u, v) \in E \\ \rho|V| + 1 & \text{否则} \end{cases}$$

**步骤 2: 代价分析**

- **情况 1:** 若  $G$  有哈密顿环  $H$ , 则对应 TSP 回路代价:

$$C_{\text{opt}} = \sum_{(u,v) \in H} c(u, v) = |V|$$

- **情况 2:** 若  $G$  无哈密顿环, 则任何 TSP 回路至少包含一条非  $E$  中的边, 其最小代价为:

$$C_{\min} \geq (\rho|V| + 1) + (|V| - 1) > \rho|V|$$

**步骤 3: 算法行为** 对构造的 TSP 实例  $\langle G', c \rangle$  运行算法  $A$ :

- 若  $G$  有哈密顿环,  $A$  输出回路代价  $C_A \leq \rho C_{\text{opt}} = \rho|V|$
- 若  $G$  无哈密顿环,  $A$  输出回路代价  $C_A > \rho|V|$

**步骤 4: 矛盾导出** 通过比较  $C_A$  与  $\rho|V|$  可判定  $G$  是否存在哈密顿环。这意味着:

$$\text{哈密顿环问题} \in P$$

根据定理 34.13 (哈密顿环的 NP 完全性) 和定理 34.4 ( $P=NP$  的充分条件), 将导致  $P = NP$ , 与前提  $P \neq NP$  矛盾。

□

**Input:** 哈密顿环问题实例  $G = (V, E)$

**Output:** 判定  $G$  是否存在哈密顿环

1 构造完全图  $G' = (V, E')$ , 其中  $E' = \{(u, v) : u, v \in V \text{ 且 } u \neq v\}$ ;

2 定义代价函数:

$$c(u, v) = \begin{cases} 1 & \text{若 } (u, v) \in E \\ \rho|V| + 1 & \text{否则} \end{cases}$$

3 运行  $\rho$ -近似算法  $A$  于  $\langle G', c \rangle$ ;

4 **if**  $A$  输出回路代价  $\leq \rho|V|$  **then**

5     **return** 存在哈密顿环;

6 **else**

7     **return** 不存在哈密顿环;

8 **end**

**Algorithm 54:** TSP 不可近似性的归约过程

### 理论意义

该证明揭示了:

- TSP 的不可近似性直接源于其与 NP 完全问题的等价关系
- 近似算法存在性将导致计算复杂性层级坍塌 ( $P = NP$ )
- 对任意  $\rho \geq 1$  的推广通过代价函数的伸缩性实现

### 11.5.7 瓶颈旅行商问题

瓶颈旅行商问题 (bottleneck traveling-salesman problem) 要求找到一条回路, 满足该回路中代价最大的边的代价是所有回路中代价最大的边的代价中最小的。假设代价函数满足三角形不等式, 证明: 该问题有一个近似比为 3 的多项式时间近似算法。

给定无向完全图  $G = (V, E)$ , 根据思考题 21-4 的结论, 可以在关于  $|V|$  和  $|E|$  的线性时间内求出一棵瓶颈生成树  $T$ 。因为  $G$  是完全图, 根据练习题 34.2-11 的结论, 一定能由  $T$  在关于  $|V|$  和  $|E|$  的多项式时间内构造出一个哈密顿环  $H$ , 且  $H$  上任意两个连续顶点  $u$  和  $v$  间在  $T$  上的距离不超过 3 条边。设  $B_T$  表示瓶颈生成树中代价最高的边。

近似比证明。根据三角形不等式 (如图 35.2-4 所示):

$$\begin{aligned} c(v_1^{(1)}, v_2^{(2)}) &\leq c(v_1^{(1)}, v_2^{(1)}) + c(v_2^{(1)}, v_2^{(2)}) \\ &\leq c(v_1^{(1)}, u) + c(u, v_2^{(1)}) + c(v_2^{(1)}, v_2^{(2)}) \\ &\leq 3c(B_T) \end{aligned}$$

因此对任意边  $(u, v) \in H$ , 有  $c(u, v) \leq 3c(B_T)$ 。

设  $H$  中代价最大的边为  $B_H$ , 则:

$$c(B_H) \leq 3c(B_T) \quad (4)$$



设最优瓶颈回路  $H^*$  中代价最大的边为  $B_H^*$ ，将其移除后得到生成树。根据瓶颈生成树的定义：

$$c(B_T) \leq c(B_H^*) \quad (5)$$

联立 (1)(2) 式可得：

$$c(B_H) \leq 3c(B_H^*)$$

□

### 11.5.8 最优回路不会自我交叉

假设旅行商问题的一个实例的所有顶点是一组平面上的点，并且代价  $c(u, v)$  是点  $u$  和  $v$  之间的欧氏距离。证明：一条最优回路不会自我交叉。

采用反证法，假设存在一条最优回路存在一个自我交叉，如图 35.2-5 所示，设  $(v_1, v_2)$  和  $(v_3, v_4)$  交叉位于  $x$ ，有

$$c(v_1, v_2) + c(v_3, v_4) = c(v_1, x) + c(v_2, x) + c(v_3, x) + c(v_4, x)$$

因为所有顶点都位于欧式平面上且代价  $c(u, v)$  是点  $u$  和  $v$  之间的欧氏距离。所以  $c$  满足三角形不等式，有  $c(v_1, x) + c(v_3, x) > c(v_1, v_3)$  且  $c(v_2, x) + c(v_4, x) > c(v_2, v_4)$ 。代入前面的等式，有  $c(v_1, v_2) + c(v_3, v_4) > c(v_1, v_3) + c(v_2, v_4)$ ，用边  $(v_1, v_3)$  和  $(v_2, v_4)$  替换原回路中的  $(v_1, v_2)$  和  $(v_3, v_4)$  得到一个代价更小的回路，与原回路是最优回路相矛盾。因此，一条最优回路不会自我交叉。证明完毕。

### 11.5.9 TSP 不可近似性

证明对于任意常数  $c \geq 0$ ，一般旅行商问题不存在具有近似比为  $|V|^c$  的多项式时间的近似算法。

证明：采用反证法。假设对于某个常数  $c \geq 0$ ，一般旅行商问题存在近似比为  $|V|^c$  的多项式时间算法  $A$ 。我们将说明如何使用  $A$  来在多项式时间内求解哈密顿环问题的一个实例。根据定理 34.13，哈密顿环问题是 NP 完全的，根据定理 34.4，若哈密顿环问题存在一个多项式时间算法，则  $P = NP$ 。

设  $G = (V, E)$  是哈密顿环问题的一个实例，使用假想的近似算法  $A$  来检测  $G$  是否包含一个哈密顿环。按照如下方法将  $G$  转化为旅行商问题的一个实例，设  $G' = (V, E')$  为关于  $V$  的完全图，即  $E' = \{(u, v) : u, v \in V \text{ and } u \neq v\}$ 。其代价函数为：

$$c(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E, \\ |V|^{c+1} + 1 & \text{otherwise.} \end{cases}$$

可以在关于  $|V|$  和  $|E|$  的多项式时间内构造出  $G'$  和  $c$ 。

现在考虑旅行商问题的一个实例  $\langle G', c \rangle$ 。若  $G$  中存在一个哈密顿环  $H$ ，则  $c$  赋予  $H$  中每条边的代价为 1， $H$  的代价为  $|V|$ ， $\langle G', c \rangle$  中有一条回路代价为  $|V|$ 。若  $G$  中不存在一个哈密顿环，则  $G'$  中的回路必须包含不属于  $E$  的边，该回路的代价至少为  $(|V|^{c+1} + 1) + (|V| - 1) = |V|^{c+1} + |V| = |V|^c(|V| + 1) > |V|^c \cdot |V|$ 。即该回路不是  $G$  中的一个哈密顿环的因子超过  $|V|^c$ 。

假设我们用近似算法  $A$  来求解旅行商问题  $\langle G', c \rangle$ 。因为  $A$  保证能够返回的回路代价不超过最优回路代价的  $|V|^c$  倍。若  $G$  中包含一个哈密顿环，则  $A$  返回一个满足上述要求的回路。若  $G$  中不包含一个哈密顿环，则  $A$  返回一个代价大于  $|V|^{c+1}$  的回路。因此，可以使用  $A$  在多项式时间内求解哈密顿环问题。

**Input:** 效益函数  $f[1..n][0..m]$ , 总资金  $m$ , 项目数  $n$   
**Output:** 最大效益值  $F[n][m]$ , 最优分配方案  $alloc[1..n]$

```

1 // 初始化阶段: 处理单个项目情况
2 for  $x \leftarrow 0$  to  $m$  do
3    $F[1][x] \leftarrow f[1][x]$ ;
4    $tempAlloc[1][x] \leftarrow x$ ;
5   // 记录初始分配方案
6 end
7 // 动态规划递推阶段
8 for  $k \leftarrow 2$  to  $n$  do
9   for  $x \leftarrow 0$  to  $m$  do
10     $F[k][x] \leftarrow 0$ ;
11    for  $x_k \leftarrow 0$  to  $x$  do
12       $current \leftarrow f[k][x_k] + F[k-1][x-x_k]$ ;
13      if  $current > F[k][x]$  then
14         $F[k][x] \leftarrow current$ ;
15         $tempAlloc[k][x] \leftarrow x_k$ ;
16        // 保存当前最优分配
17      end
18    end
19  end
20 end
21 // 回溯最优分配方案
22  $remaining \leftarrow m$ ;
23 for  $k \leftarrow n$  downto 1 do
24    $alloc[k] \leftarrow tempAlloc[k][remaining]$ ;
25    $remaining \leftarrow remaining - alloc[k]$ ;
26 end
27 return  $(F[n][m], alloc)$ ;

```

**Algorithm 21:** Investment-Allocation-DP

**Input:** 带权图  $G = (V, E)$ , 权重函数  $w : E \rightarrow \mathbb{R}^+$ , 源顶点  $s \in V$

**Output:** 源点  $s$  到所有顶点的最短距离  $dist[\cdot]$ , 前驱节点数组  $prev[\cdot]$

```

1 foreach 顶点  $v \in V$  do
2    $dist[v] \leftarrow \infty$ ;
3    $prev[v] \leftarrow \text{NIL}$ ;
4 end
5  $dist[s] \leftarrow 0$ ;
6  $Q \leftarrow \text{MINPRIORITYQUEUE}(V)$ ;           // 基于  $dist$  值的最小优先队列
7 while  $Q \neq \emptyset$  do
8    $u \leftarrow Q.\text{EXTRACTMIN}()$ ;
9   foreach 邻居顶点  $v \in \text{ADJ}[u]$  do
10    // 松弛操作 (Relaxation)
11     $alt \leftarrow dist[u] + w(u, v)$ ;
12    if  $alt < dist[v]$  then
13       $dist[v] \leftarrow alt$ ;
14       $prev[v] \leftarrow u$ ;
15       $Q.\text{DECREASEKEY}(v, alt)$ ;           // 更新队列中  $v$  的优先级
16    end
17  end
18 end
19 return  $(dist, prev)$ ;

```

**Algorithm 24:** Dijkstra 算法

**Input:** $s[1..n]$ : 活动开始时间数组 $f[1..n]$ : 活动结束时间数组 (已按  $f_i$  升序排列) $n$ : 总活动数**Output:** 最大兼容活动集大小

```

1 function DP-ACTIVITY-SELECTOR( $s, f, n$ )
2   let  $c[0..n]$  be new array
3    $c[0] \leftarrow 0$ 
4   for  $i \leftarrow 1$  to  $n$  do
5      $l \leftarrow 1, r \leftarrow n - 1$ 
6     while  $l \leq r$  do
7        $mid \leftarrow \lfloor (l + r) / 2 \rfloor$ 
8        $f[mid] \leq s[i]$ 
9        $\Rightarrow$ 
10       $l \leftarrow mid + 1$ 
11       $f[mid] > s[i]$ 
12       $\Rightarrow$ 
13       $r \leftarrow mid - 1$ 
14    end
15     $k \leftarrow r$ 
16     $c[i] \leftarrow \max(c[k] + 1, c[i - 1])$ 
17 end
18 return  $c[n]$ 

```

**Algorithm 26:** 动态规划活动选择算法

**Input:** 一组活动  $A = \{a_1, a_2, \dots, a_n\}$ , 其中每个活动  $a_i$  有开始时间  $s_i$  和结束时间  $f_i$

**Output:** 分配讲堂的列表, 每个讲堂包含一组不重叠的活动

```
1 按照开始时间  $s_i$  对活动进行升序排序;
2 初始化一个最小堆  $H$ , 按结束时间排序;
3 初始化空的讲堂列表  $classrooms$ ;
4 foreach 活动  $a_i \in A$  do
5   if  $H$  不为空 且  $H$  的堆顶活动  $a_{top}$  满足  $f_{top} \leq s_i$  then
6     从堆  $H$  中弹出  $a_{top}$ ;
7     将  $a_i$  分配到  $a_{top}$  所在的讲堂  $c$ ;
8   end
9   else
10    创建新讲堂  $c_{new}$ ;
11    将  $a_i$  分配到  $c_{new}$ ;
12    将  $c_{new}$  加入  $classrooms$ ;
13  end
14  将  $a_i$  的结束时间  $f_i$  插入堆  $H$ ;
15 end
16 return  $classrooms$ ;
```

**Algorithm 27:** 最小讲堂分配算法 (区间图着色)

**Input:**  $n$ : number of people,  $t[1..n]$ : array of crossing times

**Output:** Minimum total time required

```

1 Sort  $t[1..n]$  in ascending order;
2 Initialize  $ans \leftarrow 0$ ,  $lef \leftarrow n$ ;
3 while  $lef > 0$  do
4   if  $lef == 1$  then
5      $ans \leftarrow ans + t[1]$ ;
6     Break;
7   else
8     if  $lef == 2$  then
9        $ans \leftarrow ans + t[2]$ ;
10      Break;
11    else
12      if  $lef == 3$  then
13         $ans \leftarrow ans + t[1] + t[2] + t[3]$ ;
14        Break;
15      else
16         $s1 \leftarrow t[1] \times 2 + t[lef] + t[lef - 1]$ ;
17         $s2 \leftarrow t[1] + t[2] \times 2 + t[lef]$ ;
18         $ans \leftarrow ans + \min(s1, s2)$ ;
19         $lef \leftarrow lef - 2$ ;
20      end
21    end
22  end
23 end
24 return  $ans$ ;

```

**Algorithm 31:** Minimum Time River Crossing

**Input:** 有向图  $G = (V, E)$   
**Output:** 强连通分量集合  $SCCs$

```

1 Algorithm TARJAN( $G$ )
2  $index \leftarrow 0$ 
3  $S \leftarrow \emptyset$ 
4 foreach  $v \in V$  do
5    $index[v] \leftarrow$  未定义
6    $low[v] \leftarrow$  未定义
7    $onStack[v] \leftarrow$  false
8 end
9  $SCCs \leftarrow \emptyset$ 
10 foreach  $v \in V$  do
11   if  $index[v] =$  未定义 then
12     STRONGCONNECT( $v$ )
13   end
14 end
15 return ( $SCCs$ )

16 Function STRONGCONNECT( $v$ )
17  $index[v] \leftarrow index$ 
18  $low[v] \leftarrow index$ 
19  $index \leftarrow index + 1$ 
20  $S.push(v)$ 
21  $onStack[v] \leftarrow$  true
22 foreach  $(v, w) \in E$  do
23   if  $index[w] =$  未定义 then
24     STRONGCONNECT( $w$ )
25      $low[v] \leftarrow \min(low[v], low[w])$ 
26   end
27   else if  $onStack[w]$  then
28      $low[v] \leftarrow \min(low[v], index[w])$ 
29   end
30 end
31 if  $low[v] = index[v]$  then
32    $C \leftarrow \emptyset$ 
33   repeat
34      $w \leftarrow S.pop()$ 
35      $onStack[w] \leftarrow$  false
36      $C \leftarrow C \cup \{w\}$ 
37   until  $w = v$ 
38    $SCCs \leftarrow SCCs \cup \{C\}$ 
39 end

```

Algorithm 32: Tarjan 算法求解强连通分量

表 4: 经典 Cost 函数对比: 实际计算与理论性质

| 方法     | Cost 函数形式 $C_{method}$                                                                           | 紧致性 $\rho$ |
|--------|--------------------------------------------------------------------------------------------------|------------|
| MST 下界 | $C = L_{current} + W_{MST}$                                                                      | 中等         |
| 线性松弛   | $C = V_{selected} + \sum \left( v_i \cdot \min \left( 1, \frac{W_{remain}}{w_i} \right) \right)$ | 高          |
| 贪心启发式  | $C = C_{current} + \hat{C}_{remain}$                                                             | 低-中        |
| 曼哈顿距离  | $C = g(n) + h(n)$                                                                                | 低          |

**Input** : An array  $A[1 \dots n]$  of  $n$  elements

**Output:** The elements in  $A$  sorted in nondecreasing order

```

1 quicksort (A,low,high)
2 if low < high then
3   |   SPLIT(A [low...high ], w)
4   |   // w is the new position of A [low ]
5   |   quicksort (A,low, w-1)
6   |   quicksort (A, w +1, high)
7 end

```

Algorithm 39: Quicksort

**Input** : An array  $A$  of  $n$  elements

**Output:** The elements in  $A$  sorted in nondecreasing order

```

1 rquicksort (A,low,high)
2 if low < high then
3   |   v ← random (low, high) interchange A [low ] and A [v ]
4   |   SPLIT(A [low...high ], w, v)
5   |   rquicksort (A,low, w-1)
6   |   rquicksort (A,w +1, high)
7 end

```

Algorithm 40: Randomized Quicksort



**Input** : An array  $A$  of  $n$  elements, a number  $k$  such that  $1 \leq k \leq n$

**Output:** The  $k$ th smallest element in  $A$

```

1 select(A, 1, n, k)
2 p ← high- low + 1
3 if p < 44 then
4   | sort A and return A [k]
5 end
6 let q = ⌊p/5⌋. Divide A into q groups of 5 elements each. If 5 doesn't divide p, then discard the
   remaining elements.
7 // Although some elements are discarded here, the subsequent grouping still
   takes them into account, so it will not affect the answer.
8 Sort each of the q groups individually and extract its median element. Let the set of medians be
   M
9 mm ← select (M, 1, q, ⌊q/2⌋) // median of medians
10 Partition A into three arrays:
11  $A_1 \leftarrow \{A | A < mm\}$ ,  $A_2 \leftarrow \{A | A = mm\}$ ,  $A_3 \leftarrow \{A | A > mm\}$ 
12 if  $|A_1| \geq k$  then
13   | return select ( $A_1$ , 1,  $|A_1|$ , k)
14 else
15   | if  $k \leq |A_1| + |A_2|$  then
16     | return mm
17   else
18     | return select ( $A_3$ , 1,  $|A_3|$ ,  $k - |A_1| - |A_2|$ )
19   end
20 end

```

**Algorithm 41:** SELECT algorithm

**Input:** An array  $A$  of  $n$  elements, a number  $k$  such that  $1 \leq k \leq n$

**Output:** The  $k$ th smallest element in  $A$

```

1 rselect( $A, 1, n, k$ )
2  $v \leftarrow \text{random}(\text{low}, \text{high})$ 
3  $x \leftarrow A[v]$ 
4 Partition  $A[\text{low}...\text{high}]$  into three arrays:  $A_1 \leftarrow \{A|A < x\}$ ,  $A_2 \leftarrow \{A|A = x\}$ ,  $A_3 \leftarrow \{A|A > x\}$ 
5 if  $|A_1| \geq k$  then
6   return rselect( $A_1, 1, |A_1|, k$ )
7 else
8   if  $k \leq |A_1| + |A_2|$  then
9     return  $x$ 
10  else
11    return rselect( $A_3, 1, |A_3|, k - |A_1| - |A_2|$ )
12  end
13 end

```

**Algorithm 42:** Randomized SELECT

**Input:**  $n$  元序列  $A$ , 具有  $r$  个排位的有序序列  $K$

**Output:**  $A$  中第  $k_i$  小的元素

```

1 multiselect( $A, K$ )
2  $r \leftarrow |K|$ 
3 if  $r > 0$  then
4    $k = k_{\frac{r}{2}}$ 
5    $a \leftarrow \text{SELECT}(A, k)$ 
6    $\text{Ans}[\frac{r}{2}] \leftarrow a$ 
7    $A_1 = [a_i|a_i < a], A_2 = [a_i|a_i > a]$ 
8    $K_1 \leftarrow [k_1, k_2, \dots, k_{\frac{r}{2}}], K_2 \leftarrow [k_{\frac{r}{2}+1} - k, \dots, k_r - k]$ 
9   // 特别注意这里需要减去  $k$ 
10  multiselect( $A_1, K_1$ )
11  multiselect( $A_2, K_2$ )
12  return  $\text{Ans}$ 
13 end

```

**Algorithm 43:** Multiselect

**Input:** Positive integer  $n$ , positive integer  $a$   
**Output:** Positive integer  $a^n$

```

1 Let the binary digits of  $n$  be  $b_k, b_{k-1}, \dots, b_0$ 
2  $c \leftarrow 1$ 
3 for  $j \leftarrow k$  downto  $0$  do
4    $c \leftarrow c \cdot 2$ 
5   if  $b_j$  is odd then
6      $c \leftarrow c \cdot a$ 
7   end
8 end
9 return  $c$ 

```

Algorithm 44: EXP

**Input:**  $n$   
**Output:**  $n$  是合数或者素数

```

1  $q \leftarrow 0, m \leftarrow n - 1$ 
2 repeat
3    $m \leftarrow \frac{m}{2}$ 
4    $q \leftarrow q + 1$ 
5 until  $m$  是奇数;
6  $a \leftarrow \text{Random}(2, n - 1)$ 
7  $x_0 \leftarrow \text{EXPMOD}(a, m, n)$  //  $x_0 = a^m \pmod n, O(\log^3 n)$ 
8 for  $i \leftarrow 1$  to  $q$  do
9    $x_i \leftarrow x_{i-1}^2 \pmod n$  //  $O(\log^2 n)$ 
10  if  $x_i = 1$  and  $x_{i-1} \neq n - 1$  and  $x_{i-1} \neq 1$  then
11    return 合数 // 100% correct
12  end
13 end
14 if  $x_q \neq 1$  then
15   return 合数 // 100% correct
16 end
17 return 素数 // not 100% correct

```

Algorithm 45: 单次的 Miller-Rabin 素数判定算法

**Input:** 长度为  $n$  的文本串  $X$ , 长度为  $m$  的模式串  $Y$

**Output:** 模式串  $Y$  在文本串  $X$  中的第一个匹配位置, 未匹配返回 0

```

1 在小于  $M$  的素数中随机选择素数  $p$ 
2  $j \leftarrow 1$ 
3 计算  $I_p(Y), I_p(X_j)$ 
4 while  $j \leq n - m + 1$  do
5   if  $I_p(Y) = I_p(X_j)$  then
6     return  $j$ 
7   end
8    $I_p(X_{j+1}) \leftarrow f(I_p(X_j), p)$ 
9    $j \leftarrow j + 1$ 
10 end
11 return 0

```

**Algorithm 46:** Rabin-Karp 算法

表 5: 近似算法对比

| 类型     | 时间复杂度                   | 解质量                  | 典型问题     |
|--------|-------------------------|----------------------|----------|
| PTAS   | $O(n^{f(1/\epsilon)})$  | $(1 + \epsilon)$ -近似 | 欧几里得 TSP |
| FPTAS  | $O((1/\epsilon)^k n^c)$ | $(1 + \epsilon)$ -近似 | 0-1 背包   |
| 伪多项式算法 | $O(N^k n^c)$            | 可能为精确解               | 子集和问题    |